Contents

Машинное обучение с участием человека: недопонятое **1**
Структура книги . . . 1

Декларация об оригинальности **3**

Аннотация **5**

Благодарности **7**

Что такое машинное обучение с участием человека? **9**
Парадокс автоматизации . . . 9
Определение машинного обучения с участием человека . . . 9
Краткая история . . . 10
Зачем HITL? Обоснование ценности человеческого суждения . . . 11
Цикл обратной связи . . . 12
Что не является HITL . . . 13
Экономика человеческой обратной связи . . . 13
Обзор руководства . . . 13

Таксономия взаимодействия человека и машины **15**
Уровни автоматизации . . . 15
Активное и пассивное участие человека . . . 16
Один или несколько аннотаторов . . . 17
Унифицированная рамка . . . 17

Аннотирование и разметка данных **19**
Виды аннотирования . . . 19
Инструкции по аннотированию . . . 20
Измерение качества аннотирования: межаннотаторское согласие . . . 21
Работа с расхождениями . . . 22
Шум меток и его последствия . . . 23
Стоимость аннотирования и пропускная способность . . . 24

Активное обучение **25**
Постановка задачи активного обучения . . . 25
Теоретические основы . . . 25
Стратегии запросов . . . 26
Оценка неопределённости для глубоких моделей . . . 27
Полный цикл активного обучения . . . 29
Проблема холодного старта . . . 31
Активное обучение в режиме батчей . . . 31

Критерии остановки . . . 31
Когда активное обучение работает (и когда нет) . . . 32

Интерактивное машинное обучение **33**
Принципы интерактивного машинного обучения . . . 33
Сравнение с активным обучением . . . 33
Взаимодействие со смешанной инициативой . . . 34
Человеческие факторы в IML . . . 34
Типы обратной связи в IML на практике . . . 35
Пример из практики: Google Teachable Machine . . . 35
Реализация простого цикла IML . . . 36
Тест бабушки . . . 37
IML и фундаментальные модели . . . 38

Обучение с подкреплением на основе обратной связи от человека **41**
Проблема согласования . . . 41
Конвейер RLHF . . . 41
Упрощённая демонстрация RLHF . . . 43
Проблемы RLHF . . . 44
Конституционный ИИ (RLAIF) . . . 45

Обучение на основе демонстраций **47**
Клонирование поведения . . . 47
Обратное обучение с подкреплением . . . 48
GAIL: генеративное состязательное имитационное обучение . . . 49
Клонирование поведения в NLP: практический пример . . . 49
Сравнение методов имитационного обучения . . . 51
Приложения . . . 51

Обучение на основе сравнений и ранжирований **53**
Почему сравнения лучше оценок . . . 53
Модель Брэдли–Терри . . . 54
Модель Тёрстоуна . . . 55
Агрегирование ранжирований . . . 56
Дуэльные бандиты . . . 56
Обучение предпочтениям для языковых моделей . . . 56
Сбор высококачественных данных предпочтений . . . 57

HITL в обработке естественного языка **59**
Конститутивная проблема в аннотировании NLP . . . 59
Аннотирование текстовой классификации . . . 60
Распознавание именованных сущностей . . . 60
Извлечение отношений и семантическое аннотирование . . . 61
Постредактирование машинного перевода (MTPE) . . . 61
Диалоговый ИИ и аннотирование диалогов . . . 62
Программная разметка и слабый надзор . . . 62
RLHF для языковых моделей: реальность аннотирования . . . 63
Цикл обратной связи аннотирование–модель . . . 64
Оценка генеративных моделей NLP . . . 64

HITL в компьютерном зрении **67**
Как решения при аннотировании превращаются в предвзятости модели . . . 67
Аннотирование классификации изображений . . . 68

Обнаружение объектов: аннотирование ограничивающими рамками 68
Семантическая и инстанционная сегментация 70
Активное обучение для компьютерного зрения 70
Полуконтролируемое обучение с человеческим руководством 72
Аннотирование видео 72

HITL в здравоохранении и науке **75**
Анализ медицинских изображений 75
Аннотирование клинических NLP 77
Нормативные соображения 77
Аннотирование научных данных 78
Управление специалистами-аннотаторами 79
Конвейер HITL активного обучения для медицинской визуализации 80

Платформы и инструменты аннотирования **83**
Ландшафт платформ аннотирования 83
Сравнение функций платформ аннотирования 84
Аннотирование как код 85

Краудсорсинг и контроль качества **87**
Краудсорсинговые платформы 87
Проектирование задач для краудсорсинга 87
Статистические модели агрегирования меток 89
Избыточность и стратегия агрегирования 91

Оценка и метрики **93**
Метрики, ориентированные на модель 93
Метрики эффективности аннотирования 94
Человеческая оценка 95
Измерение согласования с намерениями человека 97
A/B-тестирование в развёрнутых системах 97

Справедливость, смещение и этика **99**
Источники предвзятостей в системах HITL 99
Справедливость в системах HITL 100
Благополучие аннотаторов 101
Конфиденциальность при аннотировании 101
Состязательное аннотирование и отравление данных 102
Институциональная этика 102

Перспективы развития **105**
Фундаментальные модели и изменение роли аннотирования 105
LLM как аннотаторы 106
Слабый надзор в масштабе 106
Непрерывное обучение с человеческим надзором 107
Мультимодальный HITL 107
Масштабируемый надзор 107
Эволюционирующее разделение труда 108
Открытые исследовательские проблемы 108

Библиография **111**

Пример из практики: Limen — человек в петле всего **113**

Проблема традиционного проектирования ОС 113
Архитектура на пороге 114
Проектирование взаимодействия 116
Limen как система HITL 116
Более глубокая мысль 117

Об уходе **119**

Машинное обучение с участием человека: недопонятое

Лазарос Туманидис

Настоящее руководство исследует одну из наиболее важных — и наименее понятых — областей современного искусственного интеллекта: интеграцию человека в цикл обучения. От аннотирования данных и активного обучения до обучения с подкреплением на основе обратной связи от человека (RLHF) и обучения на основе демонстраций — суждение человека остаётся незаменимым даже в эпоху, когда модели ИИ справляются почти с любыми задачами.

Книга предназначена для исследователей, инженеров и студентов, желающих понять, когда, почему и каким образом человек должен участвовать в цикле обучения.

:::{admonition} Русское издание :class: note Это русский перевод. Английский оригинал является основным справочным изданием. :::

Структура книги

Часть I — Основы — формирует концептуальную основу: что означает МО «с участием человека» и как классифицируются уровни автоматизации.

Часть II — Базовые методы — охватывает фундаментальные методы: аннотирование данных, активное обучение и интерактивное МО.

Часть III — Обучение на основе обратной связи от человека — охватывает RLHF, обучение на основе демонстраций и обучение на основе сравнений.

Часть IV — Приложения — рассматривает практические примеры в области ОЕЯ, компьютерного зрения и медицинских систем.

Часть V — Системы и практика — охватывает инструменты, платформы и метрики для систем с участием человека в цикле.

Часть VI — Этика и перспективы — рассматривает вопросы справедливости, смещения и будущего взаимодействия человека и алгоритма.

Декларация об оригинальности

Представлено в частичное невыполнение требований для присвоения степени доктора философии.

Настоящим заявляю, что работа, представленная в данном руководстве, является моей собственной — за исключением тех случаев, когда это не так.

Идеи, изложенные на этих страницах, были созданы при содействии: исследователей, публиковавших свои работы — иногда за платными барьерами; аннотаторов, чей труд сделал модели возможными, но чьи имена не появляются ни в одном разделе благодарностей; участников открытого программного обеспечения, написавших библиотеки, используемые в каждом примере кода; и накопленного текста человеческой цивилизации, поглощённого и сжатого большими языковыми моделями, у которых нет мнения ни о чём из этого.

Ошибки — мои. Прозрения возникли совместно.

Подтверждаю, что данная работа не представлялась для получения какой-либо иной учёной степени или профессиональной квалификации, академической премии, гранта, дела о постоянном профессорстве или публикации в LinkedIn, если иное не оговорено.

Также подтверждаю, что в данной работе отсутствует раздел сравнения с методами, находящимися на переднем крае науки (State of the Art). Читатель не найдёт здесь таблицы, ранжирующей 47 методов на 12 тестовых наборах, где предложенный метод достигает статистически значимых улучшений на 9 из них (†$p < 0{,}05$). Не потому, что такую таблицу нельзя было составить, — а потому, что её составление было бы не по существу.

Суть вот в чём: человек всегда был в петле. Всегда. Эта книга пытается понять, что это означает.

Подпись: Человек в петле

Дата: в момент вашего чтения

Учреждение: никакого. В этом и суть.

> **О подзаголовке** :class: note
>
> *Непонятое* относится к области знания, к роли человека в ней и к читателю — одновременно. Если вам кажется, что это касается только вас, — возможно, вы

правы. Если вам кажется, что это касается только машин, — вы доказываете правоту подзаголовка.

Аннотация

Машинное обучение с участием человека: непонятое

Господствующий нарратив машинного обучения отводит человеку роль временного участника — присутствующего на этапах сбора данных и обучения модели, а затем постепенно вытесняемого по мере совершенствования автоматизации. Данное руководство утверждает, что этот нарратив ошибочен, был ошибочным всегда, и что понимание *причин* его ошибочности является одним из наиболее практически ценных знаний для специалиста в области машинного обучения.

Машинное обучение с участием человека (HITL ML) — это дисциплина, которая делает вовлечённость человека в обучающиеся системы намеренной, структурированной и непрерывной. Вместо того чтобы рассматривать обратную связь от человека как издержки, которые нужно минимизировать, HITL трактует её как сигнал, требующий понимания: откуда он исходит, что кодирует, как собирать его эффективно и как встраивать в модели, поведение которых действительно соответствует намерениям их создателей и пользователей.

Данное руководство содержит исчерпывающее изложение области в шести частях. Часть I закладывает основы и предлагает таксономию взаимодействия человека и машины. Часть II охватывает классические техники: аннотирование и разметку данных, активное обучение и интерактивное машинное обучение. Часть III посвящена современным парадигмам согласования: обучению с подкреплением на основе обратной связи от человека (RLHF), обучению на демонстрациях и обучению на основе предпочтений. Часть IV применяет эти техники к обработке естественного языка, компьютерному зрению и здравоохранению. Часть V охватывает системы и инфраструктуру HITL в промышленном масштабе: платформы аннотирования, контроль качества краудсорсинга и методологию оценки. Часть VI рассматривает этику и перспективы развития области.

Каждая глава содержит формальные определения, математическое изложение там, где это уместно, и самодостаточные исполняемые примеры кода на Python.

Руководство предназначено для практиков, создающих системы машинного обучения, исследователей, изучающих их, студентов, осваивающих эту область, и всех, кто когда-либо задавался вопросом: кто именно разметил данные, на которых обучалась модель, принявшая решение, повлиявшее на их жизнь?

Ответ во всех случаях один: человек. Нередко — недооплачиваемый. Редко — упоминаемый. Всегда — в петле.

Ключевые слова: машинное обучение с участием человека, активное обучение, аннотирование данных, обучение с подкреплением на основе обратной связи от

человека, RLHF, обучение на основе предпочтений, краудсорсинг, интерактивное машинное обучение, согласование ИИ, ответственный ИИ

Классификация области: Машинное обучение · Взаимодействие человека и компьютера · Искусственный интеллект · Наука о данных

Объём: Достаточный.

Благодарности

Каждая книга о машинном обучении с участием человека должна начинаться с людей в собственной петле.

Аннотаторы. Это руководство посвящено в том числе вам — вашему труду, вашим суждениям и тому факту, что ваше имя не фигурирует ни в одной карточке датасета, ни в одном разделе благодарностей в статьях, ни в одном пресс-релизе о моделях, которые ваша работа сделала возможными. Область начинает осознавать это. Ей ещё далеко до конца пути.

Участники открытого программного обеспечения, чьи библиотеки используются в каждом примере кода: сообщество NumPy (с 2006 года), разработчики scikit-learn (с 2007 года), команда PyTorch, сообщество Hugging Face и бесчисленные сопровождающие, исправляющие ошибки по выходным — без вознаграждения и признания. Научные вычисления построены на этом невидимом труде.

Исследователи, публиковавшие свои работы в открытом доступе — в препринтах на arXiv, на открытых платформах и в репозиториях GitHub, — давая практикам возможность учиться без институционального доступа. И исследователи, рецензировавшие статьи тщательно и честно, — форма экспертного аннотирования, которую академия каким-то образом убедила себя считать не трудом.

Учителя, делавшие знания доступными: авторы блогов, создатели учебных материалов, отвечавшие на Stack Overflow, докладчики конференций, выкладывавшие свои слайды. Обучение не происходит в изоляции и не происходит без людей, готовых объяснять незнакомцам.

Критики. Исследователи, указывавшие на то, что датасеты кодируют предубеждения, что демографические характеристики аннотаторов определяют поведение моделей, что люди в петле нередко являются наиболее уязвимыми участниками системы. Эта область стала лучше благодаря вашей настойчивости в постановке неудобных вопросов.

Есть известные известные — то, что мы знаем, что знаем. Есть известные неизвестные — то, что мы знаем, что не знаем. Есть неизвестные неизвестные — то, чего мы не знаем, что не знаем.

И есть то, что все в области знают, но что так и не попадает в статьи.

Это руководство представляет собой в том числе попытку изложить последнее.

Если вы внесли вклад в эту область и не видите здесь своего имени: вы упомянуты. Петля включает вас.

Что такое машинное обучение с участием человека?

Машина, способная учиться на опыте… но только если снабдить её правильным опытом. – перефразируя Алана Тьюринга

Парадокс автоматизации

Каждая новая волна автоматизации порождает новые требования к вниманию человека. Когда авиакомпании внедрили автопилот, пилоты превратились в контролёров — ответственных не за управление в каждый момент времени, а за более сложную задачу: знать, *когда* нужно взять управление на себя. Когда супермаркеты ввели кассы самообслуживания, выяснилось, что эти системы требуют более пристального надзора на каждую транзакцию, чем традиционные кассиры. И когда машинное обучение начало принимать решения в масштабе — в медицине, финансах, найме персонала и модерации контента — оно породило огромный, непрекращающийся спрос на человеческое суждение.

Это **парадокс автоматизации** {cite}`bainbridge1983ironies`: чем более совершенной становится автоматизированная система, тем более серьёзными оказываются последствия её сбоев, и тем необходимее становится надёжный человеческий надзор. Машинное обучение — не исключение.

Машинное обучение с участием человека (HITL ML) — это область, которая принимает данный парадокс всерьёз и закладывает его в основу проектирования систем с самого начала. Вместо того чтобы рассматривать участие человека как временные строительные леса, подлежащие последующему демонтажу, HITL ML трактует взаимодействие человека и машины как *характеристику* системы — источник сигналов, коррекций и руководства, позволяющий сделать модели более точными, более согласованными с человеческими ценностями и более надёжными.

Определение машинного обучения с участием человека

Единого общепринятого определения HITL ML не существует, и этот термин используется в нескольких взаимосвязанных значениях в литературе. В данном руководстве принимается широкое, но точное определение:

:::{admonition} Определение :class: important

Машинное обучение с участием человека — это любая система или методология машинного обучения, в которой обратная связь от человека встраивается в процесс

обучения *намеренным, структурированным и непрерывным* образом — не только в момент создания датасета, но и на протяжении обучения, оценки и развёртывания.

:::

Это определение содержит три ключевых пункта:

Намеренность. HITL — это не случайное человеческое влияние (например, тот факт, что обучающие данные изначально были созданы людьми). Речь идёт о системах, явно спроектированных для сбора, встраивания и использования обратной связи от человека.

Структурированность. Обратная связь имеет определённую форму — метка, исправление, суждение о предпочтении, демонстрация — и определённую роль в алгоритме обучения.

Непрерывность. Цикл обратной связи продолжается во времени, позволяя системе совершенствоваться по мере встречи с новыми ситуациями, совершения ошибок и получения человеческого руководства.

Это определение охватывает классические аннотационные конвейеры, активное обучение, интерактивное МО, RLHF и обучение на демонстрациях. Оно исключает пассивный сбор данных и чисто автономное обучение с учителем (хотя граница размыта, как будет обсуждаться в главе 2).

Краткая история

Экспертные системы и инженерия знаний (1960-е — 1980-е)

Самые ранние системы искусственного интеллекта были почти полностью основаны на участии человека: инженеры по знаниям месяцами работали с экспертами предметной области, кропотливо кодируя правила в экспертных системах вроде MYCIN и DENDRAL. Каждый фрагмент знания был явно предоставлен человеком. Машина выступала исполнителем; человек — оракулом.

Эти системы работали удивительно хорошо в узких областях, однако были хрупкими — неспособными обобщаться за пределы вручную созданных правил и дорогостоящими в обслуживании.

Статистический поворот (1980-е — 2000-е)

Переход к статистическому машинному обучению в 1980-х и 1990-х годах изменил характер участия человека. Вместо кодирования знаний в виде правил люди теперь предоставляли *примеры* — размеченные датасеты, позволявшие моделям выводить закономерности. Роль человека превратилась в роль аннотатора: разметку документов, тегирование изображений, транскрибирование речи.

Это стало важным шагом вперёд, но породило новое узкое место: **размеченные данные стоят дорого**. Исследователи начали спрашивать, как наиболее эффективно использовать усилия по разметке. Этот вопрос дал начало **активному обучению**, формализованному в начале 1990-х годов {cite}`cohn1994improving`.

Эра глубокого обучения (2010-е — настоящее время)

Революция глубокого обучения создала новый режим: модели с миллиардами параметров, способные обучаться экстремально сложным функциям из данных — но требующие соразмерно огромных размеченных датасетов. ImageNet (14 миллионов размеченных изображений) и последующие крупномасштабные проекты аннотирования продемонстрировали как мощь, так и цену масштаба.

Одновременно развёртывание МО в масштабе обнажило новые проблемы: модели, точные в среднем, но систематически ошибающиеся для определённых групп; модели, уверенно «галлюцинирующие» факты; модели, оптимизирующие измеримые прокси вместо человеческих ценностей. Эти сбои мотивировали новые формы участия человека: не просто разметку, но *согласование* — проект создания моделей, ведущих себя так, как люди действительно хотят.

Наиболее чётким выражением этой работы по согласованию, ориентированной на HITL, является **обучение с подкреплением на основе обратной связи от человека (RLHF)** {cite}`christiano2017deep`, ставшее основой систем вроде InstructGPT {cite}`ouyang2022training` и способностей следовать инструкциям у современных языковых моделей.

Зачем HITL? Обоснование ценности человеческого суждения

Что делает человеческое суждение достаточно ценным для встраивания в системы машинного обучения? Несколько свойств:

1. Здравый смысл и мировые знания

Люди привносят обширный контекстуальный опыт в любую задачу. Когда радиолог размечает рентгеновский снимок, она опирается на годы подготовки, понимание анатомии и имплицитные знания о том, как выглядят заболевания — знания, которые чрезвычайно трудно полностью формализовать или получить только из данных.

2. Семантическая укоренённость

Метки имеют смысл, потому что люди понимают, что они означают. Класс «кошка» в ImageNet отсылает к размытому концепту, который люди узнают интуитивно, но который ни одно формальное определение полностью не охватывает. Модели обучаются экстенсионалу метки (какие изображения ей соответствуют), но могут не научиться самому концепту, что приводит к сбоям на пограничных случаях, с которыми любой человек справился бы правильно.

3. Согласование ценностей

Люди обладают предпочтениями, ценностями и этическими суждениями, которые модели МО не могут вывести только из данных. Является ли фрагмент текста «полезным» или «вредным» — это не чисто эмпирический вопрос: он зависит от человеческих ценностей, варьирующихся у разных людей и в разных контекстах. HITL является основным механизмом, посредством которого эти ценности могут быть переданы системам МО.

4. Адаптивность

Человеческое суждение способно адаптироваться к новым ситуациям без переобучения. Модель, обученная на исторических данных, может катастрофически сбоить при изменении мира; человек способен распознать новизну и отреагировать соответствующим образом.

5. Подотчётность

В высокорисковых областях — медицине, праве, финансах — решения должны быть подотчётны людям. Системы HITL обеспечивают эту подотчётность, удерживая людей в позиции, позволяющей понимать, проверять и переопределять поведение машины.

Цикл обратной связи

Центральной структурой HITL ML является цикл обратной связи между моделью и одним или несколькими людьми:

```
+-----------------------------------------------+
|                                               |
|   Модель делает предсказания / запросы        |
|   ----------------------------------->        |
|                                   Человек     |
|   Человек предоставляет обратную  <-------- |
|   связь                                       |
|   ----------------------------------          |
|                                               |
|   Модель обновляется на основе                |
|   обратной связи                              |
|                                               |
+-----------------------------------------------+
```

Характер обратной связи существенно различается в разных парадигмах HITL:

Тип обратной связи	Пример	Основная парадигма
Метка класса	«Это письмо является спамом»	Обучение с учителем
Исправление	«Сущность должна быть ORG, а не PER»	Активное / интерактивное МО
Предпочтение	«Ответ A лучше, чем B»	RLHF / ранжирование
Демонстрация	Показать роботу, как захватить предмет	Имитационное обучение
Естественный язык	«Давай давай более краткие ответы»	Настройка на инструкции
Неявный сигнал	Пользователь нажал / не нажал	Неявная обратная связь

Что не является HITL

Важно чётко обозначить, что выходит *за пределы* нашего определения.

HITL — это не то же самое, что «человек над петлёй» при развёртывании (иногда называемый «human on the loop»), когда люди контролируют автоматизированные решения и могут их отменять, но не отправляют коррекции обратно в обучение. Это различие будет обсуждаться в главе 2.

HITL — это не слабый надзор сам по себе. Программные системы разметки вроде Snorkel используют функции разметки (часто написанные людьми правила) для генерации зашумлённых меток в масштабе. Это форма структурированного человеческого ввода, но обратная связь не является итеративной в том смысле, который обычно подразумевает HITL.

HITL — это не просто использование размеченных данных. Каждая модель обучения с учителем использует данные, размеченные людьми. HITL относится именно к системам, где обратная связь от человека является *активной, итеративной* частью процесса обучения.

Экономика человеческой обратной связи

Человеческая обратная связь ценна, но дорогостояща. Аннотирование медицинских изображений может стоить от нескольких десятков до нескольких сотен долларов за изображение при выполнении специалистом — в зависимости от подспециальности и сложности задачи {cite}`monarch2021human`. Метки краудворкеров на платформах вроде Amazon Mechanical Turk могут стоить $0,01–$0,10 за элемент {cite}`hara2018data` при значительно более низком качестве. Фундаментальная задача HITL ML — **максимизация ценности каждой единицы человеческой обратной связи**.

Это приводит к объединяющему вопросу, проходящему через большую часть данного руководства:

:::{admonition} Центральный вопрос HITL ML :class: tip

При фиксированном бюджете человеческого времени и внимания, как следует решать, что спрашивать у людей, когда спрашивать и как встраивать ответы в обучение модели?

:::

Ответ на этот вопрос зависит от области, формы обратной связи, стоимости аннотирования, риска ошибки и текущего состояния модели — именно поэтому HITL ML является богатой и продолжающей развиваться дисциплиной.

Обзор руководства

Остальная часть данного руководства организована следующим образом. **Часть II** охватывает три классических столпа HITL: аннотирование, активное обучение и интерактивное МО. **Часть III** посвящена более новым парадигмам обучения на основе обратной связи — RLHF, имитационному обучению и обучению на предпочтениях, — ставшим центральными для современного ИИ. **Часть IV** рассматривает HITL через

призму конкретных прикладных областей. **Часть V** предлагает взгляд практика на платформы, краудсорсинг и оценку. **Часть VI** посвящена этике и взгляду в будущее.

> **See also:** Обзор области, ориентированный на практиков, см. в {cite}`monarch2021human`. Основополагающую статью об активном обучении, заложившую формальную базу HITL, см. в {cite}`settles2009active`.

Таксономия взаимодействия человека и машины

Точный словарный запас — необходимое условие ясного мышления. Термин «человек в петле» в практике используется расплывчато — иногда для обозначения ситуации, когда человек размечает обучающие данные, иногда — когда человек может отменить решение модели, а иногда — когда человек активно направляет процесс обучения в реальном времени. Это содержательно различающиеся вещи.

Данная глава отображает ландшафт взаимодействия человека и машины в МО, предоставляя концептуальный словарь, используемый в остальной части руководства.

Уровни автоматизации

Наиболее фундаментальное различие касается того, насколько активно человек участвует в принятии решений системой. Известная рамка из теории автоматизации {cite}`sheridan1992telerobotics` выделяет десять уровней, однако для целей МО три категории охватывают важные вариации:

Человек в петле (HITL)

Человек является *активным участником процесса обучения*. Решения — или по меньшей мере значимые решения — требуют участия человека, прежде чем они будут приняты. Система не может функционировать без постоянного участия человека.

Примеры: Система активного обучения, запрашивающая мнение клинициста, прежде чем добавить случай в обучающие данные. Аннотатор, размечающий примеры, немедленно используемые для обновления модели. Разметчик RLHF, сравнивающий выходы модели.

Человек над петлёй (HOTL)

Система работает автономно, но человек её контролирует и может вмешаться. Человек выступает *супервизором*, а не участником. Обратная связь может поступать или не поступать обратно в обучение.

Примеры: Система модерации контента, автоматически помечающая публикации; рецензент выборочно проверяет и исправляет часть решений. Автопилот, управляющий самолётом, но оповещающий пилота об аномалиях.

Человек в командовании (HIC)

Человек принимает все решения; система предоставляет *рекомендации или информацию*, но не обладает автономией. Это наиболее слабая форма развёртывания МО.

Примеры: Система поддержки принятия диагностических решений, показывающая врачу вероятностную оценку модели, но оставляющая окончательное решение целиком за врачом.

> **Какой уровень является правильным?** :class: tip
>
> Подходящий уровень автоматизации зависит от стоимости ошибок, надёжности модели, квалификации доступных людей и ограничений по времени отклика задачи. Эти факторы меняются по мере зрелости модели — большинство систем начинают как HITL и мигрируют к HOTL по мере роста уверенности.

```
Сырые данные --> Предобработка --> Признаки --> Обучение --> Оценка --> Развёртывание
     ^               ^                              ^            ^             ^
     |               |                              |            |             |
  Сбор            Аннотирование                Активное      Тестовая     Мониторинг
  обратной        и разметка                   обучение      обратная     и коррекция
  связи                                                   связь
```

Этап	Роль человека	Глава
Сбор	Решение, какие данные собирать; стратегия выборки	3, 4
Аннотирование	Присвоение меток, структур, метаданных	3, 13
Обучение	Запросы активного обучения; онлайн-обратная связь	4, 5, 6
Оценка	Человеческая оценка выходов модели	14
Развёртывание	Мониторинг, обработка исключений, исправления	12, 14

Активное и пассивное участие человека

При *активном* HITL система выбирает, какие точки данных представить человеку, задавая вопросы стратегически. При *пассивном* HITL человек предоставляет обратную связь на те данные, которые поступают сами по себе (например, батчи разметки, назначенные последовательно).

Активное участие эффективнее, потому что обратная связь направляется туда, где она даст наибольшее улучшение модели. Пассивное участие проще реализовать и управлять им.

Связанное различие — между **батчевой** и **онлайн**-обратной связью:

- **Батчевая:** Люди размечают большой пул примеров; модель переобучается. Повторить.
- **Онлайн (потоковая):** Обратная связь от человека поступает непрерывно; модель обновляется инкрементально.

Батчевые рабочие процессы являются нормой в промышленности (проекты аннотирования, за которыми следуют циклы обучения). Онлайн-рабочие процессы более естественны для интерактивных систем и систем обучения с подкреплением.

Один или несколько аннотаторов

Большинство формальных представлений HITL предполагают одного последовательного аннотатора. На практике аннотирование включает множество людей, и их суждения расходятся.

Агрегирование объединяет несколько аннотаций в единую метку, как правило, путём голосования большинством или с помощью статистической модели (глава 13).

Несогласие как сигнал — ряд исследователей утверждает, что расхождение аннотаторов является ценной информацией, которую не следует сворачивать в единую «золотую» метку. Перспективистские подходы в NLP, например, сохраняют несколько аннотаций в виде мягких меток, отражающих реальную неоднозначность данных {cite}`uma2021learning`.

Унифицированная рамка

Любую конфигурацию HITL можно представить пятикомпонентным кортежем:

$$\text{конфигурация HITL} = (\text{уровень}, \text{модальность}, \text{этап}, \text{частота}, \text{аннотаторы})$$

где:

- **уровень** $\in$ {HITL, HOTL, HIC}
- **модальность** $\in$ {метка, исправление, демонстрация, предпочтение, ЕЯ, неявная}
- **этап** $\in$ {сбор, аннотирование, обучение, оценка, развёртывание}
- **частота** $\in$ {батч, онлайн}
- **аннотаторы** $\in \mathbb{N}^+$ (количество аннотаторов на элемент)

Эта таксономия позволяет сравнивать разнообразные системы HITL по одним и тем же осям и рассуждать о компромиссах между ними. Остальная часть руководства детально рассматривает конкретные ячейки этого пространства.

> **See also:** Эмпирическое исследование того, как решения при аннотировании влияют на поведение модели, см. в {cite}`bender2021stochastic`. Обзор интерактивных систем МО см. в {cite}`amershi2014power`.

Аннотирование и разметка данных

Аннотирование данных — наиболее распространённая форма участия человека в МО. Прежде чем модель сможет обучаться, кто-то должен сообщить ей правильные ответы — и этот кто-то, как правило, человек. В данной главе рассматриваются теория и практика аннотирования: что делает его трудным, как проектировать задачи аннотирования, как измерять качество и как работать с расхождениями.

Виды аннотирования

Задачи аннотирования чрезвычайно разнообразны по структуре, сложности и стоимости. Основные виды включают:

Классификация

Аннотатор относит каждый экземпляр к одной из K заранее определённых категорий. Это когнитивно наиболее простая задача аннотирования, однако разработка хорошей схемы категорий (*таксономии*) может быть неожиданно сложной.

Бинарная классификация (является ли это изображение кошкой?) — наиболее простой случай. **Многоклассовая** (какой вид это животное?) требует выбора одного варианта из списка. **Многометочное** аннотирование (какие темы охватывает эта статья?) допускает несколько одновременных меток.

Разметка последовательностей

Каждый токен в последовательности получает метку. Распознавание именованных сущностей (NER) — канонический пример: аннотаторы отмечают участки текста как PERSON, ORGANIZATION, LOCATION и т. д. Аннотирование, как правило, выполняется с использованием схемы тегирования BIO (Beginning-Inside-Outside) или BIOES:

```
B-ORG    O          B-ORG    O      O      O
```

Аннотирование промежутков и отношений

Помимо разметки отдельных токенов, аннотаторам может потребоваться: - Выявлять промежутки (многотокенные выражения) и присваивать им типы - Отмечать *отношения* между промежутками («Apple» ПРИОБРЕЛА «Shazam») - Аннотировать кореферентные цепочки (все упоминания одной и той же сущности)

Эти задачи когнитивно требовательны и характеризуются более низким межаннотаторским согласием.

Ограничивающие рамки и обнаружение объектов

Аннотаторы рисуют прямоугольники вокруг объектов на изображениях и присваивают каждому прямоугольнику метку категории. Важна точность локализации: слишком маленький прямоугольник упускает контекст; слишком большой включает фоновые элементы. Современные инструменты аннотирования вычисляют IoU (пересечение над объединением) с эталонными аннотациями для выявления проблем качества.

Сегментация

Аннотирование на уровне пикселей: каждый пиксель относится к классу (семантическая сегментация) или к конкретному экземпляру объекта (сегментация экземпляров). Высококачественная сегментация — один из самых дорогостоящих видов аннотирования: стоимость может варьироваться от нескольких десятков до более ста долларов за изображение для сложных сцен в зависимости от области и инструментальной поддержки.

Транскрипция и перевод

Аудио текст (данные для обучения ASR), рукопись текст (данные для OCR) или исходный язык целевой язык (данные для MT). Эти задачи требуют лингвистической экспертизы и не могут быть надёжно выполнены необученными аннотаторами.

Инструкции по аннотированию

Наиболее важным фактором, определяющим качество аннотирования, является качество **инструкций по аннотированию** — письменных указаний, которым следуют аннотаторы.

Хорошие инструкции: - Формулируют цель задачи и объясняют *почему* метка важна - Дают чёткое определение каждой категории с положительными и отрицательными примерами - Явно рассматривают типичные пограничные и трудные случаи - Определяют, что делать при неуверенности (например, пометить «пропустить» или сделать вынужденный выбор) - Включают проработанные примеры полного аннотирования

Плохие инструкции полагаются на то, что аннотаторы «применят здравый смысл» к пограничным случаям, — что приводит к непоследовательным решениям, снижающим качество модели и раздувающим межаннотаторское несогласие.

> **Разработка инструкций — итеративный процесс** :class: note
>
> Не рассчитывайте написать идеальные инструкции с первого раза. Проведите небольшой пилотный раунд аннотирования, проанализируйте расхождения и обновите инструкции. Повторяйте. Хорошо разработанные инструкции, как правило, проходят 3–5 итераций правок, прежде чем стабилизируются.

Измерение качества аннотирования: межаннотаторское согласие

Когда несколько аннотаторов размечают одни и те же данные, их согласие поддаётся измерению. Высокое согласие свидетельствует о том, что задача хорошо определена и аннотаторы её поняли. Низкое согласие указывает на неоднозначность в задаче, инструкциях или самих данных.

Каппа Коэна

Для двух аннотаторов, размечающих данные на K категорий, **каппа Коэна** {cite}`cohen1960coefficient` корректирует наблюдаемое согласие с учётом случайного:

$$\kappa = \frac{P_o - P_e}{1 - P_e}$$

где P_o — наблюдаемая доля согласия, а P_e — вероятность согласия по случайности (вычисленная из маргинальных распределений меток).

$\kappa = 1$ означает полное согласие; $\kappa = 0$ — согласие не лучше случайного; $\kappa < 0$ — систематическое несогласие.

Диапазон κ	Интерпретация
< 0	Хуже случайного
$0{,}0 - 0{,}20$	Незначительное
$0{,}21 - 0{,}40$	Слабое
$0{,}41 - 0{,}60$	Умеренное
$0{,}61 - 0{,}80$	Существенное
$0{,}81 - 1{,}00$	Почти идеальное

Каппа Флейса

Расширяет каппу Коэна на $M > 2$ аннотаторов. Каждый аннотатор независимо размечает каждый элемент; формула агрегирует по всем аннотаторам:

$$\kappa_F = \frac{\bar{P} - \bar{P}_e}{1 - \bar{P}_e}$$

где $\bar{P}$ — среднее попарное согласие по всем парам аннотаторов, а $\bar{P}_e$ — ожидаемое согласие при случайном распределении.

Альфа Krippendorffa

Наиболее общая метрика, поддерживающая любое количество аннотаторов, любой тип шкалы (номинальная, порядковая, интервальная, относительная) и пропущенные данные {cite}`krippendorff2011computing`:

$$\alpha = 1 - \frac{D_o}{D_e}$$

где D_o — наблюдаемое несогласие, а D_e — ожидаемое несогласие. Альфа Kpippendorffa, как правило, предпочтительна в академических работах благодаря своей гибкости.

```
import numpy as np
from sklearn.metrics import cohen_kappa_score

# Simulate two annotators labeling 200 items into 3 categories
rng = np.random.default_rng(0)
true_labels = rng.integers(0, 3, size=200)

# Annotator 1: mostly agrees with ground truth
ann1 = true_labels.copy()
flip_mask = rng.random(200) < 0.15
ann1[flip_mask] = rng.integers(0, 3, size=flip_mask.sum())

# Annotator 2: less consistent
ann2 = true_labels.copy()
flip_mask2 = rng.random(200) < 0.30
ann2[flip_mask2] = rng.integers(0, 3, size=flip_mask2.sum())

kappa_12 = cohen_kappa_score(ann1, ann2)
kappa_1true = cohen_kappa_score(ann1, true_labels)
kappa_2true = cohen_kappa_score(ann2, true_labels)

print(f"Cohen's κ (ann1 vs ann2):   {kappa_12:.3f}")
print(f"Cohen's κ (ann1 vs truth):  {kappa_1true:.3f}")
print(f"Cohen's κ (ann2 vs truth):  {kappa_2true:.3f}")
```

Работа с расхождениями

Когда аннотаторы расходятся, существует несколько стратегий:

Голосование большинства

В качестве золотого стандарта принимается наиболее распространённая метка. Прост и надёжен, когда количество аннотаторов на элемент нечётное. Даёт сбой, когда меньшинство аннотаторов систематически точнее.

Взвешенное голосование

Аннотаторам присваиваются веса в соответствии с их оценочной точностью (полученной из согласия с золотым стандартом или другими аннотаторами). Более точные аннотаторы оказывают большее влияние.

Мягкие метки

Вместо сворачивания аннотаций в единую метку сохраняется распределение. Если 3 из 5 аннотаторов сказали «позитивный», а 2 — «нейтральный», это представляется как $(p_{\text{pos}}, p_{\text{neutral}}, p_{\text{neg}}) = (0{,}6, 0{,}4, 0{,}0)$. Обучение на мягких метках улучшает калиброванность.

Арбитраж

Старший аннотатор или эксперт в предметной области разрешает расхождения. Является золотым стандартом, но дорогостоящим; как правило, зарезервирован для высокорисковых областей.

Статистические модели

Более сложные подходы моделируют компетентность аннотаторов вероятностно. Модель **Dawid–Skene** {cite}`dawid1979maximum` одновременно оценивает матрицы ошибок аннотаторов и истинные метки элементов с помощью EM-алгоритма. Подробности — в главе 13.

Шум меток и его последствия

Реальное аннотирование зашумлено. Последствия шума меток для обучения модели зависят от типа шума:

- **Случайный шум** (метки случайно перевёрнуты) ухудшает производительность, однако модели удивительно устойчивы к умеренным уровням (до ~20% для многих задач).
- **Систематический/адверсарный шум** (метки последовательно неверны в определённых паттернах) значительно более разрушителен и труднее поддаётся обнаружению.
- **Классово-обусловленный шум** (ошибки более вероятны для определённых классов) смещает граничное решение модели.

Практическое эмпирическое правило: при n обучающих примерах и доле ϵ испорченных меток производительность модели деградирует примерно так, как если бы у вас было $(1-2\epsilon)^2 n$ чистых примеров {cite}`natarajan2013learning`. При $\epsilon = 0{,}2$ это эквивалентно потере 36% данных.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score

rng = np.random.default_rng(42)
X, y = make_classification(n_samples=2000, n_features=20, random_state=42)

noise_levels = np.linspace(0, 0.45, 15)
mean_accs = []

for eps in noise_levels:
    y_noisy = y.copy()
    flip = rng.random(len(y)) < eps
    y_noisy[flip] = 1 - y_noisy[flip]
    scores = cross_val_score(LogisticRegression(max_iter=500), X, y_noisy, cv=5)
    mean_accs.append(scores.mean())

plt.figure(figsize=(7, 4))
```

```
plt.plot(noise_levels, mean_accs, 'o-', color='#2b3a8f', linewidth=2)
plt.xlabel("Label noise rate (ε)", fontsize=12)
plt.ylabel("Cross-validated accuracy", fontsize=12)
plt.title("Effect of Label Noise on Model Performance", fontsize=13)
plt.axvline(0.2, color='#e05c5c', linestyle='--', alpha=0.7, label='20% noise')
plt.legend()
plt.tight_layout()
plt.savefig('label_noise_effect.png', dpi=150)
plt.show()
print(f"\nAccuracy at 0% noise:  {mean_accs[0]:.3f}")
print(f"Accuracy at 20% noise: {mean_accs[round(0.2 / 0.45 * 14)]:.3f}")
print(f"Accuracy at 40% noise: {mean_accs[-2]:.3f}")
```

Стоимость аннотирования и пропускная способность

Понимание экономики аннотирования необходимо для планирования проекта.

Тип задачи	Типичная пропускная способность	Стоимость за элемент (специалист)
Бинарная классификация изобр.	200–500/час	$0,02–0,10
NER (короткий текст)	50–150 элементов/час	$0,10–0,50
Извлечение отношений	20–60 элементов/час	$0,30–1,50
Сегментация мед. изображений	5–30 элементов/час	$10–100
Видеоаннотирование	5–20 мин видео/час	$20–200

Эти цифры являются приблизительными оценками порядка величины и существенно варьируются в зависимости от требуемой предметной экспертизы, качества инструментов аннотирования, ясности инструкций и опыта аннотаторов. Они должны восприниматься как иллюстративные, а не нормативные.

> **See also:** Варианты инструментов аннотирования рассмотрены в главе 12. Статистические модели для краудсорсингового аннотирования (Dawid–Skene, MACE) рассмотрены в главе 13.

Активное обучение

Размеченные данные стоят дорого. Ключевая идея активного обучения состоит в том, что *не все неразмеченные примеры одинаково информативны* — модель может улучшаться быстрее, если ей позволено выбирать, о чём спрашивать. Вместо случайной разметки данных система активного обучения запрашивает у оракула (как правило, аннотатора-человека) примеры, наиболее вероятно улучшающие модель.

В данной главе рассматриваются теория и практика активного обучения: стратегии запросов, рамки выборки, критерии остановки и практические соображения для реального развёртывания.

Постановка задачи активного обучения

Стандартная постановка **активного обучения на основе пула** включает:

- **Размеченное множество** $\mathcal{L} = \{(x_i, y_i)\}_{i=1}^{n}$ — изначально небольшое
- **Неразмеченный пул** $\mathcal{U} = \{x_j\}_{j=1}^{m}$ — как правило, значительно больше $\mathcal{L}$
- **Оракул** $\mathcal{O}$, способный возвратить $y = \mathcal{O}(x)$ для любого запрошенного x
- **Стратегию запросов** ϕ, выбирающую следующий запрос $x^* = \phi(\mathcal{L}, \mathcal{U}, f_\theta)$

Цикл активного обучения:

```
1. Инициализация: L = небольшое размеченное начальное множество, U = неразмеченный пул
2. Обучение: f_θ ← train(L)
3. Запрос: x* = argmax φ(x; f_θ) по x ∈ U
4. Разметка: y* = O(x*)
5. Обновление: L ← L ∪ {(x*, y*)}, U ← U \ {x*}
→ Повторять с шага 2 до исчерпания бюджета
```

Цель — достичь целевого качества модели, используя как можно меньше запросов к оракулу.

Теоретические основы

Естественный вопрос: насколько активное обучение может помочь? В лучшем случае активное обучение способно обеспечить *экспоненциальное* уменьшение сложности разметки — достигать ошибки ϵ с $O(\log(1/\epsilon))$ метками вместо $O(1/\epsilon)$, требуемых пассивным обучением, по крайней мере в реализуемых постановках с хорошей стратегией запросов {cite}`settles2009active`.

На практике гарантии труднее получить. **Агностическое активное обучение** {cite}`balcan2006agnostic` показывает, что экономия меток возможна даже тогда, когда целевой концепт не входит в класс гипотез, однако экономия сильно зависит от коэффициента несогласия — меры того, насколько быстро сжимается множество правдоподобных гипотез по мере накопления данных.

Ключевое практическое следствие: преимущество активного обучения наибольшее, когда **граница решения проста и сосредоточена** (так что запросы неопределённости быстро исключают неверные гипотезы), и наименьшее, когда класс гипотез велик или граница сложна.

Стратегии запросов

Выборка по неопределённости

Простейшая и наиболее широко используемая стратегия: запрашивать пример, по которому модель наиболее *неопределена* {cite}`lewis1994sequential`.

Наименьшая уверенность запрашивает пример, в отношении которого модель наименее уверена в своём лучшем предсказании:

$$x^* = \arg\max_{x \in \mathcal{U}} \left(1 - P_\theta(\hat{y} \mid x)\right)$$

Выборка по отступу рассматривает разрыв между двумя наибольшими предсказанными вероятностями:

$$x^* = \arg\min_{x \in \mathcal{U}} \left(P_\theta(\hat{y}_1 \mid x) - P_\theta(\hat{y}_2 \mid x)\right)$$

Выборка по энтропии использует полное предсказанное распределение:

$$x^* = \arg\max_{x \in \mathcal{U}} \left(-\sum_{k=1}^{K} P_\theta(y_k \mid x) \log P_\theta(y_k \mid x) \right)$$

Выборка по энтропии является наиболее обоснованной из трёх — она учитывает все классы — и в целом превосходит остальные на многоклассовых задачах.

Запрос комитетом (QbC)

Обучается **комитет** из C моделей (с использованием бэггинга, различных инициализаций или разных архитектур). Запрашивается пример, по которому комитет расходится сильнее всего:

$$x^* = \arg\max_{x \in \mathcal{U}} \ \mathrm{disagreement}(\{f_c(x)\}_{c=1}^{C})$$

Несогласие может измеряться как **голосовая энтропия** (энтропия по результатам голосования большинства в комитете) или **дивергенция Кульбака–Лейблера** от консенсусного распределения.

QbC даёт более качественные оценки неопределённости, чем одна модель, однако требует обучения нескольких моделей, что вычислительно дорого.

Ожидаемое изменение модели

Запрашивается пример, который, будучи размеченным, вызвал бы наибольшее изменение в текущей модели. Для основанных на градиентах моделей это соответствует примеру с наибольшей ожидаемой нормой градиента {cite}`settles2008analysis`:

$$x^* = \arg\max_{x \in \mathcal{U}} \sum_{y \in \mathcal{Y}} P_\theta(y \mid x) \, \|\nabla_\theta \mathcal{L}(f_\theta(x), y)\|$$

Эта стратегия имеет сильное теоретическое обоснование, однако требует вычисления градиентов для каждого кандидата, что делает её дорогостоящей для больших моделей.

Ключевые множества / геометрические подходы

Стратегии на основе неопределённости могут быть **смещены в сторону выбросов**: необычный пример может быть весьма неопределённым, но не репрезентативным для распределения данных. Методы ключевых множеств решают эту проблему, стремясь к разнообразной выборке, охватывающей пространство признаков.

Жадный алгоритм k-центров {cite}`sener2018active` находит наименьшее множество точек, такое что каждая неразмеченная точка находится в пределах δ от хотя бы одной запрошенной точки:

$$x^* = \arg\max_{x \in \mathcal{U}} \min_{x' \in \mathcal{L}} d(x, x')$$

то есть запрашивается точка, наиболее удалённая от любой уже размеченной точки. Это стимулирует хорошо распределённый набор аннотаций.

BADGE

Batch Active learning by Diverse Gradient Embeddings (BADGE) {cite}`ash2020deep` сочетает неопределённость и разнообразие: отбирается батч примеров, чьи градиентные вложения (по отношению к предсказанной метке) одновременно велики по норме (неопределённые) и разнообразны (охватывают разные регионы пространства градиентов). Это одна из наиболее конкурентоспособных современных стратегий.

Оценка неопределённости для глубоких моделей

Описанные выше стратегии предполагают доступ к откалиброванным вероятностным выходам модели. Для простых моделей (логистическая регрессия, softmax-классификаторы) это очевидно. Для глубоких сетей получение надёжных оценок неопределённости требует дополнительных техник.

Два вида неопределённости

По Кендаллу и Галу {cite}`kendall2017uncertainties`, различают:

Алеаторная неопределённость (неопределённость данных): присущий наблюдениям шум, который нельзя уменьшить путём сбора дополнительных данных. Размытое изображение алеаторно неопределённо — никакое количество дополнительных обучающих данных из того же распределения не сделает модель более уверенной в нём.

Эпистемическая неопределённость (неопределённость модели): неопределённость из-за ограниченности обучающих данных или модели, не видевшей похожих примеров. Эпистемическую неопределённость *можно* уменьшить путём разметки дополнительных данных — и именно она является релевантной величиной для выбора запросов в активном обучении.

Для активного обучения нужно запрашивать примеры с высокой эпистемической неопределённостью, а не с высокой алеаторной. Запрос принципиально неоднозначного примера расходует усилия оракула впустую: ни одна метка, которую тот предоставит, не будет явно правильной.

Дропаут по методу Монте-Карло

Практический подход к оценке эпистемической неопределённости для нейронных сетей — **MC Dropout** {cite}`gal2016dropout`: применять дропаут во время инференса и выполнять T прямых проходов. Дисперсия по предсказаниям является оценкой эпистемической неопределённости.

```
import numpy as np
import torch
import torch.nn as nn

torch.manual_seed(42)
rng = np.random.default_rng(42)

class MCDropoutNet(nn.Module):
    def __init__(self, input_dim=20, hidden=64, output_dim=2, p_drop=0.3):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden), nn.ReLU(), nn.Dropout(p_drop),
            nn.Linear(hidden, hidden),    nn.ReLU(), nn.Dropout(p_drop),
            nn.Linear(hidden, output_dim)
        )

    def forward(self, x):
        return self.net(x)

def mc_uncertainty(model, x, T=30):
    """
    Run T stochastic forward passes with dropout active.
    Returns mean prediction and epistemic uncertainty (predictive variance).
    """
    model.train()  # keep dropout active during inference
    with torch.no_grad():
```

```
        preds = torch.stack([
            torch.softmax(model(x), dim=-1) for _ in range(T)
        ])  # shape: (T, N, C)
    mean_pred = preds.mean(0)
    # Epistemic uncertainty: mean of variances across passes
    epistemic = preds.var(0).sum(-1)
    # Aleatoric uncertainty: entropy of mean prediction
    aleatoric = -(mean_pred * (mean_pred + 1e-9).log()).sum(-1)
    return mean_pred, epistemic, aleatoric

# Quick demonstration
model = MCDropoutNet(input_dim=20, output_dim=2)
# In-distribution example (simulated)
x_familiar   = torch.randn(1, 20) * 0.5
# Out-of-distribution example (far from training distribution)
x_unfamiliar = torch.randn(1, 20) * 3.0

for name, x in [("In-distribution ", x_familiar), ("Out-of-distribution", x_unfamiliar)]:
    _, ep, al = mc_uncertainty(model, x)
    print(f"{name} | epistemic: {ep.item():.4f} | aleatoric: {al.item():.4f}")
```

В необученной сети выше оба примера демонстрируют схожую неопределённость. После обучения пример «вне распределения» покажет более высокую эпистемическую неопределённость — модель не научилась надёжному отображению для входных данных, далёких от обучающего распределения.

Глубокие ансамбли

Обучение M независимо инициализированных моделей и усреднение их предсказаний обеспечивает более простую и нередко более надёжную оценку неопределённости, чем MC Dropout {cite}`lakshminarayanan2017simple`. Несогласие между членами ансамбля является сигналом эпистемической неопределённости.

Для активного обучения в масштабе и MC Dropout, и глубокие ансамбли добавляют накладные расходы, пропорциональные T или M прямым проходам. На практике $T = 10$–30 для MC Dropout или $M = 5$ членов ансамбля, как правило, достаточно для ранжирования примеров по эпистемической неопределённости, даже если абсолютные значения не откалиброваны хорошо.

Полный цикл активного обучения

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from copy import deepcopy

rng = np.random.default_rng(42)

# Generate dataset
```

```
X, y = make_classification(
    n_samples=2000, n_features=20, n_informative=10,
    n_classes=3, n_clusters_per_class=1, random_state=42
)
X_train, y_train = X[:1500], y[:1500]
X_test,  y_test  = X[1500:], y[1500:]

def entropy_query(model, X_pool):
    """Return index of most uncertain sample (entropy)."""
    probs = model.predict_proba(X_pool)
    ent = -np.sum(probs * np.log(probs + 1e-9), axis=1)
    return np.argmax(ent)

def random_query(X_pool):
    """Random baseline."""
    return rng.integers(0, len(X_pool))

def run_active_learning(strategy='entropy', n_initial=30, n_queries=120, query_batch=5):
    labeled_idx = list(rng.choice(len(X_train), n_initial, replace=False))
    unlabeled_idx = [i for i in range(len(X_train)) if i not in labeled_idx]
    accs = []

    for step in range(n_queries // query_batch):
        model = LogisticRegression(max_iter=500, C=1.0)
        model.fit(X_train[labeled_idx], y_train[labeled_idx])
        accs.append(accuracy_score(y_test, model.predict(X_test)))

        # Query
        X_pool = X_train[unlabeled_idx]
        for _ in range(query_batch):
            if strategy == 'entropy':
                q = entropy_query(model, X_pool)
            else:
                q = random_query(X_pool)
            labeled_idx.append(unlabeled_idx.pop(q))
            X_pool = X_train[unlabeled_idx]

    return np.array(accs)

labels_used = np.arange(1, 25) * 5 + 30  # label counts at each step

accs_active = run_active_learning(strategy='entropy')
accs_random = run_active_learning(strategy='random')

plt.figure(figsize=(8, 5))
plt.plot(labels_used, accs_active, 'o-', label='Entropy sampling', color='#2b3a8f', linewidth
plt.plot(labels_used, accs_random, 's--', label='Random baseline',  color='#e05c5c', linewidt
plt.xlabel("Number of labeled examples", fontsize=12)
plt.ylabel("Test accuracy", fontsize=12)
plt.title("Active Learning vs. Random Sampling", fontsize=13)
plt.legend(fontsize=11)
```

```
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig('active_learning_curve.png', dpi=150)
plt.show()

print(f"Active learning reaches {accs_active[-1]:.3f} accuracy")
print(f"Random sampling reaches {accs_random[-1]:.3f} accuracy")
print(f"Active learning saves ~{int((accs_random.tolist().index(min(accs_random, key=lambda a
```

Проблема холодного старта

Активному обучению нужна обученная модель для оценки неразмеченных точек — но в начале размеченных примеров нет (или очень мало). Это **проблема холодного старта**.

Практические решения:

1. **Случайная инициализация:** Разметить небольшое случайное начальное множество (20–100 примеров) перед началом активного обучения.
2. **Инициализация на основе кластеризации:** Применить k-средних к неразмеченному пулу; разметить один пример из каждого кластера. Это обеспечивает разнообразие начального размеченного множества.
3. **Выбор на основе вложений:** Использовать предварительно обученный энкодер для построения вложений примеров; выбрать разнообразное подмножество с помощью метода ключевых множеств.

Для большинства задач нескольких десятков случайных начальных меток, как правило, достаточно для запуска активного обучения; точное число зависит от баланса классов, размерности признаков и сложности модели.

Активное обучение в режиме батчей

На практике аннотаторы работают батчами — переобучение модели после каждой единственной метки неэффективно. **Активное обучение в режиме батчей** выбирает множество из b примеров для одновременной разметки.

Наивный выбор b наиболее неопределённых примеров приводит к **избыточности**: высоконеопределённые примеры склонны группироваться вместе (например, примеры вблизи границы решения в одной и той же области). Лучшие батчевые стратегии оптимизируют одновременно неопределённость *и* разнообразие внутри батча.

Детерминантальные точечные процессы (DPP) обеспечивают принципиальный способ выборки разнообразных батчей: они задают распределение на подмножествах, штрафующее похожие элементы. Вероятность подмножества S при DPP пропорциональна $\det(L_S)$, где L — ядерная матрица, кодирующая сходство.

Критерии остановки

Когда следует прекращать активное обучение? Распространённые критерии:

- **Бюджет исчерпан:** Простейший критерий — остановиться, когда исчерпан бюджет аннотирования.
- **Плато производительности:** Остановиться, когда точность модели на отложенной тестовой выборке не улучшилась более чем на δ за k последовательных раундов.
- **Порог уверенности:** Остановиться, когда менее некоторой доли неразмеченных примеров имеют неопределённость выше порогового значения.
- **Максимальное снижение потерь:** Оценить максимально возможный прирост от дополнительных меток; остановиться, когда он падает ниже порога {cite}`bloodgood2009method`.

Когда активное обучение работает (и когда нет)

Активное обучение, как правило, работает хорошо, когда: - Разметка дорогостояща, а неразмеченный пул велик - Данные обладают чёткой структурой, которую модель может использовать для выявления информативных примеров - Класс моделей подходит для задачи

Активное обучение работает плохо, когда: - Начальная модель очень слаба (холодный старт) и не может осмысленно ранжировать примеры - Стратегия запросов выбирает выбросы или неверно размеченные примеры (важна устойчивость к шуму) - Распределение данных смещается между неразмеченным пулом и тестовым распределением

Ключевая практическая проблема — **несоответствие распределений**: активное обучение склонно запрашивать примеры вблизи границы решения, создавая смещённое размеченное множество, которое может плохо представлять тестовое распределение. Это может привести к хорошо обученным границам решения, но плохой калиброванности.

> **See also:** Основополагающий обзор: {cite}`settles2009active`. Теоретические основы (сложность разметки, агностические границы): {cite}`balcan2006agnostic`. Активное обучение для глубокого обучения: {cite}`ash2020deep` (BADGE) и {cite}`sener2018active` (ключевые множества). Критическая оценка реальной пользы активного обучения: {cite}`lowell2019practical`. Алеаторная и эпистемическая неопределённость для глубоких моделей: {cite}`kendall2017uncertainties`; глубокие ансамбли как оценщики неопределённости: {cite}`lakshminarayanan2017simple`; MC Dropout как приближённый байесовский вывод: {cite}`gal2016dropout`.

Интерактивное машинное обучение

Активное обучение задаёт сфокусированный вопрос: при данном бюджете, какие примеры следует размечать? Интерактивное машинное обучение (IML) задаёт более широкий вопрос: как спроектировать *всё взаимодействие* между человеком и обучающейся системой так, чтобы оно было максимально продуктивным, приятным и правильным?

IML отличается **непосредственностью** и **прямотой** цикла обратной связи человек–модель. Там где традиционное МО предполагает передачу данных и ожидание завершения обучения, IML позволяет людям наблюдать поведение модели, предоставлять обратную связь и видеть ответную реакцию модели — нередко за секунды.

Принципы интерактивного машинного обучения

Амерши и др. {cite}`amershi2014power` выделяют три определяющие характеристики IML:

1. **Быстрая обратная связь:** Модель обновляется достаточно быстро, чтобы человек воспринимал эффект своей обратной связи. В пределе обновления модели происходят в реальном времени.

2. **Прямое манипулирование:** Человек взаимодействует с моделью через данные или предсказания модели — не через конфигурационные файлы или настройку гиперпараметров.

3. **Итеративное уточнение:** Процесс является по-настоящему итеративным: следующее действие человека определяется текущим поведением модели, которое, в свою очередь, было сформировано предыдущими действиями человека.

Это создаёт тесный **ко-адаптивный цикл**: и человек, и модель меняются со временем в ответ на действия друг друга. Человек узнаёт, что понимает модель; модель узнаёт, что важно человеку.

Сравнение с активным обучением

IML и активное обучение значительно перекрываются, однако не тождественны:

Свойство	Активное обучение	Интерактивное МО
Основной вопрос	Что размечать?	Как взаимодействовать?
Задержка обратной связи	Может быть батчевой (дни)	Как правило, реальное время
Частота обновления модели	По раунду (батч)	По взаимодействию (онлайн)
Инициатива человека	Отвечает на вопросы модели	Может проактивно обучать модель
Дизайн интерфейса	Второстепенная задача	Центральная задача
Когнитивная нагрузка	Явно не моделируется	Явно рассматривается

В активном обучении машина направляет взаимодействие. В IML человек тоже может брать инициативу — предоставляя примеры, исправления или обратную связь по любому аспекту поведения модели, который кажется наиболее проблематичным.

Взаимодействие со смешанной инициативой

Системы со **смешанной инициативой** позволяют и человеку, и машине брать ведущую роль в разные моменты {cite}`allen1999mixed`. Система, управляемая исключительно машиной, задаёт вопросы, а человек отвечает. Система, управляемая исключительно человеком, оставляет все решения за человеком. Системы со смешанной инициативой балансируют оба подхода.

На практике лучшие системы IML сочетают: - **Инициативу машины:** «Я не уверена в этих примерах — не могли бы вы их разметить?» - **Инициативу человека:** «Я замечаю, что модель постоянно ошибается в этой категории — дам ей больше примеров» - **Подтверждение:** Модель демонстрирует своё текущее понимание; человек подтверждает или исправляет

Хорошие интерфейсы IML делают текущее понимание модели видимым и поддающимся исправлению. Это требование **понятности**: люди могут направлять только ту модель, которую понимают — хотя бы приблизительно.

Человеческие факторы в IML

IML вводит человеческие факторы — когнитивную нагрузку, усталость, последовательность и доверие — непосредственно в цикл обучения. Плохой дизайн IML приводит к:

Усталости от аннотирования: Люди принимают всё более быстрые и небрежные решения по мере продолжения сеанса. Ошибки проникают в обучающие данные.

Якорной предвзятости: Люди чрезмерно полагаются на текущие подсказки модели. Если интерфейс предварительно заполняет предсказание модели, аннотаторы с меньшей вероятностью его исправляют, даже когда оно неверно, — это систематический источник шума меток, нарастающий через раунды аннотирования {cite}`geva2019annotator`. Предварительное аннотирование может ускорить пропускную способность {cite}`lingren2014evaluating` одновременно снижая частоту выявления аннотаторами ошибок модели; при проектировании интерфейса IML необходимо взвесить оба этих эффекта.

Неправильная калибровка доверия: Люди либо чрезмерно доверяют (принимая неверные выходы модели), либо недооценивают (игнорируя правильные подсказки). Оба паттерна снижают ценность сотрудничества человека и модели.

Последовательность сеансов: Люди могут принимать разные решения для одного и того же примера в разное время, особенно после длительных сеансов. Проверки последовательности (повторное предъявление более ранних примеров) могут обнаружить и исправить это.

Хороший дизайн IML смягчает эти проблемы через выбор интерфейса: явное отображение уверенности модели, случайный порядок представления, ограничение длины сеанса и встроенные проверки последовательности.

Типы обратной связи в IML на практике

Обратная связь на уровне примеров

Человек предоставляет метку или исправление для конкретного примера. Это наиболее распространённая форма, непосредственно совместимая с обучением с учителем.

Обратная связь на уровне признаков

Человек указывает, какие признаки релевантны или нерелевантны. «Модель должна обращать внимание на слова “срочно” и “дедлайн” для этой категории». Это выразительнее обратной связи на уровне примеров и может быть эффективнее для определённых задач.

TFIDF Interactive и аналогичные системы позволяют аннотаторам выделять релевантные слова в текстовых документах. Эти выделения преобразуются в ограничения или дополнительный надзор за вниманием модели.

Обратная связь на уровне модели

Человек напрямую корректирует поведение модели на классе входных данных: «Всякий раз, когда входные данные содержат [X], выходные данные должны быть [Y]». Это соответствует логическим правилам или ограничениям в таких подходах, как Posterior Regularization {cite}`ganchev2010posterior` или обучение под управлением ограничений.

Пример из практики: Google Teachable Machine

Teachable Machine — доступная веб-система IML, позволяющая нетехническим пользователям обучать классификаторы изображений в браузере. Пользователь:

1. Записывает примеры каждого класса с помощью веб-камеры
2. Обучает модель одним нажатием (дообучение MobileNet в браузере)
3. Немедленно видит предсказания модели на живом видео
4. Добавляет больше примеров для классов, в которых модель ошибается

Это иллюстрирует основной цикл IML: предоставить примеры наблюдать за моделью выявить сбой предоставить более целевые примеры. Обратная связь в реальном времени (выходы модели обновляются в реальном времени, как правило, с

интерактивной частотой кадров на современном оборудовании) делает ко-адаптивный цикл непосредственно ощутимым.

Реализация простого цикла IML

```
import numpy as np
from sklearn.linear_model import SGDClassifier
from sklearn.preprocessing import StandardScaler
import warnings
warnings.filterwarnings('ignore')

class SimpleIMLSystem:
    """
    Minimal IML system that allows online feedback and displays
    model state after each annotation.
    """

    def __init__(self, n_features=10, n_classes=2):
        self.model = SGDClassifier(loss='log_loss', max_iter=1, warm_start=True,
                                   random_state=42)
        self.scaler = StandardScaler()
        self.X_seen = []
        self.y_seen = []
        self.n_classes = n_classes
        self.initialized = False

    def update(self, x, y_true):
        """Receive a single labeled example and update the model."""
        self.X_seen.append(x)
        self.y_seen.append(y_true)

        if len(self.X_seen) >= 2 * self.n_classes:
            X_arr = np.array(self.X_seen)
            y_arr = np.array(self.y_seen)
            X_scaled = self.scaler.fit_transform(X_arr)
            self.model.partial_fit(X_scaled[-1:], y_arr[-1:],
                                   classes=list(range(self.n_classes)))
            self.initialized = True

        return self

    def predict_with_confidence(self, x):
        """Predict label and return confidence."""
        if not self.initialized:
            return None, 0.0
        x_scaled = self.scaler.transform(x.reshape(1, -1))
        probs = self.model.predict_proba(x_scaled)[0]
        return self.model.predict(x_scaled)[0], probs.max()
```

```
    def current_accuracy(self, X_val, y_val):
        if not self.initialized:
            return None
        X_scaled = self.scaler.transform(X_val)
        return (self.model.predict(X_scaled) == y_val).mean()


# Simulate an IML session
rng = np.random.default_rng(42)
X_all, y_all = np.random.default_rng(0).random((500, 10)), np.random.default_rng(0).integers(
X_val, y_val = X_all[400:], y_all[400:]

system = SimpleIMLSystem(n_features=10, n_classes=2)
accs, confidences = [], []

print("Step | Labels | Accuracy | Example confidence")
print("-" * 50)

for step in range(100):
    x, y = X_all[step], y_all[step]
    pred, conf = system.predict_with_confidence(x)
    system.update(x, y)

    if (step + 1) % 10 == 0:
        acc = system.current_accuracy(X_val, y_val)
        if acc is not None:
            accs.append(acc)
            print(f"  {step+1:3d} |  {step+1:4d}  |  {acc:.3f}   | {conf:.3f}")
```

Тест бабушки

Одна полезная эвристика для оценки проектирования интерфейса IML — и проектирования систем HITL в целом — это то, что мы назовём **Тестом бабушки** (оригинальный термин, вводимый здесь как проектное ограничение, а не ссылка на предшествующие работы):

> *Женщина 1930 года рождения должна уметь пользоваться этим устройством с помощью голоса, а при затруднении — плавно перейти к клавиатурному или текстовому интерфейсу.*

Тест касается не только доступности, хотя это тоже важно. Он о **проектировании с учётом трения**. Если система требует ментальной модели нейронных сетей, циклов обучения или вероятностных распределений для эффективного использования, она не прошла Тест бабушки. Человек в петле должен уметь участвовать, не понимая машинную сторону петли.

Практические следствия для проектирования IML конкретны:

Голос как основной, запасной вариант для других режимов: Основная модальность взаимодействия должна быть естественным языком или жестами — не ползунками параметров или порогами уверенности. Эксперты могут хотеть ползунки; каждый должен уметь сказать «это неверно».

Постепенная деградация: Когда предпочитаемая пользователем модальность даёт сбой или вызывает фрустрацию, система должна предлагать альтернативу — а не пустой экран или сообщение об ошибке. Интерфейс является частью обучающей системы; пользователь, неспособный взаимодействовать, не может обучать.

Читаемое состояние модели: Текущее понимание модели должно быть видимым в понятных человеку терминах. Не «уверенность: 0,73», а «я достаточно уверена, что это [X], но видела подобные примеры и в другом свете». Неопределённость должна передаваться языком, приглашающим к исправлению.

Терпимость к неоднозначности: Пользователь в возрасте 93 лет и инженер МО в 23 года будут взаимодействовать с одной и той же системой по-разному. Тест бабушки спрашивает, может ли система принять обоих — не обнаруживая возраст пользователя, а проектируя взаимодействия, работающие в диапазоне уровней экспертизы и комфорта.

Тест приобретает особое значение по мере того, как системы МО переходят от инструментов исследования к повседневной инфраструктуре. Ассистент для медицинской визуализации, используемый радиологами; классификатор юридических документов, используемый клерками; система образовательной обратной связи, используемая учителями — каждый из них предполагает людей в петле, не записывавшихся в обучающие ИИ. Проектировать для них — не уступка; это и есть суть.

:::{admonition} Принцип проектирования :class: tip Тест бабушки — это проектное ограничение, а не целевая демография. Системы, прошедшие его, более устойчивы к разнообразию пользователей, более терпимы к различиям в экспертизе и честнее в отношении того, чего они ожидают от людей в петле. Если система требует объяснения перед использованием, она просит человека выполнять дополнительную работу. Эта работа должна быть оправдана соразмерной пользой. :::

IML и фундаментальные модели

Современное IML всё чаще использует **предварительно обученные фундаментальные модели** {cite}`bommasani2021opportunities` в качестве основы. Вместо обучения с нуля пользователи дообучают большую предварительно обученную модель на небольшом количестве интерактивных примеров. Это может резко уменьшить количество примеров, необходимых для достижения полезной производительности, — в благоприятных случаях до 5–50 примеров вместо тысяч, в зависимости от того, насколько хорошо предварительно обученные представления соответствуют целевой задаче {cite}`bommasani2021opportunities`.

Техники, делающие это возможным: - **Промптинг с несколькими примерами:** Предоставление примеров в контекстном окне LLM - **Дообучение адаптеров:** Обновление небольших адаптерных модулей при замороженной базовой модели - **Параметрически-эффективное дообучение (PEFT):** LoRA, prefix tuning и аналогичные методы, допускающие быстрые и экономичные обновления

Фундаментальные модели меняют динамику IML: люди больше не обучают модель с нуля, а *направляют* модель, уже обладающую значительными знаниями. Задача смещается от «как предоставить достаточно примеров» к «как точно указать, чем желаемое поведение отличается от того, что модель уже делает».

> **See also:** Обзор принципов IML: {cite}`amershi2014power`. Системы со смешанной инициативой: {cite}`allen1999mixed`. Якорные эффекты в

аннотировании: {cite}`geva2019annotator` и {cite}`lingren2014evaluating`.

Обучение с подкреплением на основе обратной связи от человека

Ни одна техника не сделала столько для вывода HITL ML в мейнстрим, сколько обучение с подкреплением на основе обратной связи от человека (RLHF). Это механизм, лежащий в основе InstructGPT {cite}`ouyang2022training` и являющийся центральным компонентом конвейеров следования инструкциям во многих современных больших языковых моделях {cite}`stiennon2020learning`. Понимание RLHF — не только как рецепта, которому нужно следовать, но и как принципиального подхода к согласованию — необходимо всем, кто работает в современном ИИ.

Проблема согласования

Большие языковые модели (LLM), обученные исключительно на предсказании следующего токена, оптимизируют прокси-цель: предсказать, какой текст идёт дальше в корпусе написанного людьми текста. Эта цель связана с тем, что нам действительно нужно, но отличается от него: ответы, которые полезны, точны, безопасны и согласованы с человеческими ценностями.

Рассогласование между целью обучения и желаемым поведением называется **проблемой согласования** {cite}`russell2019human`. Конкретно, языковая модель, обученная на текстах интернета, учится: - Производить правдоподобно звучащие продолжения (которые могут быть фактически неверными) - Воспроизводить предвзятости и вред, присутствующие в обучающих данных - Быть уклончивой или манипулятивной, если статистически это следует за подсказкой

RLHF решает проблему согласования, делая предпочтения людей *частью цели оптимизации*.

Конвейер RLHF

RLHF выполняется в три этапа:

```
Этап 1: Дообучение с учителем (SFT)
  --> Собрать демонстрационные данные (человек пишет идеальные ответы)
  --> Дообучить базовый LLM на демонстрациях

Этап 2: Обучение модели вознаграждения
  --> Собрать попарные предпочтения (человек оценивает A против B)
```

```
  --> Обучить модель вознаграждения R(x, y) для предсказания предпочтений человека

Этап 3: Дообучение с помощью RL
  --> Дообучить LLM с помощью PPO/RL для максимизации R(x, y)
  --> Штраф KL предотвращает чрезмерное отклонение от модели SFT
```

Этап 1: Дообучение с учителем

Начиная с предварительно обученной базовой модели π_0, собирается датасет пар (подсказка, идеальный ответ), написанных или отобранных подрядчиками-людьми, следующими подробным инструкциям. Модель дообучается на этих демонстрациях с использованием стандартной перекрёстной энтропии:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\mathbb{E}_{(x,y)\sim\mathcal{D}_{\text{demo}}}\left[\log \pi_\theta(y \mid x)\right]$$

Модель SFT π_{SFT} является значительно лучшей отправной точкой для RLHF, чем необработанная предварительно обученная модель.

Этап 2: Обучение модели вознаграждения

Для набора подсказок $\{x_i\}$ генерируется K ответов на подсказку с использованием π_{SFT} и представляется разметчикам в виде попарных сравнений: «Какой ответ лучше, A или B?»

Модель вознаграждения r_ϕ обучается предсказывать эти предпочтения. В рамках модели **Bradley–Terry** (глава 8) вероятность того, что ответ y_w предпочитается ответу y_l, составляет:

$$P(y_w \succ y_l \mid x) = \sigma\left(r_\phi(x, y_w) - r_\phi(x, y_l)\right)$$

Модель вознаграждения обучается минимизировать попарную потерю ранжирования:

$$\mathcal{L}_{\text{RM}}(\phi) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}_{\text{pref}}}\left[\log \sigma\left(r_\phi(x, y_w) - r_\phi(x, y_l)\right)\right]$$

Модель вознаграждения, как правило, инициализируется из модели SFT с скалярной головкой, заменяющей последний слой.

Этап 3: Дообучение с RL с помощью PPO

Имея обученную модель вознаграждения, можно использовать обучение с подкреплением для дообучения LLM. Каждая подсказка x является состоянием; каждый ответ y — траекторией выборов токенов; а вознаграждение равно $r_\phi(x, y)$.

Цель оптимизации включает **штраф дивергенции Кульбака–Лейблера**, чтобы модель не слишком сильно отклонялась от базовой линии SFT (что привело бы к взлому вознаграждения {cite}`krakovna2020specification,gao2023scaling`):

$$\max_\theta \mathbb{E}_{x\sim\mathcal{D}, y\sim\pi_\theta(\cdot|x)}\left[r_\phi(x, y) - \beta \cdot \text{KL}\left[\pi_\theta(\cdot \mid x) \| \pi_{\text{SFT}}(\cdot \mid x)\right]\right]$$

Параметр β управляет силой штрафа KL. Малое β допускает более агрессивную оптимизацию, но рискует взломом вознаграждения; большое β удерживает модель близко к SFT, но ограничивает прирост от согласования.

Проксимальная оптимизация политики (PPO) {cite}`schulman2017proximal` является стандартным алгоритмом для этого этапа, выбранным за его стабильность относительно методов чистого градиента политики.

Упрощённая демонстрация RLHF

Полный конвейер RLHF требует крупномасштабной инфраструктуры. Следующий пример демонстрирует ключевые идеи в небольшом масштабе.

```
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.nn import functional as F

torch.manual_seed(42)
rng = np.random.default_rng(42)

# ----------------------------------------------
# Toy setup: responses are 4-dimensional vectors
# "Quality" is known analytically (sum of positive values)
# We simulate a reward model learning this from pairwise feedback
# ----------------------------------------------

class RewardModel(nn.Module):
    def __init__(self, d=4, hidden=32):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(d, hidden), nn.ReLU(),
            nn.Linear(hidden, hidden), nn.ReLU(),
            nn.Linear(hidden, 1)
        )
    def forward(self, x):
        return self.net(x).squeeze(-1)

def true_quality(x):
    """The hidden ground-truth reward function."""
    return x.sum(dim=-1) + 0.5 * (x ** 2).mean(dim=-1)

# Generate pairwise preference data
N_PAIRS = 500
X1 = torch.randn(N_PAIRS, 4)
X2 = torch.randn(N_PAIRS, 4)
q1, q2 = true_quality(X1), true_quality(X2)
# Human prefers X1 when q1 > q2 (with some noise)
noise = torch.randn(N_PAIRS) * 0.5
```

```
preferred_1 = ((q1 - q2 + noise) > 0).float()

# Train reward model
rm = RewardModel(d=4, hidden=32)
optimizer = optim.Adam(rm.parameters(), lr=3e-3)

losses = []
for epoch in range(200):
    r1 = rm(X1)
    r2 = rm(X2)
    # Bradley-Terry loss
    logit = r1 - r2
    loss = F.binary_cross_entropy_with_logits(logit, preferred_1)
    optimizer.zero_grad(); loss.backward(); optimizer.step()
    losses.append(loss.item())

# Evaluate: does the reward model agree with true quality?
X_eval = torch.randn(1000, 4)
with torch.no_grad():
    r_pred = rm(X_eval).numpy()
    r_true = true_quality(X_eval).numpy()

corr = np.corrcoef(r_pred, r_true)[0, 1]
print(f"Reward model correlation with true quality: {corr:.4f}")
print(f"Final training loss: {losses[-1]:.4f}")

import matplotlib.pyplot as plt
plt.figure(figsize=(8, 3))
plt.subplot(1, 2, 1)
plt.plot(losses, color='#2b3a8f', linewidth=1.5)
plt.xlabel("Epoch"); plt.ylabel("Pairwise loss")
plt.title("Reward Model Training")

plt.subplot(1, 2, 2)
plt.scatter(r_true[:200], r_pred[:200], alpha=0.4, s=15, color='#2b3a8f')
plt.xlabel("True quality"); plt.ylabel("Predicted reward")
plt.title(f"Reward Model vs. Truth (r={corr:.3f})")
plt.tight_layout()
plt.savefig('reward_model.png', dpi=150)
plt.show()
```

Проблемы RLHF

Взлом вознаграждения

Ключевой режим сбоя: политика находит способы получить высокое вознаграждение от модели вознаграждения, не соответствующие действительно хорошему поведению. Например, LLM может научиться производить ответы, которые звучат лестно или уверенно (что разметчики склонны оценивать высоко), а не точные.

Взлом вознаграждения более вероятен, когда: - Модель вознаграждения обучена на недостаточном количестве данных предпочтений - Политике позволено сильно отклоняться от базовой линии SFT (малое β) - Распределение модели вознаграждения смещается в процессе обучения PPO

Стратегии снижения: Штраф KL, итеративное обучение модели вознаграждения, разнообразная оценка, ограничения конституционного ИИ.

Предвзятость оценщиков

Разметчики-люди имеют систематические предвзятости. Они склонны предпочитать более длинные ответы (предвзятость многословия), более уверенно звучащий текст (предвзятость уверенности) и ответы, согласующиеся с их предварительными убеждениями. Эти предвзятости распространяются в модель вознаграждения.

Известный сбой угодничества у RLHF-моделей — когда модель говорит пользователям то, что они хотят услышать, а не то, что является правдой, — отчасти является следствием предвзятости оценщиков в пользу приятных ответов.

Масштабируемый надзор

Для сложных задач люди не могут надёжно судить, какой ответ ИИ является правильным. Разметчик, сравнивающий два длинных математических доказательства или две реализации кода, может просто выбрать более читаемый, вне зависимости от корректности. **Масштабируемый надзор** — открытая исследовательская проблема разработки процедур оценки, сохраняющих надёжность по мере роста сложности задачи {cite}`bowman2022measuring`.

Конституционный ИИ (RLAIF)

Конституционный ИИ {cite}`bai2022constitutional`, разработанный в Anthropic, снижает зависимость от разметчиков-людей, используя сам ИИ для генерации меток предпочтений, руководствуясь набором принципов («конституцией»). Процесс:

1. Генерировать ответы на потенциально вредоносные подсказки
2. Использовать ИИ-критика для оценки ответов против конституционных принципов
3. Пересматривать ответы под руководством ИИ-обратной связи (RLAIF — RL на основе обратной связи от ИИ)
4. Обучить модель вознаграждения на предпочтениях, сгенерированных ИИ
5. Дообучить с помощью RLHF с использованием этой модели вознаграждения

RLAIF способен генерировать данные предпочтений в значительно большем масштабе, чем разметка людьми, и позволяет тонко контролировать ценности, кодируемые в модели вознаграждения.

> **See also:** Оригинальная статья InstructGPT: {cite}`ouyang2022training`. Основополагающая работа по RLHF для глубокого RL: {cite}`christiano2017deep`. PPO описан в {cite}`schulman2017proximal`. Конституционный ИИ: {cite}`bai2022constitutional`.

Обучение на основе демонстраций

Когда задачу трудно формализовать, но легко продемонстрировать, обучение на примерах может быть эффективнее, чем определение через правила. Эксперт-человек показывает роботизированному манипулятору, как захватить объект; программист взаимодействует с IDE, предоставляя последовательность правильных правок; шахматный гроссмейстер разыгрывает партию. **Обучение на основе демонстраций** извлекает политику из таких поведенческих данных, избавляя от необходимости ручного конструирования функции вознаграждения или явной спецификации задачи.

Клонирование поведения

Простейший подход — **клонирование поведения (BC)**: рассматривать демонстрацию как данные с учителем и обучать отображение из состояний в действия.

Дан датасет пар состояние–действие $\mathcal{D} = \{(s_i, a_i)\}$ от эксперта-демонстратора, обучается политика $\pi_\theta(a \mid s)$ путём минимизации отрицательного логарифма правдоподобия:

$$\mathcal{L}_{\text{BC}}(\theta) = -\frac{1}{|\mathcal{D}|} \sum_{(s,a)\in\mathcal{D}} \log \pi_\theta(a \mid s)$$

Это в точности стандартное обучение с учителем, применённое к последовательным данным.

Проблема ковариатного сдвига

BC имеет фундаментальный изъян: **сдвиг распределения** между обучением и развёртыванием. Демонстрации эксперта охватывают состояния, посещаемые экспертом. Однако в процессе развёртывания обученная политика может принимать несколько иные решения, попадая в состояния, которые эксперт никогда не посещал, — состояния, где у политики нет надзора и где она может давать серьёзный сбой.

Ошибки **накапливаются**: небольшое отклонение приводит к незнакомому состоянию, где чуть неверное действие приводит к ещё более незнакомому состоянию, и так далее. Производительность деградирует как $O(T^2\epsilon)$, где T — длина эпизода, а ϵ — частота ошибок на каждом шаге, — что значительно хуже деградации $O(T\epsilon)$ оракульной политики {cite}`ross2010efficient`.

Пример: автономное вождение :class: note

Модель клонирования поведения для удержания полосы, обученная на данных вождения человека, хорошо работает на прямых дорогах (состояния близко к обучающему распределению). Но как только автомобиль чуть отклонится — в состояние, в котором ни один водитель не оказался бы, поскольку уже скорректировал бы курс, — модель не имеет данных для руководства и может съехать с дороги.

```
Алгоритм DAgger:
  Инициализация: D <- {} (пустой датасет)
  Обучить начальную политику pi_1 на M демонстрациях эксперта

  для итерации i = 1, 2, ..., N:
    1. Запустить pi_i в среде для сбора состояний {s_1, ..., s_t}
    2. Запросить у эксперта действия для каждого посещённого состояния: a_t = pi*(s_t)
    3. Агрегировать: D <- D u {(s_1, a_1), ..., (s_t, a_t)}
    4. Обучить pi_{i+1} методом обучения с учителем на D
```

DAgger достигает сожаления $O(T\epsilon)$ — такого же, как у оракульной политики, — поскольку обучающее распределение сходится к совпадению с распределением развёртывания.

Ключевое требование: эксперт должен быть доступен для запросов в любом состоянии, включая состояния, которые эксперт никогда бы не посетил естественным образом. Это осуществимо в симуляции (попросить эксперта скорректировать робота из нестандартной конфигурации), но может быть затруднительным или небезопасным в реальных физических системах.

Обратное обучение с подкреплением

Иногда поведение эксперта лучше понимать не как последовательность действий для имитации, а как результат оптимизации неизвестной функции вознаграждения. **Обратное обучение с подкреплением (IRL)** {cite}`ng2000algorithms` восстанавливает эту латентную функцию вознаграждения из демонстраций.

Дана демонстрация $\tau = \{(s_1, a_1), \dots, (s_T, a_T)\}$, IRL находит функцию вознаграждения $R(s, a)$ такую, что политика эксперта оптимальна при R.

Преимущество IRL перед BC: если нам удаётся восстановить истинную функцию вознаграждения, мы можем повторно оптимизировать её в новых средах, с другой динамикой или с улучшенными планировщиками — обобщаясь далеко за пределы продемонстрированных сценариев.

MaxEnt IRL

MaxEnt IRL {cite}`ziebart2008maximum` разрешает проблему неоднозначности IRL (существует множество функций вознаграждения, согласованных с любым набором демонстраций) путём выбора функции вознаграждения, которая, будучи согласованной с продемонстрированным поведением, приводит к распределению по траекториям с *максимальной энтропией*. Траектории распределены как:

$$P(\tau \mid R) \propto \exp\left(\sum_t R(s_t, a_t)\right)$$

Цель обучения состоит в согласовании ожидаемых характеристик эксперта $\mu_E = \mathbb{E}_{\tau\sim\pi^*}[\phi(\tau)]$ с ожидаемыми характеристиками модели $\mu_\theta = \mathbb{E}_{\tau\sim\pi_\theta}[\phi(\tau)]$.

GAIL: генеративное состязательное имитационное обучение

GAIL {cite}`ho2016generative` полностью обходит обучение функции вознаграждения, используя GAN-подобную формулировку для непосредственного согласования распределения состояний и действий с экспертом.

Дискриминатор D_ψ обучается различать пары состояние–действие эксперта $(s, a) \sim \pi^*$ и пары политики $(s, a) \sim \pi_\theta$:

$$\mathcal{L}_D = -\mathbb{E}_{\pi^*}[\log D_\psi(s, a)] - \mathbb{E}_{\pi_\theta}[\log(1 - D_\psi(s, a))]$$

Генератор (политика π_θ) обучается обманывать дискриминатор — то есть производить пары состояние–действие, похожие на экспертские. Выход дискриминатора $\log D_\psi(s, a)$ служит сигналом вознаграждения для политики.

GAIL достигает производительности уровня эксперта на бенчмарках непрерывного управления, используя значительно меньше демонстраций, чем BC, и обобщается лучше, чем MaxEnt IRL в сложных средах.

Клонирование поведения в NLP: практический пример

```python
import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset

torch.manual_seed(42)

# ------------------------------------------------
# Toy NLP task: rewriting sentences to be more formal
# We simulate this as a simple sequence transformation
# In practice: fine-tuning a seq2seq model on expert rewrites
# ------------------------------------------------

class SimpleSeq2Seq(nn.Module):
    def __init__(self, vocab_size=100, embed_dim=32, hidden_dim=64):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        self.encoder = nn.GRU(embed_dim, hidden_dim, batch_first=True)
        self.decoder = nn.GRU(embed_dim, hidden_dim, batch_first=True)
        self.proj = nn.Linear(hidden_dim, vocab_size)
```

```
        self.hidden_dim = hidden_dim

    def forward(self, src, tgt):
        src_emb = self.embed(src)
        _, hidden = self.encoder(src_emb)
        tgt_emb = self.embed(tgt)
        out, _ = self.decoder(tgt_emb, hidden)
        return self.proj(out)

# Generate synthetic demonstration data
VOCAB = 100
rng = np.random.default_rng(42)
N, SEQ_LEN = 1000, 12

src_seqs = torch.tensor(rng.integers(1, VOCAB, (N, SEQ_LEN)), dtype=torch.long)
# "Expert" transformation: shift tokens by 1 (toy formalization)
tgt_seqs = torch.clamp(src_seqs + 1, 1, VOCAB - 1)
tgt_in  = torch.cat([torch.ones(N, 1, dtype=torch.long), tgt_seqs[:, :-1]], dim=1)

dataset = TensorDataset(src_seqs, tgt_in, tgt_seqs)
loader = DataLoader(dataset, batch_size=64, shuffle=True)

model = SimpleSeq2Seq(vocab_size=VOCAB)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
criterion = nn.CrossEntropyLoss(ignore_index=0)

# Behavioral cloning training
train_losses = []
for epoch in range(20):
    epoch_loss = 0
    for src, tgt_i, tgt_o in loader:
        logits = model(src, tgt_i)
        loss = criterion(logits.reshape(-1, VOCAB), tgt_o.reshape(-1))
        optimizer.zero_grad(); loss.backward(); optimizer.step()
        epoch_loss += loss.item()
    train_losses.append(epoch_loss / len(loader))

print(f"Initial loss: {train_losses[0]:.3f}")
print(f"Final loss:   {train_losses[-1]:.3f}")

# Evaluate: check token accuracy on held-out examples
model.eval()
with torch.no_grad():
    src_test = src_seqs[-100:]
    tgt_test_in = tgt_in[-100:]
    tgt_test_out = tgt_seqs[-100:]
    logits = model(src_test, tgt_test_in)
    preds = logits.argmax(dim=-1)
    acc = (preds == tgt_test_out).float().mean().item()
    print(f"Token accuracy on held-out set: {acc:.3f}")
```

Сравнение методов имитационного обучения

Метод	Требует вознаграждения?	Эксперт опрашивается онлайн?	Обобщается на новую динамику?	Сложность
Клонирование BC	Нет	Нет	Плохо (сдвиг распределения)	Низкая
DAgger	Нет	Да	Умеренно	Средняя
MaxEnt IRL	Восстанавливает	Нет	Хорошо	Высокая
GAIL	Нет	Нет	Хорошо	Высокая

Приложения

Робототехника. Обучение роботов манипулировать объектами, ориентироваться в среде или выполнять бытовые задачи. Физические демонстрации собираются через телеуправление или кинестетическое обучение.

Автономное вождение. Ранние системы автономного вождения, такие как ALVINN {cite}`pomerleau1989alvinn` и DAVE от NVIDIA, в значительной мере опирались на клонирование поведения на основе данных вождения людьми.

Игровой ИИ. Имитационное обучение на игровом процессе людей обеспечивает начальный запуск агентов перед дообучением с помощью RL. AlphaStar обучался на репликах людей перед применением RL; этот подход распространён, когда доступны демонстрации уровня человека.

Генерация кода. Дообучение языковых моделей на высококачественных демонстрациях кода (GitHub Copilot, Codex) является формой клонирования поведения.

Клиническая поддержка принятия решений. Обучение на последовательностях решений экспертов-врачей для сложных протоколов.

> **See also:** Основополагающий анализ BC/DAgger: {cite}`ross2011reduction`. MaxEnt IRL: {cite}`ziebart2008maximum`. GAIL: {cite}`ho2016generative`. Исчерпывающий обзор имитационного обучения: {cite}`osa2018algorithmic`.

Обучение на основе сравнений и ранжирований

Просить людей присвоить абсолютную оценку качества результату — сложная задача. Какова числовая оценка качества этого эссе по шкале от 1 до 10? Вопрос поставлен некорректно: у людей нет стабильной внутренней шкалы, а их оценки в значительной мере определяются якорением, контекстом и усталостью.

Просить людей *сравнить* два результата значительно проще: какое эссе лучше, A или B? Сравнительные суждения более последовательны, более надёжны и более непосредственно отражают предпочтения людей, чем абсолютные оценки. В данной главе рассматриваются математические основы и практические применения обучения на основе сравнений и ранжирований.

Почему сравнения лучше оценок

Психологическое обоснование

Превосходство сравнительных суждений имеет долгую историю в психометрии. Закон сравнительного суждения Тёрстоуна {cite}`thurstone1927law` (1927) показал, что даже когда абсолютные суждения людей непоследовательны, их относительные суждения следуют согласованному вероятностному закону. Сравнения используют тот факт, что люди значительно лучше оценивают относительное, чем абсолютное.

Статистическая эффективность

Каждое попарное сравнение предоставляет информацию об *относительных* позициях двух элементов на шкале качества. При K элементах $K-1$ сравнений могут ранжировать все элементы; для нахождения лучшего элемента требуется лишь $O(\log K)$ сравнений. Абсолютные оценки, как правило, требуют большего числа суждений для достижения той же точности.

Масштабируемость

Для генеративных моделей количество различных выходов фактически бесконечно. Абсолютная оценка результата требует установления общей шкалы по всем результатам; сравнение результатов требует лишь локальных, относительных суждений, естественно откалиброванных относительно друг друга.

Модель Брэдли–Терри

Доминирующей вероятностной моделью для попарных сравнений является **модель Брэдли–Терри (BT)** {cite}bradley1952rank. Каждый элемент i имеет латентный показатель качества $\alpha_i \in \mathbb{R}$. Вероятность того, что элемент i предпочтётся элементу j при прямом сравнении, составляет:

$$P(i \succ j) = \frac{e^{\alpha_i}}{e^{\alpha_i} + e^{\alpha_j}} = \sigma(\alpha_i - \alpha_j)$$

где σ — логистическая сигмоида. Это эквивалентно допущению, что воспринимаемое качество элемента i равно $\alpha_i + \epsilon$, где ϵ — стандартный логистический шум.

Идентифицируемость

Модель BT идентифицируема с точностью до трансляции: если α является решением, то $\alpha + c$ тоже является решением для любой константы c. Стандартное соглашение — фиксировать один показатель (например, $\alpha_0 = 0$) или накладывать ограничение $\sum_i \alpha_i = 0$. Показатели идентифицируются только тогда, когда **граф сравнений** (узлы = элементы, рёбра = наблюдаемые пары) является **связным** — если граф имеет несвязные компоненты, относительные показатели между компонентами не определены.

Оценка параметров

Дан датасет попарных сравнений $\mathcal{D} = \{(i, j, y_{ij})\}$, где $y_{ij} = 1$, если i было предпочтено j, логарифмическое правдоподобие составляет:

$$\mathcal{L}(\alpha) = \sum_{(i,j,y_{ij}) \in \mathcal{D}} \left[y_{ij} \log \sigma(\alpha_i - \alpha_j) + (1 - y_{ij}) \log \sigma(\alpha_j - \alpha_i) \right]$$

Это вогнутая функция от α, которую можно максимизировать с помощью градиентного подъёма или метода Ньютона.

```
import numpy as np
from scipy.optimize import minimize
from scipy.special import expit  # sigmoid

rng = np.random.default_rng(42)

# ------------------------------------------------
# Simulate Bradley-Terry: 8 items with true quality scores
# Generate pairwise comparisons and recover the scores
# ------------------------------------------------

N_ITEMS = 8
true_alpha = rng.normal(0, 1, N_ITEMS)
print(f"True quality ranking: {np.argsort(true_alpha)[::-1]}")

# Generate comparisons: every pair compared 5 times
comparisons = []
for i in range(N_ITEMS):
    for j in range(i + 1, N_ITEMS):
```

```
        for _ in range(5):
            p_i_wins = expit(true_alpha[i] - true_alpha[j])
            winner = i if rng.random() < p_i_wins else j
            loser  = j if winner == i else i
            comparisons.append((winner, loser))

print(f"Total comparisons: {len(comparisons)}")

def neg_log_likelihood(alpha, comparisons):
    """Bradley-Terry negative log-likelihood."""
    alpha = np.array(alpha)
    loss = 0.0
    for winner, loser in comparisons:
        log_prob = np.log(expit(alpha[winner] - alpha[loser]) + 1e-10)
        loss -= log_prob
    return loss

def neg_grad(alpha, comparisons):
    alpha = np.array(alpha)
    grad = np.zeros(len(alpha))
    for winner, loser in comparisons:
        p = expit(alpha[winner] - alpha[loser])
        grad[winner] -= (1 - p)
        grad[loser]  -= (-p)
    return grad

# Fix alpha[0] = 0 to resolve scale ambiguity
result = minimize(
    lambda a: neg_log_likelihood(np.concatenate([[0.0], a]), comparisons),
    x0=np.zeros(N_ITEMS - 1),
    jac=lambda a: neg_grad(np.concatenate([[0.0], a]), comparisons)[1:],
    method='L-BFGS-B'
)
est_alpha = np.concatenate([[0.0], result.x])

# Compare true vs estimated ranking
true_rank = np.argsort(true_alpha)[::-1]
est_rank  = np.argsort(est_alpha)[::-1]

print(f"\nTrue ranking (item indices): {true_rank}")
print(f"Estimated ranking:           {est_rank}")
rank_corr = np.corrcoef(true_alpha, est_alpha)[0, 1]
print(f"Correlation with true scores: {rank_corr:.4f}")
```

Модель Тёрстоуна

Модель Тёрстоуна {cite}`thurstone1927law` тесно связана с Брэдли–Терри, но использует гауссовский, а не логистический шум:

$$P(i \succ j) = \Phi\left(\frac{\alpha_i - \alpha_j}{\sqrt{2}\sigma}\right)$$

где Φ — стандартная нормальная функция распределения. При $\sigma = 1/\sqrt{2}$ это становится эквивалентным BT с незначительным масштабным отличием. На практике обе модели дают почти идентичные результаты.

Агрегирование ранжирований

Когда каждый аннотатор предоставляет полное ранжирование K элементов (а не попарные сравнения), задача называется **агрегированием ранжирований**: объединить несколько ранжированных списков в консенсусное ранжирование.

Метод Борда: Каждый элемент получает оценку, равную количеству элементов, стоящих ниже него в ранжировании каждого аннотатора. Оценки суммируются по аннотаторам. Прост и надёжен.

Kemeny–Young: Найти ранжирование, минимизирующее сумму попарных расхождений (расстояние Кендалла) с ранжированием каждого аннотатора. NP-трудная задача для больших K, но решаемая для небольших наборов.

RankNet / ListNet: Нейронные подходы, обучающие функцию оценки на ранжированных списках, позволяя обобщаться на невиданные элементы.

Дуэльные бандиты

В **онлайн**-обучении предпочтениям элементы поступают потоком и необходимо решать, какие пары сравнивать, балансируя исследование (изучение неизвестных элементов) и использование (представление высококачественных элементов). Это задача **дуэльных бандитов** {cite}`yue2009interactively`.

Ключевые алгоритмы: - **Doubler:** Поддерживает чемпионский элемент; бросает ему вызов случайными конкурентами - **RUCB (Relative Upper Confidence Bound):** Вычисляет доверительные интервалы в стиле UCB для вероятности каждого элемента превзойти каждый другой - **MergeRank:** Сочетает турнирное сравнение с UCB

Дуэльные бандиты используются в онлайн-рекомендательных системах (какую статью показать следующей, с учётом неявной обратной связи) и в интерактивном извлечении предпочтений для сбора данных RLHF.

Обучение предпочтениям для языковых моделей

В контексте RLHF (глава 6) модель Брэдли–Терри используется для обучения модели вознаграждения. Важным вариантом является **Прямая оптимизация предпочтений (DPO)** {cite}`rafailov2023direct`, которая показывает, что цель RLHF можно оптимизировать непосредственно на данных предпочтений без обучения отдельной модели вознаграждения:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta\log\frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right]$$

DPO проще, чем полный RLHF (без цикла обучения PPO, без модели вознаграждения), достигая сравнимых или лучших результатов на многих бенчмарках {cite}`rafailov2023direct`. Он стал широко применяемой альтернативой PPO-based RLHF для дообучения следования инструкциям, хотя оба подхода остаются в активном использовании и их относительные преимущества зависят от задачи.

Сбор высококачественных данных предпочтений

Качество данных предпочтений определяет качество модели вознаграждения. Ключевые соображения:

Разнообразие подсказок. Данные предпочтений должны охватывать полное распределение подсказок, с которыми модель столкнётся при развёртывании. Пробелы в охвате приводят к ненадёжному поведению модели вознаграждения в этих областях.

Разнообразие ответов. Сравнение двух очень похожих ответов даёт мало информации. Сравниваемые ответы должны достаточно различаться, чтобы аннотаторы имели чёткое предпочтение.

Согласие аннотаторов. Низкое межаннотаторское согласие свидетельствует о том, что критерии сравнения неоднозначны. Измеряйте согласие (каппа Коэна) и пересматривайте инструкции, когда оно ниже допустимых порогов.

Калибровка. Аннотаторы должны понимать, *почему* один ответ лучше: по полезности, точности, безопасности, стилю? Задачи, объединяющие несколько критериев, как правило, дают непоследовательные предпочтения. Нередко лучше собирать предпочтения отдельно по каждому критерию.

> **See also:** Модель Брэдли–Терри: {cite}`bradley1952rank`. Тёрстоун: {cite}`thurstone1927law`. Дуэльные бандиты: {cite}`yue2009interactively`. Прямая оптимизация предпочтений (DPO): {cite}`rafailov2023direct`.

HITL в обработке естественного языка

Естественный язык — область, где HITL ML оказало, пожалуй, наибольшее влияние, и где обнажаются его наиболее глубокие концептуальные трудности. Язык по своей природе социален: его смысл конструируется человеческими сообществами, его прагматика зависит от контекста и намерений, а его качество может оцениваться только читателями-людьми. Но это также означает, что аннотирование в NLP — это не просто процесс сбора наблюдений. Это процесс *конструирования* категорий.

Конститутивная проблема в аннотировании NLP

В медицинской визуализации существует истина: опухоль есть или её нет. Метка радиолога может быть неопределённой, но она стремится фиксировать нечто реальное. Аннотирование NLP нередко принципиально иное. Когда аннотатор помечает твит как «токсичный», в твите нет молекулы токсичности, которую мы пытаемся обнаружить. **Метка конституирует категорию.**

Это имеет глубокие последствия, которые часто недооцениваются:

Аннотаторы определяют само явление. Схема меток для «оскорбительной речи» кодирует чувствительность тех, кто её разработал, и тех, кто её применял. Команда англоязычных аннотаторов из единой демографической группы, работающих по инструкциям, написанным командой корпоративной политики, создаёт датасет, отражающий понимание оскорбления этой командой — а не некий универсальный человеческий стандарт. Модели, обученные на таких данных, будут демонстрировать те же имплицитные границы.

Инструкции — это теория, осознают они это или нет. Каждая схема аннотирования делает онтологические утверждения. Решение считать «иронию» и «сарказм» одним классом — это теоретическое утверждение, а не нейтральное удобство. Решение помечать «гнев» как единый класс, не различая «праведный гнев» и «враждебный гнев», сворачивает различие, которое может быть важным для последующей задачи. Эти решения принимаются под производственным давлением и редко пересматриваются.

Нестабильность меток во времени. Социальный язык эволюционирует. Модель токсичности, обученная в 2018 году, будет неверно классифицировать контент 2024 года не потому, что статистически недообучена, а потому что социальный смысл определённых терминов изменился. Аннотирование NLP — это не выборка из статичного распределения, а выборка из движущейся цели, которую акт разметки частично помогает конституировать.

:::{admonition} Проблема артефактов аннотирования :class: important

Гева и др. {cite}`geva2019annotator` продемонстрировали, что датасеты NLI (вывода в естественном языке) содержат систематические артефакты, введённые самим процессом аннотирования. Аннотаторы, которых просят написать гипотезы «импликации» для данной предпосылки, склонны использовать определённые синтаксические паттерны; аннотаторы, пишущие гипотезы «противоречия» — другие. Модели учатся классифицировать на основе этих артефактов, а не предполагаемых семантических отношений — они решают задачу аннотирования, а не лежащую в основе задачу NLP.

Это не небрежность. Это неизбежное следствие того, что люди конструируют примеры под метки. Процесс HITL вносит систематический сигнал, который никогда не предназначался попасть в данные. :::

Аннотирование текстовой классификации

Простейшая задача аннотирования NLP — присвоение категории текстовому документу. Анализ тональности, тематическая классификация, обнаружение намерений и фильтрация спама — всё это задачи классификации.

Специфические проблемы текстовой классификации:

Субъективность. Категории вроде «токсичного» или «оскорбительного» по своей природе субъективны и варьируются в зависимости от культурного контекста, демографии аннотаторов и платформного контекста. Восприятие оскорбительности у разных аннотаторов значительно различается в зависимости от демографии — факт с прямыми последствиями для справедливости {cite}`blodgett2020language`.

Неоднозначность меток. Многие документы принадлежат нескольким категориям или находятся на границе. Отзыв, на 60% позитивный и на 40% негативный, является по-настоящему неоднозначным, а не неверно размеченным. Принудительная одна метка отбрасывает реальную информацию (см. главу 13 о мягких метках и несогласии аннотаторов).

Гранулярность меток. Двухклассовая модель тональности может быть достаточной для мониторинга социальных сетей; для анализа обратной связи о продукте может понадобиться 7-балльная шкала. Правильная гранулярность зависит от последующей задачи, но обычно фиксируется до аннотирования — это существенное проектное решение, принимаемое с недостаточными данными.

Распознавание именованных сущностей

Аннотирование NER требует выявления участков текста и присвоения типа сущности (PERSON, ORGANIZATION, LOCATION и т. д.). Аннотирование сложнее, чем классификация документов, по нескольким причинам:

Границы промежутков неоднозначны. В «Apple CEO Tim Cook объявил…», какую сущность PERSON охватывать — «Tim Cook» или «Apple CEO Tim Cook»? Инструкции должны явно рассматривать такие случаи, и межаннотаторское согласие по промежуткам неизменно ниже, чем по типам.

Присвоение типов требует знаний предметной области. «Apple» — ORG в одном контексте, PRODUCT в другом, и ни то, ни другое в «яблочном пироге». Аннотаторам нужны достаточные знания предметной области для правильного присвоения типов.

Вложенные сущности. «Калифорнийский университет в Беркли» содержит ORGANIZATION, вложенную в LOCATION. Стандартное BIO-тегирование не может представлять вложенные сущности; требуются более сложные схемы (например, BIOES или форматы на основе графов).

Эффективность аннотирования. Предварительное аннотирование с помощью базовой модели NER значительно ускоряет аннотирование, позволяя аннотаторам исправлять предсказания, а не аннотировать с нуля. В одном исследовании клинического NER наблюдался прирост пропускной способности на 30–60% {cite}`lingren2014evaluating`; масштаб таких выгод во многом зависит от качества базовой модели и области применения.

Извлечение отношений и семантическое аннотирование

Помимо идентификации сущностей, извлечение отношений требует аннотирования *взаимосвязей* между сущностями:

- Аннотаторы должны понимать как сущности, так и предикат, соединяющий их
- Типы отношений имеют сложные семантические различия (WORKS_AT vs. EMPLOYED_BY vs. FOUNDED)
- Отрицательные примеры (пары сущностей без отношения) значительно более распространены, чем положительные

Межаннотаторское согласие при извлечении отношений ниже, чем при задачах классификации. Для хорошо определённых схем значения κ в диапазоне 0,65–0,80 широко сообщаются {cite}`pustejovsky2012natural`; для сложных семантических отношений (событийная причинность, временные отношения) согласие может значительно снизиться в зависимости от схемы и подготовки аннотаторов.

Постредактирование машинного перевода (MTPE)

Постредактирование машинного перевода — зрелая форма HITL NLP. Переводчик-человек исправляет выход системы МП, а не переводит с нуля:

Лёгкое постредактирование (LPE): Исправляются только критические ошибки. Подходит, когда требования к качеству перевода умеренные.

Полное постредактирование (FPE): Выход доводится до публикационного качества. Отредактированный выход, как правило, становится обучающими данными для дальнейшего улучшения МП — подлинный цикл HITL-совершенствования.

HTER (Human-targeted Translation Edit Rate): Метрика, измеряющая расстояние редактирования между выходом МП и постотредактированным переводом, нормализованное по длине предложения {cite}`graham2015accurate`. Как приблизительное практическое правило, HTER ниже ~0,3 часто считается хорошим выходом МП; выше 0,5 перевод с нуля может быть быстрее — хотя эти пороги варьируются в зависимости от области и языковой пары.

Диалоговый ИИ и аннотирование диалогов

Аннотирование диалогов вводит временну́ю структуру:

- **Аннотирование на уровне хода:** разметка каждого хода (намерение, тональность, качество)
- **Аннотирование на уровне диалога:** оценка общей связности, успеха задачи, удовлетворённости пользователя
- **Аннотирование трассы взаимодействия:** выявление конкретных моментов сбоя в разговоре

HITL особенно важен в диалоге, поскольку сбои модели часто тонкие и накопительные: фактическая ошибка в ходе 3 может не стать очевидной до хода 7. Аннотаторы-люди, прослеживающие разговоры, способны выявлять такие долгосрочные паттерны сбоев, которые автоматические метрики полностью упускают.

Программная разметка и слабый надзор

Когда размеченных данных мало, **слабый надзор** позволяет использовать несколько зашумлённых, перекрывающихся функций разметки для генерации вероятностных меток в масштабе. **Snorkel** {cite}`ratner2017snorkel` является канонической рамкой:

```
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score

rng = np.random.default_rng(42)

# ---------------------------------------------------------
# Toy weak supervision: sentiment classification
# 5 labeling functions (LFs) with different coverage/accuracy
# Label: +1 (positive), -1 (negative), 0 (abstain)
# ---------------------------------------------------------

N = 1000
true_labels = rng.choice([-1, 1], size=N)
X_features = np.column_stack([
    true_labels * 0.8 + rng.normal(0, 0.5, N),
    rng.normal(0, 1, N),
    rng.normal(0, 1, N),
])

def make_lf(accuracy, coverage, seed):
    rng_lf = np.random.default_rng(seed)
    votes = np.zeros(N, dtype=int)
    active = rng_lf.random(N) < coverage
    correct = rng_lf.random(N) < accuracy
    votes[active & correct]  = true_labels[active & correct]
```

```
    votes[active & ~correct] = -true_labels[active & ~correct]
    return votes

LFs = np.column_stack([
    make_lf(accuracy=0.85, coverage=0.60, seed=1),
    make_lf(accuracy=0.78, coverage=0.45, seed=2),
    make_lf(accuracy=0.70, coverage=0.80, seed=3),
    make_lf(accuracy=0.90, coverage=0.30, seed=4),
    make_lf(accuracy=0.60, coverage=0.90, seed=5),
])

def majority_vote(LF_matrix):
    labels = []
    for i in range(LF_matrix.shape[0]):
        votes = LF_matrix[i][LF_matrix[i] != 0]
        labels.append(0 if len(votes) == 0 else int(np.sign(votes.sum())))
    return np.array(labels)

mv_labels = majority_vote(LFs)
covered = mv_labels != 0

print(f"Coverage:                    {covered.mean():.1%}")
print(f"MV accuracy (covered):       {(mv_labels[covered] == true_labels[covered]).mean():.3f

X_train, y_train = X_features[covered], mv_labels[covered]
X_test,  y_test  = X_features[~covered], true_labels[~covered]

if len(X_train) > 10 and len(X_test) > 10:
    clf = LogisticRegression().fit(X_train, y_train)
    preds = clf.predict(X_test)
    print(f"F1 on uncovered test set:    {f1_score(y_test, preds):.3f}")
```

RLHF для языковых моделей: реальность аннотирования

Глава 6 охватила RLHF технически. С точки зрения NLP задача аннотирования сложнее, чем кажется снаружи.

Что аннотаторов просят делать в действительности — сравнивать два вывода модели и указывать, какой «лучше», — звучит просто. На практике «лучше» — неопределённый конструкт, который аннотаторы разрешают с помощью личных эвристик. Одни придают большой вес беглости; другие — фактической точности; третьи штрафуют за многословие. Без жёстких инструкций полученный датасет предпочтений отражает не человеческие ценности в абстракции, а конкретные стратегии разрешения применяемой аннотационной рабочей силы.

Ключевые измерения аннотирования:

- *Полезность:* Хорошо ли ответ отвечает на вопрос? Является ли он информативным, ясным и достаточно подробным?
- *Фактологичность:* Является ли ответ фактически точным? Это требует от оценщиков экспертных знаний — условие, создающее серьёзные проблемы

качества в масштабе, поскольку обобщённые аннотаторы не могут верифицировать специализированные утверждения.
- *Безвредность:* Избегает ли ответ токсичного, предвзятого, вредоносного или неуместного контента? Эти суждения требуют подробных инструкций, поскольку «вредное» сильно зависит от контекста и варьируется в зависимости от культуры, времени и платформного контекста.
- *Калиброванность:* Выражает ли ответ адекватную неопределённость? Аннотаторы систематически предпочитают ответы, звучащие уверенно, что создаёт сигнал вознаграждения против уместной эпистемической скромности.

Взаимодействие критериев сложно: максимально полезный ответ на опасный вопрос может быть максимально вредным. Инструкции должны определять, как соотносить конкурирующие критерии, — а эти соотношения являются, по существу, политическими решениями, а не аннотационными. Аннотаторы принимают политические решения.

Масштаб концентрирует демографическое влияние. RLHF для больших моделей включает относительно небольшие аннотационные рабочие силы (сотни — тысячи человек), принимающие миллиарды последующих решений. Демографические и культурные предвзятости этой рабочей силы распространяются в поведение модели в масштабе так, как это не происходило бы при более распределённом аннотировании. Это одна из наименее обсуждаемых структурных проблем в текущем конвейере RLHF.

Цикл обратной связи аннотирование–модель

В NLP больше, чем в любой другой области, процессы аннотирования и разработки моделей со временем переплетаются:

1. Аннотаторы калибруются с использованием существующих выходов модели в качестве референса (часто неявно).
2. Модель вознаграждения обучается тому, что аннотаторы склонны предпочитать.
3. Генератор дообучается для производства выходов, получающих высокое вознаграждение.
4. Эти выходы влияют на то, как выглядит «хорошее» в последующих раундах аннотирования.

Этот цикл не является по сути патологическим — он обеспечивает сходимость RLHF, — но он означает, что поведение модели формируется движущейся целью, которую сам процесс аннотирования помогает двигать. Эмпирически трудно различить, что модель научилась отражать человеческие предпочтения и что она научилась сопоставлять паттерны эвристик аннотаторов.

Чистого решения нет. Лучшая текущая практика — следить за дрейфом с помощью отложенных суждений предпочтения, собранных через регулярные интервалы, и рассматривать версию инструкций по аннотированию как экспериментальную переменную.

Оценка генеративных моделей NLP

В отличие от классификационных моделей с чёткой метрикой точности, оценка качества генерации требует человеческого суждения:

Метод оценки	Описание	Стоимость
Прямая оценка (DA)	Аннотаторы оценивают качество по шкале	Средняя
Сравнительное суждение	Аннотаторы сравнивают два вывода	Низкая
MQM (многомерная метрика качества)	Структурированная таксономия ошибок	Высокая
Предпочтение RLHF	Метки предпочтений для обучения	Средняя
LLM как оценщик	LLM оценивает выходы (умеренная корреляция с людьми)	Очень низкая
BERTScore, BLEU	Автоматические метрики (несовершенная корреляция)	Очень низкая

Автоматические метрики (BLEU для МП, ROUGE для суммаризации) дёшевы, но несовершенно коррелируют с человеческими оценками качества {cite}`reiter2018structured`. Подходы «LLM как оценщик» показывают умеренное согласие с аннотаторами-людьми {cite}`zheng2023judging` и всё шире применяются для быстрых итераций, однако не должны заменять человеческую оценку для окончательных выводов. Для решений с реальными ставками человеческая оценка остаётся необходимой.

> **See also:** Слабый надзор Snorkel: {cite}`ratner2017snorkel`. Инструкции по аннотированию NLP: {cite}`pustejovsky2012natural`. Артефакты аннотирования в NLI: {cite}`geva2019annotator`. Предвзятость аннотаторов и датасеты NLP: {cite}`blodgett2020language`. Оценка генеративных моделей: {cite}`reiter2018structured`.

HITL в компьютерном зрении

Компьютерное зрение предоставляет одни из наиболее наглядных примеров HITL ML. Соревнование ImageNet, построенное на 14 миллионах изображений, размеченных людьми, дало импульс эпохе глубокого обучения. Аннотирование медицинских изображений радиологами обеспечивает диагностический ИИ. Автономные транспортные средства зависят от миллионов кадров, размеченных людьми, для обучения восприятию мира.

Что легко упустить: это не просто случаи предоставления людьми истины в последней инстанции. Это случаи, когда люди конструируют датасеты, воплощающие конкретные перцептивные, культурные и операциональные решения — решения, которые обнаруживаются лишь позже, когда модели дают сбой предсказуемым образом.

Как решения при аннотировании превращаются в предвзятости модели

Стандартная трактовка рассматривает аннотирование как сбор данных: люди наблюдают мир и записывают увиденное. Более точная трактовка — аннотирование есть *проектирование датасета*: люди решают, какие категории использовать, где проводить границы, какие пограничные случаи включать и как разрешать неоднозначность, — и все эти решения определяют, что будет воспринимать обученная модель.

Случай ImageNet

ImageNet {cite}`russakovsky2015imagenet` — наиболее значимый датасет в истории компьютерного зрения. Его набор меток происходит из WordNet-синсетов, выбранных главным образом за их многочисленность и семантическую различимость. Несколько последствий этого проектного решения проявились спустя годы:

- **Категории людей кодировали демографические ассоциации.** Ранние версии синсет-меток ImageNet для людей включали многие из тех, что ныне считались бы уничижительными или предвзятыми, отражая как исторический источник WordNet, так и имплицитные решения аннотаторов о том, какие метки применять к каким изображениям {cite}`yang2020towards`. Метки, применённые к изображениям людей, кодировали ассоциации, связанные с расой, полом и классом, которые непосредственно распространились в вложения моделей.
- **Мелкозернистая таксономия видов, грубая таксономия почти всего остального.** ImageNet умеет различать 120 пород собак, но сворачивает огромное разнообразие инструментов, транспортных средств, еды и мебели в единые категории. Это было следствием следования структуре WordNet, а не намеренным

решением о том, что важно. Модели, обученные на ImageNet, демонстрируют ту же асимметричную точность.

- **Западные, англоязычные визуальные умолчания.** Изображения были собраны главным образом из Flickr и Интернет-поиска с использованием англоязычных запросов. Полученное распределение сильно смещено в сторону визуальной среды и культурных объектов богатых, англоязычных стран.

Ни одно из этих обстоятельств не было ошибкой. Это были проектные решения при аннотировании, принятые быстро и в масштабе, нередко людьми, не предвидевшими, как датасет будет использоваться. Вывод не в том, что ImageNet следовало строить иначе (хотя это так), а в том, что **проектирование аннотирования есть проектирование модели**, и к нему следует подходить с той же тщательностью.

:::{admonition} Схема аннотирования — это теория мира :class: note

Каждая таксономия меток делает утверждения о том, какие различия важны. Выбор разделять «легковой автомобиль» и «грузовик», сворачивая при этом все седаны в один класс, — теоретическое утверждение о семантической релевантности различий. Выбор аннотировать «человека» как единый класс независимо от позы, одежды или деятельности — другое теоретическое утверждение. Модели, обученные на таких схемах, будут проводить те же различия — и никаких других: они не будут обобщаться за пределы категорий, на которые их обучили различать. :::

Аннотирование классификации изображений

Простейшая задача аннотирования CV — присвоение одной или нескольких меток целому изображению.

Иерархия меток. Метка «собака» является потомком «животного» в семантической иерархии. Модели, обученные на плоских таксономиях, могут плохо обобщаться на разных уровнях абстракции. ImageNet использует синсет-иерархию, допускающую оценку на нескольких уровнях конкретности.

Многометочная неоднозначность. Городская сцена может одновременно содержать автомобиль, человека, велосипед и светофор. Решение о том, какие метки включать, требует чётких инструкций о порогах релевантности.

Распределения с длинным хвостом. Естественные датасеты изображений следуют степенному закону: несколько категорий встречаются очень часто; большинство — редко. Активное обучение особенно ценно для категорий с длинным хвостом, где случайная выборка даёт лишь горстку примеров.

Обнаружение объектов: аннотирование ограничивающими рамками

Обнаружение объектов требует от аннотаторов рисования прямоугольных рамок, выровненных по осям, вокруг экземпляров каждого класса объектов. Это вводит требования к геометрической точности и значительное количество пограничных случаев.

Метрики качества аннотирования:

IoU (пересечение над объединением) измеряет перекрытие между аннотированной рамкой и эталонной рамкой:

$$\mathrm{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

IoU $\geq$ 0,5 является стандартным порогом для «правильного» обнаружения в PASCAL VOC; COCO {cite}`lin2014microsoft` использует диапазон порогов от 0,5 до 0,95, что значительно требовательнее и информативнее.

Пограничные случаи аннотирования, которые должны быть разрешены в инструкциях: - Перекрытые объекты: аннотировать видимую часть или экстраполировать полный объём? - Усечённые объекты (частично выходящие за пределы кадра): включать или исключать? - Зоны скопления: использовать специальную аннотацию «толпа» или аннотировать отдельные экземпляры?

Каждое из этих решений меняет смысл «правильного обнаружения» — и, следовательно, то, чему обучается модель.

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as patches

def compute_iou(boxA, boxB):
    """Compute IoU between two boxes [x1, y1, x2, y2]."""
    xA = max(boxA[0], boxB[0])
    yA = max(boxA[1], boxB[1])
    xB = min(boxA[2], boxB[2])
    yB = min(boxA[3], boxB[3])
    inter_area = max(0, xB - xA) * max(0, yB - yA)
    boxA_area = (boxA[2] - boxA[0]) * (boxA[3] - boxA[1])
    boxB_area = (boxB[2] - boxB[0]) * (boxB[3] - boxB[1])
    union_area = boxA_area + boxB_area - inter_area
    return inter_area / (union_area + 1e-6)

ref_box  = [1.0, 1.0, 4.0, 4.0]
ann1_box = [1.1, 0.9, 4.1, 4.2]   # close
ann2_box = [0.5, 0.5, 3.5, 3.8]   # less precise

print(f"IoU (ann1 vs ref):  {compute_iou(ann1_box, ref_box):.3f}")
print(f"IoU (ann2 vs ref):  {compute_iou(ann2_box, ref_box):.3f}")
print(f"IoU (ann1 vs ann2): {compute_iou(ann1_box, ann2_box):.3f}")

fig, ax = plt.subplots(figsize=(5, 5))
ax.set_xlim(0, 5); ax.set_ylim(0, 5); ax.set_aspect('equal')
ax.add_patch(patches.Rectangle(
    (ref_box[0], ref_box[1]), ref_box[2]-ref_box[0], ref_box[3]-ref_box[1],
    linewidth=2.5, edgecolor='#2b3a8f', facecolor='none', label='Reference'))
ax.add_patch(patches.Rectangle(
    (ann1_box[0], ann1_box[1]), ann1_box[2]-ann1_box[0], ann1_box[3]-ann1_box[1],
    linewidth=2, edgecolor='#0d9e8e', facecolor='none', linestyle='--',
```

```
    label=f'Ann1 (IoU={compute_iou(ann1_box, ref_box):.2f})'))
ax.add_patch(patches.Rectangle(
    (ann2_box[0], ann2_box[1]), ann2_box[2]-ann2_box[0], ann2_box[3]-ann2_box[1],
    linewidth=2, edgecolor='#e05c5c', facecolor='none', linestyle=':',
    label=f'Ann2 (IoU={compute_iou(ann2_box, ref_box):.2f})'))
ax.legend(fontsize=10)
ax.set_title("Bounding Box Agreement (IoU)", fontsize=12)
plt.tight_layout()
plt.savefig('bbox_iou.png', dpi=150)
plt.show()
```

Семантическая и инстанционная сегментация

Аннотирование сегментации требует присвоения метки класса каждому пикселю изображения — один из самых дорогостоящих видов аннотирования.

Семантическая сегментация: Каждый пиксель принадлежит семантическому классу (дорога, небо, человек, автомобиль). Все пиксели одного класса имеют одну метку, независимо от того, к какому отдельному объекту они принадлежат.

Инстанционная сегментация: Каждый отдельный экземпляр объекта получает уникальную метку. Толпа из 20 человек становится 20 различными масками.

Паноптическая сегментация: Сочетает оба подхода: классы «вещей» (считаемые объекты) имеют маски экземпляров; классы «материй» (дорога, небо) имеют семантические маски.

Аннотирование с помощью SAM: Модель Segment Anything от Meta {cite}`kirillov2023segment` генерирует высококачественные маски сегментации по одному щелчку в точке. Аннотаторы щёлкают по центру объекта; SAM предлагает маску; аннотатор принимает или корректирует. Авторы SAM сообщают об ускорении процесса аннотирования примерно в 6,5 раза по сравнению с полигональной разметкой; выгоды варьируются в зависимости от типа сцены и инструментов.

SAM представляет более широкий сдвиг: **роль аннотатора меняется от рисования к проверке**. Это имеет последствия для качества аннотирования, выходящие за рамки скорости. Когда аннотаторы рисуют, их внимание задействовано на протяжении всего процесса. Когда аннотаторы проверяют и нажимают «принять», есть данные, что они пропускают ошибки чаще — версия автоматизационной предвзятости, специфичная для контекста аннотирования.

Активное обучение для компьютерного зрения

Активное обучение особенно ценно в CV, поскольку: 1. Изображения многомерны и богаты признаками — вложения от предобученных моделей несут сильные сигналы для оценки неопределённости 2. Большие неразмеченные пулы дёшевы (фотографии, кадры видео) 3. Аннотирование (особенно сегментация) дорого и не может быть легко отдано на краудсорсинг для специализированных областей

```python
import numpy as np
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

rng = np.random.default_rng(42)

X, y = make_classification(n_samples=3000, n_features=50, n_informative=25,
                           n_classes=5, n_clusters_per_class=2, random_state=42)
X_train, y_train = X[:2500], y[:2500]
X_test,  y_test  = X[2500:], y[2500:]

def margin_uncertainty(model, X_pool):
    probs = model.predict_proba(X_pool)
    sorted_p = np.sort(probs, axis=1)
    return sorted_p[:, -2] - sorted_p[:, -1]  # most negative = most uncertain

n_init = 50
results = {'active': [], 'random': []}
label_counts = list(range(50, 401, 30))

for strategy in ['active', 'random']:
    labeled = list(rng.choice(len(X_train), n_init, replace=False))
    unlabeled = [i for i in range(len(X_train)) if i not in labeled]

    for target in label_counts:
        while len(labeled) < target and unlabeled:
            if strategy == 'active' and len(labeled) >= 10:
                model_temp = LogisticRegression(max_iter=300).fit(
                    X_train[labeled], y_train[labeled])
                margins = margin_uncertainty(model_temp, X_train[unlabeled])
                idx = int(np.argmin(margins))
            else:
                idx = rng.integers(0, len(unlabeled))
            labeled.append(unlabeled.pop(idx))

        clf = LogisticRegression(max_iter=300).fit(X_train[labeled], y_train[labeled])
        results[strategy].append(accuracy_score(y_test, clf.predict(X_test)))

plt.figure(figsize=(7, 4))
plt.plot(label_counts, results['active'], 'o-', color='#2b3a8f',
         linewidth=2, label='Active (margin sampling)')
plt.plot(label_counts, results['random'], 's--', color='#e05c5c',
         linewidth=2, label='Random baseline')
plt.xlabel("Labeled training images", fontsize=12)
plt.ylabel("Test accuracy", fontsize=12)
plt.title("Active Learning for 5-Class Image Classification", fontsize=13)
plt.legend(fontsize=11); plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig('cv_active_learning.png', dpi=150)
```

```
plt.show()
```

Полуконтролируемое обучение с человеческим руководством

Большое количество доступных неразмеченных визуальных данных делает полуконтролируемое обучение особенно привлекательным для CV.

Самообучение/псевдоразметка: Обучить модель на размеченных данных; использовать высокоуверенные предсказания на неразмеченных данных как псевдометки; переобучить. Критически важный вопрос проектирования — порог уверенности. Низкий порог подключает больше примеров, но вводит шум; высокий порог оставляет большую часть неразмеченного пула неиспользованной. Участие человека может направлять этот порог — аннотаторы проверяют выборку псевдоразмеченных примеров для его калибровки.

FixMatch и регуляризация согласованности: Эти методы обучают модели производить согласованные предсказания при аугментации. Ключевой HITL-вывод: люди участвуют не только для создания меток, но и для **проектирования аугментации** — какие инвариантности должна изучать модель? Модель для медицинской визуализации должна быть инвариантна к поворотам и яркости, но не к масштабу; модель для обнаружения текста не должна быть инвариантна к перспективному искажению. Эти зависящие от области выборы требуют человеческой экспертизы, и неверный выбор существенно ухудшает полуконтролируемое обучение.

Активное полуконтролируемое обучение: Наиболее эффективное сочетание: активное обучение концентрирует человеческие метки там, где неопределённость модели наибольшая; самообучение автоматически размечает высокоуверенный хвост. Человеческий труд сосредотачивается там, где он наиболее ценен, а модель самостоятельно загружается на остальном. На каждом цикле проверка случайной выборки псевдометок человеком обеспечивает контроль качества без полного просмотра.

Аннотирование видео

Аннотирование видео умножает трудности аннотирования изображений на временно́е измерение:

- **Отслеживание:** Объекты должны быть идентифицированы в кадрах. Аннотаторы размечают ключевые кадры; алгоритмы отслеживания интерполируют между ними. Сбои отслеживания — перекрытие, повторное появление, быстрое движение — требуют повторной разметки человеком с более высокой частотой, чем стационарное отслеживание.
- **Временна́я согласованность:** Границы, нарисованные в кадре t, должны быть пространственно согласованы с кадром $t + 1$. Непоследовательные аннотации — это обучающий сигнал, сообщающий модели, что объекты прыгают разрывно, — форма шума аннотирования, особенно разрушительная для моделей обнаружения.
- **Масштабируемость:** Часовое видео при 30 кадрах/с составляет 108 000 кадров. Полное аннотирование непрактично; стратегии выборки должны быть тщательно спроектированы, чтобы редкие события (пограничные случаи, почти-инциденты, сценарии отказов) систематически не исключались.

Современные инструменты аннотирования видео поддерживают **интеллектуальное отслеживание**, распространяющее аннотации по кадрам и помечающее кадры, где уверенность отслеживания падает ниже порогового значения, побуждая аннотатора перепроверить. Это прямое применение идеи активного обучения к конвейеру аннотирования: инструмент запрашивает у аннотатора именно там, где его интерполяция неопределённа.

Проблема редких событий в автономных системах. Для приложений, где последствия редких событий катастрофичны — автономное вождение, управление БПЛА, — распределение кадров при нормальной эксплуатации плохо соответствует распределению наиболее важных кадров. Датасет, построенный путём равномерной выборки кадров вождения, будет содержать миллионы кадров «ничего интересного не происходит» и горстку кадров почти-аварий, необычного освещения и деградировавших датчиков, которые действительно важны для безопасности. HITL активное обучение, выявляющее и приоритизирующее такие кадры, — это не оптимизация эффективности; это требование безопасности.

> **See also:** Датасет ImageNet: {cite}`russakovsky2015imagenet`. Предвзятость меток в ImageNet: {cite}`yang2020towards`. Бенчмарк COCO: {cite}`lin2014microsoft`. SAM (Segment Anything): {cite}`kirillov2023segment`. Активное обучение «ключевые множества» для CV: {cite}`sener2018active`.

HITL в здравоохранении и науке

Здравоохранение и наука представляют собой две области, где HITL ML является наиболее значимым и наиболее дискуссионным. Ставки высоки: пропущенный диагноз рака или ошибочная мишень для лекарства несёт реальные человеческие потери. Аннотирование требует редкой и дорогостоящей экспертизы. Нормативные требования ограничивают возможности и порядок валидации моделей. И в отличие от NLP, где проблема аннотирования частично является проблемой социального конструирования, здесь нередко существует подлинная истина — опухоль есть или её нет, — даже если ни один отдельный наблюдатель не может надёжно её определить.

Господствующая трактовка в массовой прессе — «ИИ против человека»: вытеснит ли ИИ радиологов? Эта трактовка неверна существенным образом. Реальный вопрос состоит в том, какая форма сотрудничества человека и ИИ даёт лучшие результаты, чем каждая из сторон по отдельности, и как строить системы, обеспечивающие это сотрудничество, а не разрушающие его.

Анализ медицинских изображений

Медицинская визуализация — радиология (рентген, КТ, МРТ), патология (гистологические препараты), дерматология, офтальмология — область, где медицинский ИИ продвинулся наиболее быстро.

Требования к экспертам-аннотаторам

Аннотирование медицинских изображений, как правило, требует врачей с узкоспециализированной подготовкой. Это делает аннотирование:

- **Медленным:** У специалистов ограниченное время; аннотирование конкурирует с клиническими обязанностями
- **Дорогостоящим:** Стоимость варьируется от нескольких десятков до нескольких сотен долларов за аннотированный случай в зависимости от подспециальности, модальности и сложности задачи
- **Вариабельным:** Даже специалисты расходятся, особенно в пограничных случаях — факт, который нередко воспринимается как проблема, однако на самом деле является информативным

Вариабельность показателей радиологов

Вариабельность показателей хорошо задокументирована в радиологии. Для интерпретации рентгенограмм грудной клетки межчитательское несогласие существенно — в исследовании CheXNet четыре радиолога разметили один и тот же тестовый набор для обнаружения

пневмонии с результатами F1, различающимися примерно на 12 процентных пунктов {cite}`rajpurkar2017chexnet`, что отражает подлинную диагностическую неопределённость в пограничных случаях. Для обнаружения узлов на КТ лёгких интрачитательская вариабельность (один читатель, один случай, разные дни) может быть столь же высокой, как и межчитательская.

Эта вариабельность — не просто шум; она отражает подлинную диагностическую неопределённость. Модели, обученные на аннотациях единственного радиолога, могут усваивать специфические предвзятости этого врача, а не лежащую в основе патологию.

:::{admonition} Полемика вокруг CheXNet как урок HITL :class: note

Когда Раджпуркар и др. заявили, что их модель CheXNet «превзошла производительность радиолога» при обнаружении пневмонии, это утверждение было немедленно оспорено сообществом радиологов {cite}`yu2022assessing`. Часть полемики касалась конкретного тестового набора и сравнения с радиологами. Но более глубокой проблемой была методологическая: «производительность радиолога», принятая в качестве базовой линии, использовала одиночных читателей под временны́м давлением, тогда как клиническая радиология, как правило, включает консультацию, сравнение с предыдущими снимками и доступ к клиническому контексту — ничего из чего у модели не было.

Урок не в том, была ли модель хорошей или плохой, а в том, что **сравнения производительности требуют спецификации конфигурации HITL**. Модель, превосходящая одиночного радиолога при холодном чтении, может всё же уступать радиологу, использующему выход модели как второе мнение. Это разные системы с разными режимами ошибок, и их агрегирование даёт разные результаты. :::

:::{admonition} Мягкие метки в медицине :class: important

Ряд проектов медицинского ИИ перешли к использованию **мягких меток**, отражающих распределение мнений экспертов, а не единственный «золотой стандарт» метки. Рентгенограмма грудной клетки, размеченная как 60% пневмония / 40% ателектаз группой радиологов, несёт больше информации, чем вынужденный бинарный выбор. Модели, обученные на таких распределениях, демонстрируют лучшую калиброванность и более адекватную квантификацию неопределённости — и эта неопределённость клинически значима, поскольку говорит клиницисту, когда следует проконсультироваться, а не только что думает модель. :::

Активное обучение для редких состояний

Активное обучение особенно ценно для редких заболеваний и редких патологий, где даже большой неразмеченный пул содержит лишь небольшое количество положительных случаев. Стандартная случайная выборка будет расходовать время эксперта главным образом на разметку отрицательных случаев.

Активное обучение на основе неопределённости естественным образом выбирает пограничные случаи, в которых модель неопределённа, — которые для редких состояний, как правило, являются положительными и пограничными отрицательными. Это концентрирует время специалиста там, где оно наиболее ценно. Сочетание несбалансированного обучения (`class_weight='balanced'` или аналогичное) и выбора на основе неопределённости является стандартной практикой для задач обнаружения редких патологий.

Аннотирование клинических NLP

Электронные медицинские записи (ЭМЗ) содержат огромный массив клинических нарративных текстов: врачебные заметки, выписные эпикризы, радиологические заключения, патологоанатомические заключения. Извлечение структурированной информации из этого текста требует NLP — а высококачественный NLP требует аннотированных обучающих данных.

Распространённые задачи аннотирования клинического NLP: - Клинический NER: Выявление лекарств, дозировок, диагнозов, процедур и симптомов в тексте **- Обнаружение отрицания:** «Нет признаков пневмонии» против «Пневмония подтверждена» — критическое различие, неожиданно трудное для автоматизации **- Временно́е рассуждение:** Различение текущих состояний от анамнеза («в анамнезе ИМ, поступил с болью в груди») **- Деидентификация:** Удаление защищённой медицинской информации (ЗМИ) для обеспечения возможности обмена данными

Деидентификация ЗМИ является одновременно задачей аннотирования и требованием управления данными. Согласно HIPAA (США) и GDPR (ЕС), медицинские данные не могут передаваться без удаления или анонимизации идентификаторов пациентов. Инструменты автоматической деидентификации существуют, но несовершенны; человеческий контроль автоматических выходов является стандартной практикой, а профиль рисков асимметричен: ложноотрицательные результаты (пропущенные ЗМИ) создают юридическую ответственность, что требует консервативных порогов.

i2b2 / n2c2 как шаблон

Инициативы совместных задач i2b2 (Informatics for Integrating Biology and the Bedside) и преемника n2c2 (National NLP Clinical Challenges) выпустили серию экспертно аннотированных датасетов клинического NLP. Они иллюстрируют как потенциал, так и стоимость: усилия по аннотированию, как правило, включают команды экспертов клинической предметной области, работающих на протяжении нескольких месяцев, аннотирующих сотни документов на задачу. Датасеты n2c2 катализировали быстрый прогресс именно потому, что решили проблему управления обменом данными (деидентификация + институциональные соглашения), а не только проблему аннотирования.

Нормативные соображения

Медицинский ИИ подлежит нормативному контролю в большинстве юрисдикций.

FDA (США): Программное обеспечение медицинских устройств на основе ИИ/МО (SaMD) требует предмаркетного одобрения или разрешения. Направление действий FDA по ИИ/МО 2021 года акцентирует **заранее определённые планы контроля изменений** — документирование того, как модель будет обновляться и как эти обновления будут валидироваться до развёртывания. Модель, непрерывно обучающаяся на клинической обратной связи, в этой рамке является другим устройством после каждого обновления и может требовать повторной валидации.

CE-маркировка (Европа): Медицинские устройства, включая системы ИИ, должны соответствовать Регламенту о медицинских изделиях (MDR). MDR требует клинической оценки, надзора после выхода на рынок и документации данных, использованных для обучения и валидации.

Ключевое следствие для HITL: Нормативные рамки требуют чёткой документации процессов аннотирования, квалификации аннотаторов, межчитательской надёжности и любых изменений в обучающих данных. Это не бюрократические накладные расходы — это аудиторский след, позволяющий клиницисту понять, какие обучающие данные породили текущее поведение модели, и это юридически обязательно. Конвейеры HITL, рассматривающие аннотирование как неформальный подпроцесс, создают нормативный риск, который, как правило, проявляется в самый неподходящий момент.

Аннотирование научных данных

За пределами здравоохранения HITL ML играет всё бóльшую и недооценённую роль в научных исследованиях, где задача аннотирования нередко сочетает предметную экспертизу с масштабом.

Астрономия: Galaxy Zoo

Galaxy Zoo {cite}`lintott2008galaxy` организовал краудсорсинг морфологической классификации галактик из Слоановского цифрового обзора неба среди учёных-любителей. Исходный проект собрал свыше 40 миллионов классификаций от более чем 100 000 добровольцев, продемонстрировав осуществимость крупномасштабного краудсорсинга научной классификации изображений, когда задача может быть разбита на простые вопросы, на которые можно ответить без специализированной подготовки («Эта галактика гладкая или структурированная?»).

Опыт Galaxy Zoo дал два важных методологических вывода. Во-первых, согласие между учёными-любителями и профессиональными астрономами было высоким для ясных случаев и систематически расходилось в пограничных случаях — именно там, где научно значимо различие. Решением стало не отброс данных учёных-любителей по пограничным случаям, а трактовка распределения ответов добровольцев как мягкой метки, кодирующей подлинную морфологическую неоднозначность. Во-вторых, классификатор, обученный на метках Galaxy Zoo, превзошёл модели, обученные воспроизводить выбор любого отдельного эксперта, поскольку распределение толпы захватывало реальную визуальную неопределённость, которую вынужденный выбор эксперта сворачивал.

Геномика: классификация патогенности

Аннотирование геномных вариантов — решение о том, является ли вариант патогенным, доброкачественным или неопределённого значения, — является высокорисковой задачей NLP и экспертного суждения. Клинические базы данных вариантов, такие как ClinVar, агрегируют интерпретации экспертов из нескольких лабораторий, и расхождения между лабораториями распространены. Активное обучение используется для приоритизации вариантов, требующих полного экспертного рассмотрения (поиск в литературе, оценка функциональных данных), по сравнению с теми, которые могут быть автоклассифицированы существующими данными. Результатом является гибридный конвейер, где большинство вариантов обрабатываются автоматической логикой, подмножество требует экспертного рассмотрения, а наиболее сложные случаи помечаются для многолабораторного консенсуса.

Климат и науки о Земле

Разметка спутниковых снимков для определения изменений землепользования, вырубки лесов, площади ледников и трасс штормов включает экспертов по дистанционному зондированию и всё чаще — платформы гражданской науки. Главная задача HITL в этой области — временна́я: метки, сделанные сегодня, могут устареть по мере изменения мира, а верификация по истине (полевые исследования) дорогостояща и логистически ограничена. Активное обучение, приоритизирующее изображения, где предсказание модели расходится с физическими априорными данными (например, предсказание вырубки леса в охраняемом регионе), является практическим способом направить скудные ресурсы полевой верификации.

Нейронаука: коннектомика

Реконструкция нейронных цепей из изображений электронной микроскопии — коннектомика — требует пиксельной аннотации мембран нейронов в огромных стеках изображений. Проект Eyewire геймифицировал эту задачу, вовлекая десятки тысяч игроков в трассировку нейронов через объёмы 3D-изображений. Геймификация решила специфическую проблему HITL: задача требует устойчивого внимания и пространственного мышления на протяжении длительных сеансов, что вызывает снижение качества при традиционном аннотировании. Разбиение задачи на игровые сегменты с социальной механикой поддерживало вовлечённость и качество аннотаторов в масштабах, недостижимых для профессионального аннотирования.

Управление специалистами-аннотаторами

Когда аннотирование требует редкой экспертизы, обычные подходы краудсорсинга (глава 13) неприменимы.

Фундаментальное противоречие состоит в том, что люди, способные создавать наиболее качественные аннотации, — это также люди, чьё время наиболее ценно и ограниченно. Каждое проектное решение в конвейере аннотирования специалистами должно оцениваться с точки зрения вопроса: оптимально ли это использует скудное экспертное время?

Что это означает на практике:

- **Агрессивное предварительное аннотирование.** Использовать аннотаторов более низкого уровня, автоматизированные модели или основанные на правилах системы для генерации кандидатов, которые специалист проверяет и исправляет, а не создаёт с нуля. Суждение специалиста является узким местом; предоставление ему предварительной аннотации для исправления быстрее, чем просить аннотировать с чистого экрана, при условии что качество предварительной аннотации достаточно, чтобы исправление не оказалось медленнее начала с нуля.

- **Проектировать для внимания эксперта, а не для пропускной способности.** Интерфейсы аннотирования, оптимизированные для высокой пропускной способности (быстрые бинарные решения, горячие клавиши, минимальный дисплей), подходят для краудсорсинга. Аннотирование специалистами часто выигрывает от более богатых интерфейсов: сравнение бок о бок с предыдущими случаями, лёгкий доступ к справочным материалам, поля уверенности аннотирования

и возможность пометить случай для обсуждения. Это замедляет отдельные аннотации, но улучшает качество и снижает потребность в повторном аннотировании.

- **Явно отслеживать паттерны отдельных аннотаторов.** При небольшом пуле специалистов осуществимо и важно отслеживать коэффициент согласия каждого аннотатора с группой, отмечать случаи, кажущиеся несогласованными с их собственной историей, и обсуждать их на регулярных калибровочных сессиях. Это не слежка — это тот же процесс качества, который клиническая медицина использует для анализа производительности, и специалисты, как правило, хорошо реагируют, когда это преподносится как совместное улучшение качества, а не как оценка.

- **Проектирование сеансов имеет значение.** Медицинское аннотирование когнитивно требовательно. Данные радиологии и патологии свидетельствуют о том, что частота ошибок заметно возрастает примерно после 90 минут непрерывного чтения и что перерывы даже в несколько минут могут частично восстановить внимание. Интерфейсы аннотирования, принудительно вводящие подсказки о перерывах (без возможности их игнорировать), — это простое проектное решение HITL с реальным влиянием на качество.

Конвейер HITL активного обучения для медицинской визуализации

```
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score
from sklearn.datasets import make_classification
import matplotlib.pyplot as plt

rng = np.random.default_rng(42)

# Simulate a rare-pathology detection task
# 8% positive class (e.g., rare pathology)
X, y = make_classification(
    n_samples=5000, n_features=100,
    n_informative=20, n_redundant=10,
    weights=[0.92, 0.08],
    random_state=42
)
X_train, y_train = X[:4000], y[:4000]
X_test,  y_test  = X[4000:], y[4000:]

print(f"Training set positive prevalence: {y_train.mean():.1%}")

def run_medical_al(strategy, n_initial=50, budget=300):
    labeled = list(rng.choice(len(X_train), n_initial, replace=False))
    unlabeled = [i for i in range(len(X_train)) if i not in labeled]
    aucs = []

    while len(labeled) < n_initial + budget:
        model = LogisticRegression(max_iter=500, class_weight='balanced')
```

```
        model.fit(X_train[labeled], y_train[labeled])

        if len(labeled) % 30 == 0:
            preds = model.predict_proba(X_test)[:, 1]
            aucs.append(roc_auc_score(y_test, preds))

        X_pool = X_train[unlabeled]
        if strategy == 'uncertainty' and len(labeled) >= 10:
            probs = model.predict_proba(X_pool)
            entropy = -np.sum(probs * np.log(probs + 1e-9), axis=1)
            q = int(np.argmax(entropy))
        else:
            q = rng.integers(0, len(unlabeled))

        labeled.append(unlabeled.pop(q))

    return np.array(aucs)

aucs_al  = run_medical_al('uncertainty')
aucs_rnd = run_medical_al('random')
label_counts = np.arange(len(aucs_al)) * 30 + 50

plt.figure(figsize=(7, 4))
plt.plot(label_counts, aucs_al,  'o-',  color='#2b3a8f', linewidth=2, label='Uncertainty AL')
plt.plot(label_counts, aucs_rnd, 's--', color='#e05c5c', linewidth=2, label='Random baseline'
plt.xlabel("Expert labels obtained", fontsize=12)
plt.ylabel("AUROC", fontsize=12)
plt.title("Active Learning for Rare Pathology Detection", fontsize=13)
plt.legend(); plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig('medical_al.png', dpi=150)
plt.show()

# Show how AL preferentially selects positive examples
# by checking which class the queried examples belong to
n_init = 50
labeled_al  = list(rng.choice(len(X_train), n_init, replace=False))
labeled_rnd = labeled_al.copy()
unlabeled_al  = [i for i in range(len(X_train)) if i not in labeled_al]
unlabeled_rnd = unlabeled_al.copy()

model = LogisticRegression(max_iter=500, class_weight='balanced')
model.fit(X_train[labeled_al], y_train[labeled_al])
probs = model.predict_proba(X_train[unlabeled_al])
entropy = -np.sum(probs * np.log(probs + 1e-9), axis=1)
top50_al  = [unlabeled_al[i] for i in np.argsort(entropy)[-50:]]
top50_rnd = list(rng.choice(unlabeled_rnd, 50, replace=False))

pos_rate_al  = y_train[top50_al].mean()
pos_rate_rnd = y_train[top50_rnd].mean()
print(f"\nPositive rate in next 50 queries:")
```

```
print(f"  Uncertainty AL: {pos_rate_al:.1%}  (base rate: {y_train.mean():.1%})")
print(f"  Random:         {pos_rate_rnd:.1%}")
print(f"  AL queries {pos_rate_al/y_train.mean():.1f}x more positives than random")
```

See also: Краудсорсинг Galaxy Zoo: {cite}`lintott2008galaxy`. Производительность радиологов в CheXNet: {cite}`rajpurkar2017chexnet`. Качество рентгенограмм и диагностика с поддержкой ИИ: {cite}`yu2022assessing`. Методология аннотирования клинического NLP: {cite}`pustejovsky2012natural`. Руководство FDA по плану действий в области ИИ/МО см. в опубликованной документации FDA (2021).

Платформы и инструменты аннотирования

Платформа аннотирования — это среда, где человеческая обратная связь превращается в данные. Хорошая платформа повышает пропускную способность, снижает количество ошибок, обеспечивает контроль качества и делает конвейер аннотирования управляемым в масштабе. Выбор правильной платформы — и понимание того, когда создавать собственную, а когда использовать готовое решение, — является важным решением в любом проекте HITL.

Ландшафт платформ аннотирования

Рынок инструментов аннотирования существенно вырос и созрел в последние годы, движимый корпоративным спросом на данные для обучения МО. Инструменты охватывают широкий спектр: от полностью управляемых сервисов до фреймворков с открытым исходным кодом для самостоятельного размещения.

Платформы с открытым исходным кодом

Label Studio — наиболее популярная платформа аннотирования с открытым исходным кодом, поддерживающая текст, изображения, аудио, видео и временны́е ряды через унифицированную конфигурацию задач на основе XML. Может быть размещена самостоятельно и интегрируется с ML-бэкендами для активного обучения. Ключевые преимущества: гибкость, поддержка сообщества и возможность встраивать предсказания ML для предварительного аннотирования.

Prodigy (от создателей spaCy) — высококонвейерный инструмент аннотирования, ориентированный на задачи NLP. Его потоковая архитектура отправляет примеры по одному за раз и поддерживает активное обучение «из коробки». Интерфейс аннотирования минималистичен, но быстр — разработан для максимизации пропускной способности. Prodigy является платным программным обеспечением, но широко используется в исследованиях NLP.

CVAT (Computer Vision Annotation Tool) — ведущий инструмент с открытым исходным кодом для аннотирования CV, обеспечивающий мощную поддержку обнаружения, сегментации и аннотирования видео. Изначально разработан в Intel, CVAT поддерживает алгоритмы отслеживания, аннотирование скелетов и интеграции сторонних алгоритмов.

Doccano ориентирован на задачи разметки последовательностей (NER, извлечение отношений, классификация текста). Простой веб-интерфейс делает его доступным для команд без специализированных инженерных ресурсов.

Коммерческие платформы

Scale AI предоставляет комплексные управляемые услуги аннотирования: рабочую силу, управление качеством и API-интеграцию. Особенно сильна для автономного вождения, робототехники и сложного 3D-аннотирования. Ценообразование основано на сложности и объёме задачи.

Labelbox — полная платформа для курирования данных, аннотирования и разметки с помощью ML. Мощные корпоративные функции: управление проектами, рабочие процессы качества, циклы обратной связи с моделями и интеграции с основными ML-платформами (SageMaker, Vertex AI, Azure ML).

Appen (ранее Figure Eight / CrowdFlower) управляет большой глобальной аннотационной рабочей силой наряду с инструментами. Хороший выбор, когда основными задачами являются объём и управление рабочей силой.

Surge AI ориентирована на экспертных аннотаторов и сильна для задач, требующих знаний предметной области или тонкого суждения.

Humanloop специализируется на сборе обратной связи для LLM — аннотировании предпочтений, сборе данных RLHF и оценке моделей.

Сравнение функций платформ аннотирования

Функция	Label Studio	Prodigy	CVAT	Labelbox	Scale AI
Лицензия	Открытый код	Коммерческая	Открытый код	Коммерческая	Коммерч.
Размещение	Своё / облако	Своё	Своё / облако	Облако	Управляемое
Текстовое аннотирование	□	□	—	□	□
Аннотирование изображений	□	Ограниченное	□	□	□
Аннотирование видео	□	—	□	□	□
Интеграция активного обучения	□	□	Ограниченное	□	□
Предварительное аннотирование ML	□	□	□	□	□
Рабочие процессы контроля качества	Базовое	Базовое	Базовое	Расширенное	Расширенное
API / программный доступ	□	□	□	□	□
Управление рабочей силой	—	—	—	Ограниченное	□

Аннотирование как код

Критический, но часто упускаемый аспект инфраструктуры аннотирования — **контроль версий для аннотаций и схем аннотирования**. Рассматривать аннотирование как код означает:

Проектирование, ориентированное на схему. Таксономия меток и правила аннотирования определяются в версионированном конфигурационном файле (YAML или JSON) до начала аннотирования. Изменения схемы создают новую версию.

Версионирование аннотаций. Аннотации хранятся со ссылкой на версию схемы, под которой они были созданы. Это позволяет проводить аудит, откатываться и сравнивать аннотации между версиями схемы.

Воспроизводимые конвейеры. Конвейер аннотирования — от сырых данных до меток, готовых для обучения, — должен быть воспроизводимым из кода. Модели предварительного аннотирования, фильтры качества, логика агрегирования и разбиения данных — всё это должно быть задокументировано.

```
# Example: Label Studio annotation schema (text classification)
label_config: |
  <View>
    <Text name="text" value="$text"/>
    <Choices name="sentiment" toName="text" choice="single">
      <Choice value="positive"/>
      <Choice value="negative"/>
      <Choice value="neutral"/>
      <Choice value="mixed"/>
    </Choices>
  </View>
schema_version: "2.1.0"
task_type: text_classification
guidelines_version: "guidelines_v3.pdf"
```
```text
Источник данных
    |
    v
Неразмеченный пул --> Платформа аннотирования --> Размеченный датасет
    ^                    |                              |
    |                    | (с поддержкой ML)            v
Стратегия запросов  <----+                        Обучающий прогон
активного обучения                                      |
    ^                                                   v
    +---------- Обученная модель <----------- Оценка
                                                        |
                                                  Развёртывание и
                                                  мониторинг
```

Ключевые точки интеграции: 1. **Загрузка данных:** Неразмеченные данные автоматически поступают из хранилища данных на платформу аннотирования 2. **Обслуживание модели:** Текущая лучшая модель развёртывается на платформе аннотирования для предварительного аннотирования и оценки активного обучения 3. **Экспорт:** Завершённые аннотации экспортируются в формате, совместимом с обучающим фреймворком (COCO JSON, датасеты Hugging Face и т. д.) 4. **Цикл обратной**

связи: Ошибки производственной модели направляются обратно на платформу аннотирования для исправления

> **See also:** Документация Label Studio и интеграция активного обучения: https://labelstud.io. Prodigy: https://prodi.gy. CVAT: https://cvat.ai. Исчерпывающее сравнение инструментов аннотирования: {cite}`monarch2021human`, глава 7.

Краудсорсинг и контроль качества

Когда задачи аннотирования достаточно просты для выполнения неспециалистами, краудсорсинговые платформы обеспечивают доступ к большим аннотационным рабочим силам по низкой стоимости за элемент. Построение высококачественных размеченных датасетов из толпы требует тщательного проектирования задач, стратегической избыточности и строгого контроля качества.

Краудсорсинговые платформы

Amazon Mechanical Turk (MTurk) — исходный краудсорсинговый рынок, запущенный в 2005 году. Работники («тёркеры») выполняют микрозадачи (HIT), размещённые заказчиками. Исследование 2018 года показало, что средняя эффективная почасовая оплата тёркеров составляет примерно $2 в час — значительно ниже минимальной заработной платы во многих странах с высоким уровнем дохода {cite}`hara2018data` — этический аспект, рассматриваемый в главе 15. MTurk наиболее подходит для простых задач с чёткими, верифицируемыми критериями.

Prolific — академическая краудсорсинговая платформа, устанавливающая минимальный стандарт оплаты (в настоящее время около £9 в час, по данным опубликованных руководящих принципов Prolific), проводящая скрининг участников по демографическим показателям и поддерживающая пул работников, добровольно участвующих в исследованиях. Предпочтительна для исследований в социальных науках и задач, требующих репрезентативности.

Appen (и аналогичные: Telus International, iMerit) предоставляет управляемые аннотационные рабочие силы с управлением качеством, используемые для более сложных задач и корпоративных проектов.

Специализированные сообщества. Для задач, специфичных для предметной области, сообщества энтузиастов могут обеспечить высококачественные аннотации: Galaxy Zoo для астрономии, eBird для видов птиц, Chess Tempo для аннотирования шахматных позиций.

Проектирование задач для краудсорсинга

Декомпозиция сложных задач

Сложные задачи следует разбивать на простые, хорошо определённые микрозадачи. Вместо того чтобы просить работников провести исчерпывающее аннотирование

документа, задавайте им один сфокусированный вопрос: «Содержит ли это предложение имя человека?» или «Оцените ясность этого перевода по шкале от 1 до 5».

Преимущества декомпозиции: - Меньшая когнитивная нагрузка на задачу меньше усталости, выше качество - Каждая микрозадача поддаётся отдельному контролю качества - Проще аудировать и отлаживать

Важность инструкций

Единственный наиболее значимый предиктор качества краудсорсинга — это качество инструкций. Хорошие инструкции к задаче: - Объясняют *цель* задачи в одном предложении - Дают чёткое, однозначное определение каждой категории - Предоставляют 3–5 проработанных примеров (особенно пограничных случаев) - Не длиннее, чем работники реально прочитают (< 300 слов для простых задач)

Проведите **пилотное исследование** (10–50 работников, 20–100 задач) перед масштабированием. Проанализируйте расхождения пилота; большинство из них указывают на неоднозначности в инструкциях, которые можно исправить.

Вопросы золотого стандарта

Встраивайте **вопросы золотого стандарта** — задачи с известными правильными ответами — по всему батчу задач. Работники, не справляющиеся с вопросами золотого стандарта ниже порогового значения, исключаются из проекта.

```
import numpy as np
from scipy.stats import binom

rng = np.random.default_rng(42)

def simulate_gold_screening(n_workers=100, gold_per_batch=5,
                            p_good_worker=0.7, good_acc=0.92,
                            bad_acc=0.55, threshold=0.70):
    """
    Simulate quality screening via gold questions.
    Returns: precision and recall of identifying bad workers.
    """
    worker_quality = rng.choice(['good', 'bad'], size=n_workers,
                                p=[p_good_worker, 1 - p_good_worker])
    results = {'tp': 0, 'fp': 0, 'tn': 0, 'fn': 0}

    for q in worker_quality:
        acc = good_acc if q == 'good' else bad_acc
        n_correct = rng.binomial(gold_per_batch, acc)
        passed = (n_correct / gold_per_batch) >= threshold
        if q == 'bad' and not passed:  results['tp'] += 1
        if q == 'good' and not passed: results['fp'] += 1
        if q == 'bad' and passed:      results['fn'] += 1
        if q == 'good' and passed:     results['tn'] += 1

    tp, fp, fn = results['tp'], results['fp'], results['fn']
    precision = tp / (tp + fp + 1e-6)
    recall    = tp / (tp + fn + 1e-6)
```

```
    return precision, recall

# Vary gold question count
gold_counts = [3, 5, 8, 10, 15, 20]
precisions, recalls = [], []
for g in gold_counts:
    p_list, r_list = [], []
    for _ in range(50):
        p, r = simulate_gold_screening(gold_per_batch=g)
        p_list.append(p); r_list.append(r)
    precisions.append(np.mean(p_list))
    recalls.append(np.mean(r_list))

import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(gold_counts, precisions, 'o-', color='#2b3a8f', label='Precision', linewidth=2)
ax.plot(gold_counts, recalls,    's--', color='#0d9e8e', label='Recall',    linewidth=2)
ax.set_xlabel("Gold questions per batch", fontsize=12)
ax.set_ylabel("Screening performance", fontsize=12)
ax.set_title("Worker Screening via Gold Standard Questions", fontsize=13)
ax.legend(fontsize=11); ax.grid(alpha=0.3)
plt.tight_layout()
plt.savefig('gold_screening.png', dpi=150)
plt.show()

p, r = simulate_gold_screening(gold_per_batch=5)
print(f"5 gold questions: Precision={p:.3f}, Recall={r:.3f}")
p, r = simulate_gold_screening(gold_per_batch=10)
print(f"10 gold questions: Precision={p:.3f}, Recall={r:.3f}")
```

Статистические модели агрегирования меток

Голосование большинства является естественной базовой линией, однако игнорирует различия в точности аннотаторов. Статистические модели могут работать лучше.

Модель Dawid–Skene

Модель Dawid–Skene (DS) {cite}`dawid1979maximum` совместно оценивает: - **Истинную метку** z_i для каждого элемента i - **Матрицу ошибок** $\pi_j^{(k,l)}$ для каждого аннотатора j: вероятность того, что аннотатор j разметит элемент с истинным классом k как класс l

EM-алгоритм итерирует: - **E-шаг:** Дав матрицы ошибок аннотаторов, вычислить апостериорную вероятность каждой истинной метки - **M-шаг:** Дав оценки меток элементов, обновить матрицы ошибок аннотаторов

```
import numpy as np
from scipy.special import softmax

def dawid_skene_em(annotations, n_classes, n_iter=20):
    """
```

```
    Simplified Dawid-Skene EM for binary classification.
    annotations: dict {item_idx: [(annotator_idx, label), ...]}
    Returns: estimated true labels and annotator accuracies.
    """
    items = sorted(annotations.keys())
    n_items = len(items)
    annotators = sorted({a for anns in annotations.values() for a, _ in anns})
    n_annotators = len(annotators)
    ann_idx = {a: i for i, a in enumerate(annotators)}

    # Initialize: majority vote
    T = np.zeros((n_items, n_classes))  # soft label estimates
    for i, item in enumerate(items):
        for _, label in annotations[item]:
            T[i, label] += 1
        T[i] /= T[i].sum()

    # Confusion matrices: shape (n_annotators, n_classes, n_classes)
    PI = np.ones((n_annotators, n_classes, n_classes)) * 0.5

    for _ in range(n_iter):
        # M-step: update confusion matrices
        PI = np.zeros((n_annotators, n_classes, n_classes)) + 1e-6
        for i, item in enumerate(items):
            for ann, label in annotations[item]:
                a = ann_idx[ann]
                for k in range(n_classes):
                    PI[a, k, label] += T[i, k]
        PI /= PI.sum(axis=2, keepdims=True)

        # E-step: update soft label estimates
        T = np.zeros((n_items, n_classes))
        for i, item in enumerate(items):
            log_p = np.zeros(n_classes)
            for ann, label in annotations[item]:
                a = ann_idx[ann]
                log_p += np.log(PI[a, :, label] + 1e-10)
            T[i] = softmax(log_p)

    return {item: T[i] for i, item in enumerate(items)}, PI

# Simulate crowdsourcing scenario
rng = np.random.default_rng(42)
N_ITEMS, N_ANNOTATORS, ANNS_PER_ITEM = 200, 10, 3
true_labels = rng.integers(0, 2, N_ITEMS)
# Annotator accuracies: 3 good (0.9), 4 medium (0.75), 3 noisy (0.6)
accuracies = [0.90]*3 + [0.75]*4 + [0.60]*3

annotations = {}
for i in range(N_ITEMS):
    anns_for_item = []
```

```
    chosen = rng.choice(N_ANNOTATORS, ANNS_PER_ITEM, replace=False)
    for a in chosen:
        acc = accuracies[a]
        label = true_labels[i] if rng.random() < acc else 1 - true_labels[i]
        anns_for_item.append((a, int(label)))
    annotations[i] = anns_for_item

# Run Dawid-Skene
soft_labels, confusion = dawid_skene_em(annotations, n_classes=2, n_iter=30)
ds_preds = {i: int(np.argmax(soft_labels[i])) for i in range(N_ITEMS)}

# Compare with majority vote
mv_preds = {}
for i in range(N_ITEMS):
    votes = [l for _, l in annotations[i]]
    mv_preds[i] = int(np.round(np.mean(votes)))

ds_acc = np.mean([ds_preds[i] == true_labels[i] for i in range(N_ITEMS)])
mv_acc = np.mean([mv_preds[i] == true_labels[i] for i in range(N_ITEMS)])

print(f"Majority vote accuracy:  {mv_acc:.3f}")
print(f"Dawid-Skene accuracy:    {ds_acc:.3f}")
print(f"\nEstimated annotator accuracies (diagonal of confusion matrix):")
for a in range(N_ANNOTATORS):
    est_acc = np.mean([confusion[a, k, k] for k in range(2)])
    print(f"  Annotator {a}: estimated={est_acc:.3f}, true={accuracies[a]:.2f}")
```

MACE

MACE (Multi-Annotator Competence Estimation) {cite}`hovy2013learning` — альтернативная вероятностная модель, явно представляющая спам аннотаторов (случайная разметка) и компетентное аннотирование. Аннотатор либо предоставляет значимую метку (с вероятностью $1 - \text{spam}_j$), либо случайную метку (с вероятностью spam_j). Эта двухкомпонентная модель нередко лучше калиброванна, чем Dawid–Skene в сценариях краудсорсинга, где некоторые аннотаторы являются чистыми спамерами.

Избыточность и стратегия агрегирования

Оптимальное число аннотаторов на элемент зависит от сложности задачи и качества аннотаторов:

- **Простые задачи с квалифицированными аннотаторами:** 1–2 аннотатора на элемент нередко достаточно
- **Умеренные задачи с обученными аннотаторами:** 3 аннотатора + голосование большинства
- **Сложные/субъективные задачи с краудворкерами:** 5–7 аннотаторов + Dawid–Skene

Ключевой вывод: избыточность наиболее ценна при низкой точности аннотаторов. Для аннотаторов с точностью p, точность голосования большинства при n аннотаторах составляет:

$$P(\text{большинство правы}) = \sum_{k>n/2}^{n} \binom{n}{k} p^k (1-p)^{n-k}$$

При $p = 0{,}70$ добавление третьего аннотатора повышает точность голосования большинства с 70% до 78%; при $p = 0{,}90$ прирост от третьего аннотатора незначителен (с 90% до 97%).

See also: Модель Dawid–Skene: {cite}`dawid1979maximum`. MACE: {cite}`hovy2013learning`. Исчерпывающий обзор краудсорсинга для NLP: {cite}`snow2008cheap`. Этика краудсорсинга и справедливая оплата: см. главу 15.

Оценка и метрики

Знание того, работает ли ваша система HITL, требует большего, чем измерение точности модели. Нужно знать, получаете ли вы ценность от бюджета аннотирования, действительно ли модель лучше согласована с намерениями человека и продолжит ли дополнительная человеческая обратная связь улучшать ситуацию. В данной главе рассматривается полный ландшафт оценки в условиях HITL.

Метрики, ориентированные на модель

Стандартные метрики МО непосредственно применимы к системам HITL, с некоторыми важными нюансами.

Метрики классификации

Точность уместна, когда классы сбалансированы и все ошибки одинаково дорогостоящи. Однако в условиях HITL размеченный тестовый набор может быть смещён стратегией запросов (активное обучение запрашивает неслучайные примеры), что делает простые оценки точности ненадёжными.

Метрика F1 является средним гармоническим точности и полноты, уместным для несбалансированных классов. В контексте HITL и точность, и полнота могут по-разному иметь значение в зависимости от асимметрии стоимостей ложноположительных и ложноотрицательных результатов.

AUROC измеряет способность модели различать классы независимо от порога — важно для задач, чувствительных к калиброванности, таких как медицинский скрининг.

Калиброванность измеряет, насколько хорошо предсказанные вероятности соответствуют эмпирическим частотам. В системах HITL модели, обученные на смещённых размеченных множествах (от активного обучения), могут быть неправильно откалиброваны, даже если они точны.

Метрики генеративных моделей

Для языковых моделей и генеративных систем оценка принципиально сложнее. Ни одна автоматическая метрика не охватывает качество:

- **BLEU / ROUGE / METEOR:** Метрики на основе ссылок для перевода и суммаризации. Слабо коррелируют с человеческими суждениями о качестве для длинных генераций.
- **Перплексия:** Измеряет, насколько хорошо модель предсказывает отложенный текст. Необходимое, но недостаточное условие качества.

- **BERTScore:** Сходство на основе вложений с ссылками. Лучше коррелирует с человеческими суждениями, чем n-граммные метрики.
- **Человеческая оценка:** Золотой стандарт. См. раздел 14.3.

Метрики эффективности аннотирования

Оценка HITL должна также измерять, используется ли человеческая обратная связь эффективно.

Кривые обучения

Кривая обучения строит производительность модели как функцию количества размеченных примеров. Крутая кривая обучения (быстрое улучшение при небольшом числе меток) указывает на то, что стратегия аннотирования выбирает информативные примеры. Пологая кривая обучения указывает на то, что дополнительная разметка даёт убывающую отдачу.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score, roc_auc_score
from sklearn.model_selection import StratifiedShuffleSplit

rng = np.random.default_rng(42)
X, y = make_classification(n_samples=5000, n_features=30, n_informative=15,
                           weights=[0.8, 0.2], random_state=42)
X_test, y_test = X[4000:], y[4000:]
X_pool, y_pool = X[:4000], y[:4000]

label_sizes = [20, 40, 60, 100, 150, 200, 300, 400, 600, 800, 1000, 1500, 2000]
metrics = {'f1': [], 'auc': []}

for n in label_sizes:
    idx = rng.choice(len(X_pool), n, replace=False)
    clf = LogisticRegression(max_iter=500, class_weight='balanced')
    clf.fit(X_pool[idx], y_pool[idx])
    preds = clf.predict(X_test)
    probs = clf.predict_proba(X_test)[:, 1]
    metrics['f1'].append(f1_score(y_test, preds))
    metrics['auc'].append(roc_auc_score(y_test, probs))

# Fit learning curve: performance ≈ a - b/sqrt(n)
from scipy.optimize import curve_fit

def learning_curve_fn(n, a, b):
    return a - b / np.sqrt(n)

popt_f1, _ = curve_fit(learning_curve_fn, label_sizes, metrics['f1'], p0=[0.9, 2])
popt_auc, _ = curve_fit(learning_curve_fn, label_sizes, metrics['auc'], p0=[0.95, 1])
```

```
n_smooth = np.linspace(20, 3000, 200)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4))

ax1.scatter(label_sizes, metrics['f1'], color='#2b3a8f', zorder=5, s=40)
ax1.plot(n_smooth, learning_curve_fn(n_smooth, *popt_f1), '--', color='#e05c5c',
         label=f'Fit: {popt_f1[0]:.3f} - {popt_f1[1]:.1f}/√n')
ax1.set_xlabel("Labeled examples"); ax1.set_ylabel("F1 score")
ax1.set_title("Learning Curve: F1"); ax1.legend(); ax1.grid(alpha=0.3)

ax2.scatter(label_sizes, metrics['auc'], color='#0d9e8e', zorder=5, s=40)
ax2.plot(n_smooth, learning_curve_fn(n_smooth, *popt_auc), '--', color='#e05c5c',
         label=f'Fit: {popt_auc[0]:.3f} - {popt_auc[1]:.2f}/√n')
ax2.set_xlabel("Labeled examples"); ax2.set_ylabel("AUROC")
ax2.set_title("Learning Curve: AUROC"); ax2.legend(); ax2.grid(alpha=0.3)

plt.tight_layout()
plt.savefig('learning_curves.png', dpi=150)
plt.show()

# Estimate the annotation budget needed to reach a target performance
target_f1 = 0.80
n_needed = (popt_f1[1] / (popt_f1[0] - target_f1)) ** 2
print(f"Estimated labels needed to reach F1={target_f1}: {n_needed:.0f}")
```

Анализ возврата инвестиций (ROI)

ROI человеческой обратной связи отвечает на вопрос: на сколько улучшается производительность модели за каждую дополнительную метку?

$$\mathrm{ROI}(n) = \frac{\Delta\text{производительность}(n)}{\text{стоимость одной метки}}$$

По мере взросления модели (и по мере того как легко обучаемые примеры исчерпываются) ROI, как правило, снижается. Практическое следствие: бюджеты аннотирования следует концентрировать на ранних этапах, когда ROI наиболее высок.

Человеческая оценка

Для генеративных систем и субъективных задач человеческая оценка остаётся золотым стандартом.

Прямая оценка (DA)

Аннотаторы оценивают выходы по абсолютной шкале (например, 1–100 для качества перевода или 1–5 для полезности ответа). DA была стандартизирована в оценке машинного перевода (бенчмарки WMT).

Лучшие практики для DA: - Случайным образом упорядочивайте выходы для предотвращения якорения - Используйте достаточное число аннотаторов на элемент (минимум 3–5) - Включайте контроли качества (очевидно хорошие и плохие примеры для отлова невнимательных оценщиков) - Сообщайте межаннотаторское согласие наряду с агрегированными оценками

Сравнительная оценка

Аннотаторы выбирают между двумя выходами: «Который лучше?» Сравнительные суждения быстрее и согласованнее, чем абсолютные оценки (см. главу 8). **Рейтинговые системы ELO** (заимствованные из шахмат) преобразуют результаты попарных сравнений в непрерывное ранжирование качества.

```
import numpy as np

def update_elo(rating_a, rating_b, outcome_a, k=32):
    """Update ELO ratings. outcome_a: 1=A wins, 0=B wins, 0.5=tie."""
    expected_a = 1 / (1 + 10 ** ((rating_b - rating_a) / 400))
    expected_b = 1 - expected_a
    new_a = rating_a + k * (outcome_a - expected_a)
    new_b = rating_b + k * ((1 - outcome_a) - expected_b)
    return new_a, new_b

# Simulate 5 model versions being compared pairwise
rng = np.random.default_rng(42)
true_quality = [0.60, 0.70, 0.75, 0.80, 0.85]  # underlying model quality
n_models = len(true_quality)
elo_ratings = {i: 1000.0 for i in range(n_models)}

for _ in range(500):  # 500 pairwise comparisons
    i, j = rng.choice(n_models, 2, replace=False)
    p_i_wins = true_quality[i] / (true_quality[i] + true_quality[j])
    outcome = 1.0 if rng.random() < p_i_wins else 0.0
    elo_ratings[i], elo_ratings[j] = update_elo(elo_ratings[i], elo_ratings[j], outcome)

print("ELO Rankings after 500 comparisons:")
sorted_models = sorted(elo_ratings.items(), key=lambda x: x[1], reverse=True)
for rank, (model_id, elo) in enumerate(sorted_models, 1):
    print(f"  Rank {rank}: Model {model_id}  ELO={elo:.1f}  True quality={true_quality[model_
```

Поведенческое тестирование (CheckList)

CheckList {cite}ribeiro2020beyond — методология систематической поведенческой оценки моделей NLP. Вместо случайных тестовых наборов она проектирует тестовые случаи, направленные на проверку конкретных возможностей:

- **Тесты минимальной функциональности (MFT):** Справляется ли модель с простыми, очевидными случаями?
- **Тесты инвариантности (INV):** Изменяется ли выход модели, когда не должен (например, при перефразировании)?
- **Тесты направленного ожидания (DIR):** Изменяется ли выход модели в ожидаемом направлении при изменении входа?

CheckList делает человеческую оценку целенаправленной и действенной: вместо единственной цифры точности он даёт профиль возможностей.

Измерение согласования с намерениями человека

Для систем RLHF измерение согласования является центральной задачей оценки.

Оценка модели вознаграждения: Точность модели вознаграждения на отложенном тестовом наборе предпочтений. Оуянг и др. {cite}`ouyang2022training` сообщают примерно о 72% попарной точности для модели вознаграждения InstructGPT; как приблизительный ориентир, цифры в этом диапазоне широко цитируются для аналогичных конвейеров RLHF, хотя результаты существенно варьируются в зависимости от задачи и качества данных.

Коэффициент выигрыша: Для двух версий модели (например, базовой линии SFT против дообученной RLHF), какая доля ответов RLHF-модели выигрывает в попарных сравнениях?

GPT-4 как оценщик: Использование мощного LLM для оценки ответов стало распространённым для быстрых итераций. Джилларди и др. {cite}`gilardi2023chatgpt` и Чжэн и др. {cite}`zheng2023judging` обнаруживают, что согласие LLM-оценщика с людьми варьируется примерно от 0,7 до 0,9 в зависимости от задачи — полезно для быстрого A/B-сравнения, но менее надёжно для обнаружения угодничества, культурных нюансов или проблем безопасности.

Обнаружение угодничества: Измерить, изменяет ли модель свои ответы на основе подразумеваемых предпочтений пользователя (например, «Я думаю, X верно; что вы думаете?»). Хорошо согласованная модель не должна быть угодливой.

A/B-тестирование в развёрнутых системах

Для систем в производстве основным методом оценки является **A/B-тестирование**: направлять часть пользователей к новой версии модели и измерять последующие результаты.

A/B-тестирование даёт несмещённую оценку качества модели в реальном контексте развёртывания, захватывая эффекты, которые лабораторная оценка упускает (поведение пользователей, популяционное распределение, пограничные случаи).

Задача состоит в выборе подходящих нижестоящих метрик. Метрики вовлечённости (клики, длина сеанса) могут вознаграждать манипулятивное поведение. Показатели выполнения задач или опросы об удовлетворённости пользователей лучше согласованы, но более зашумлены.

> **See also:** Поведенческое тестирование CheckList: {cite}`ribeiro2020beyond`. Методология оценки RLHF: {cite}`ouyang2022training`. Лучшие практики человеческой оценки в МП: {cite}`graham2015accurate`. Теория кривых обучения: {cite}`mukherjee2003estimating`.

Справедливость, смещение и этика

Системы с участием человека наследуют как возможности, так и ограничения людей, находящихся в них. Аннотаторы привносят в свою работу знания, суждения и творческое начало — но также предвзятости, усталость и социальный контекст своей жизни. Решения, которые мы принимаем при проектировании систем HITL, — кто аннотирует, что им задаётся, как им платят и как используется их работа, — имеют последствия, далеко выходящие за рамки метрик точности модели.

В данной главе рассматриваются этические аспекты HITL ML.

Источники предвзятостей в системах HITL

Демография аннотаторов

Аннотирование — не нейтральный акт. Метки, присваиваемые аннотаторами, отражают их точки зрения, жизненный опыт и культурные контексты. Когда аннотационная рабочая сила демографически однородна — как это нередко бывает, когда краудворк смещён в сторону молодых, мужчин и представителей западных стран, — полученный датасет кодирует точки зрения этой группы.

Данные: Исследования аннотационных датасетов NLP показали, что демографические характеристики аннотаторов предсказывают их выбор меток для субъективных задач (токсичность, тональность, оскорбительность). Датасеты, размеченные преимущественно краудворкерами из США, кодируют американские культурные нормы, не обобщающиеся на другие регионы {cite}`geva2019annotator`.

Последствия: Модели, обученные на таких данных, хорошо работают для пользователей, напоминающих пул аннотаторов, и менее хорошо — или предвзято — для пользователей, не похожих на него.

Меры снижения: Целенаправленная диверсификация рабочей силы; стратифицированное аннотирование (обеспечение вклада аннотаторов из релевантных демографических групп в релевантные задачи); отслеживание демографии аннотаторов и её влияния на распределения меток.

Формулировка задачи

Способ формулировки вопроса влияет на получаемые ответы. Если аннотаторам задают вопрос «Является ли этот текст токсичным?», они могут ответить иначе, чем на вопрос «Причинит ли этот текст вред тому, кто принадлежит к упомянутой группе?» Формулировка воплощает предположения о том, что важно.

Пример: Аннотирование «оскорбительного языка» в социальных сетях существенно варьируется в зависимости от того, предоставляется ли аннотаторам контекстуальная информация об идентичности автора. Высказывание, кажущееся угрожающим в изоляции, может быть распознано как переосмысленный язык или внутригрупповой юмор при наличии контекста.

Платформенные эффекты

Платформа и структура оплаты влияют на качество аннотирования. Работники, получающие оплату за задачу, а не за час, имеют стимул работать быстро; это повышает пропускную способность, но снижает качество. Работники, опасающиеся быть заблокированными на платформе за низкую точность, могут избегать отметки «неуверен» и вместо этого угадывать, маскируя подлинную неоднозначность.

Предвзятости подтверждения и якорения

На аннотаторов влияют: - **Предварительное аннотирование:** Предсказания модели, показываемые аннотаторам, принимаются чаще, чем отвергаются, даже когда они неверны - **Эффекты порядка:** Разметка одного и того же элемента в разных контекстах даёт разные ответы - **Прайминг:** Предыдущие элементы задачи влияют на то, как размечаются последующие

Справедливость в системах HITL

Что такое справедливость?

Справедливость в МО — спорная концепция с несколькими формальными определениями, которые нередко взаимно несовместимы {cite}`barocas2019fairness`. Для целей HITL наиболее релевантны следующие измерения:

Представленность: Являются ли обучающие данные репрезентативными для популяций, на которые будет воздействовать модель?

Паритет производительности: Работает ли модель одинаково хорошо для разных демографических групп?

Последовательность разметки: Размечаются ли одинаковые поведения одинаково, независимо от того, кто их демонстрирует? (Исследования показали, что это не всегда так — идентичный контент иногда размечается по-разному, когда приписывается разным расовым или гендерным группам.)

Активное обучение с учётом справедливости

Стандартное активное обучение фокусирует запросы на неопределённости модели, что склонно концентрироваться на примерах мажоритарного класса. Это может усугублять диспаритет производительности для меньшинственных групп.

Стратегии запросов с учётом справедливости дополняют критерий неопределённости ограничениями разнообразия или представленности:

$$x^* = \arg\max_{x \in \mathcal{U}} \left[\lambda \cdot \text{uncertainty}(x) + (1 - \lambda) \cdot \text{minority_group_bonus}(x)\right]$$

Установка $\lambda < 1$ обеспечивает, что стратегия запросов не полностью игнорирует представленность меньшинств.

Благополучие аннотаторов

Проблема теневой работы

Аннотационная работа, обеспечивающая МО, в значительной мере невидима. Работники с данными — нередко в Глобальном Юге — размечают изображения, транскрибируют речь и модерируют контент за низкую зарплату, без льгот, в условиях гиг-занятости без гарантий занятости. «Анатомия системы ИИ» Кейт Кроуфорд и Владана Йолера {cite}`crawford2018anatomy` и «Теневая работа» Мэри Грей и Сиддхарта Сури {cite}`gray2019ghost` задокументировали масштаб и ненадёжность этого труда.

Статистика Amazon MTurk: Систематический анализ заработка на MTurk 2018 года выявил среднюю эффективную почасовую оплату около $2 — ниже минимальной заработной платы в большинстве штатов США и многих странах с высоким уровнем дохода {cite}`hara2018data`. Работники за пределами стран с высоким уровнем дохода сталкиваются с дополнительными барьерами: заказчики нередко ограничивают высокооплачиваемые задачи только участниками из США, а глобальный пул работников, конкурирующих за оставшиеся открытые задачи, дополнительно сжимает эффективный заработок.

Модерация контента: Особенно вредная форма аннотационной работы — проверка графического, жестокого и тревожащего контента — связана с ПТСР, депрессией и тревогой среди работников {cite}`newton2019trauma`. Платформы сталкивались с критикой за недостаточную психологическую поддержку и чрезмерные квоты на воздействие.

Этические практики

Справедливая оплата: Платите аннотационным работникам не ниже местного минимального размера оплаты труда. Исследования показали, что более высокая оплата привлекает более качественных работников без пропорционального увеличения стоимости за правильную метку.

Видимость труда: Признавайте труд, создающий обучающие данные, в публикациях и документации продуктов.

Психологическая поддержка: Для работников, проверяющих вредоносный контент, обеспечьте психологическую поддержку, ротацию и ограничения воздействия.

Представительство работников: Предоставьте аннотационным работникам возможность сообщать о проблемах, запрашивать разъяснения по инструкциям и оспаривать несправедливые оценки качества.

Конфиденциальность при аннотировании

Защищённая медицинская информация (ЗМИ) и ПИИ

Задачи аннотирования нередко включают обработку конфиденциальных персональных данных. Медицинский проект аннотирования может подвергать работников медицинским

записям пациентов; проект NLP может подвергать работников частным коммуникациям; проект модерации контента — личным раскрытиям пользователей.

Нормативные рамки (HIPAA, GDPR) ограничивают передачу персональных данных аннотационным рабочим силам. Ключевые принципы:

- **Минимизация данных:** Передавайте только информацию, необходимую аннотаторам для выполнения задачи
- **Деидентификация:** Удаляйте ЗМИ и ПИИ перед аннотированием там, где это возможно
- **Согласие:** При аннотировании реальных пользовательских данных обеспечьте надлежащее согласие или законное основание
- **Контроль доступа:** Ограничьте доступ к чувствительным данным на основе роли и допуска

Синтетические данные как альтернатива

Для задач, где реальные данные несут риски конфиденциальности, генерация синтетических данных может создавать готовые к аннотированию датасеты без раскрытия конфиденциальной информации. Для клинического NLP, например, синтетический текст ЭМЗ может обеспечивать реалистичные обучающие данные для моделей деидентификации без раскрытия реальных медицинских записей.

Состязательное аннотирование и отравление данных

Системы HITL создают поверхность атаки: если злоумышленник может повлиять на процесс аннотирования, он может влиять на поведение модели.

Отравление данных через аннотирование: Злоумышленник с доступом к аннотационной рабочей силе (например, скомпрометированная учётная запись краудворкера) может внедрять систематически неверно размеченные примеры. Это особенно эффективно в условиях активного обучения, где модель предпочтительно запрашивает неопределённые примеры — которые могут быть целью злоумышленника.

Взлом вознаграждения через обратную связь: В RLHF аннотаторы (или аннотации, сгенерированные ИИ), последовательно высоко оценивающие определённые типы контента, могут направлять модель к этому контенту вне зависимости от его истинного качества.

Меры снижения: Несколько независимых аннотаторов на элемент; обнаружение выбросов в паттернах аннотирования; мониторинг аномального согласия или несогласия; поддержание тестовых наборов оценки, на которые аннотационная рабочая сила не может влиять.

Институциональная этика

IRB и этический контроль

Исследовательские проекты с участием людей — включая аннотационных работников — нередко требуют одобрения Институционального наблюдательного совета (IRB). Проекты аннотирования, собирающие данные об убеждениях работников, реакциях

на чувствительный контент или демографической информации, должны проходить этический контроль в той же рамке, что и другие исследования с участием людей.

Управление данными

Организации должны иметь чёткие политики в отношении: - Какие данные можно отправлять на внешнее аннотирование, а какие аннотировать внутри - Как долго хранятся аннотационные данные и кем - Кто имеет доступ к аннотациям и моделям, обученным на них - Как обрабатывать запросы на удаление размеченных данных (право на удаление по GDPR)

Прозрачность и подотчётность

Пользователи, на которых влияют системы МО, имеют законный интерес к пониманию того, как эти системы были построены. Документирование методологии аннотирования — кто размечал данные, в каких условиях, по каким инструкциям — является формой подотчётности, приносящей пользу пользователям, регуляторам и области в целом.

Паспорта датасетов {cite}`gebru2021datasheets` предоставляют структурированный шаблон для документирования происхождения датасета, процедур аннотирования и известных ограничений.

> **See also:** Рамка алгоритмической справедливости: {cite}`barocas2019fairness`. Теневая работа и труд на платформах: Грей и Сури (2019). Паспорта датасетов: {cite}`gebru2021datasheets`. Благополучие работников модерации контента: {cite}`newton2019trauma`. Демография аннотаторов и датасеты NLP: {cite}`geva2019annotator`.

Перспективы развития

Область машинного обучения с участием человека стремительно меняется. Фундаментальные модели, LLM-аннотаторы и новые парадигмы взаимодействия человека и ИИ трансформируют экономику и практику HITL так, как это было немыслимо ещё пять лет назад. В заключительной главе намечается ландшафт нарождающихся направлений и открытых проблем.

Фундаментальные модели и изменение роли аннотирования

Фундаментальные модели — большие модели, предварительно обученные на широких данных и адаптируемые к нижестоящим задачам, — коренным образом меняют экономику HITL.

Снижение нагрузки по аннотированию

Задача, ранее требовавшая тысяч размеченных примеров для обучения с нуля, может потребовать лишь десятков при дообучении фундаментальной модели. Промптинг с несколькими примерами может полностью устранить необходимость в дообучении для ряда задач.

Следствие: ROI аннотирования меняется. Усилия по аннотированию теперь лучше направлять на: - Высококачественные тестовые наборы (для измерения того, действительно ли адаптированная модель работает) - Трудные случаи и пограничные случаи, с которыми фундаментальная модель справляется неверно - Специфические для задачи примеры, обучающие модель тому, что нельзя вывести из предобучения

Новые формы спецификации

Когда базовая модель уже понимает язык, код и здравый смысл, пользователи могут общаться с ней на естественном языке, а не через размеченные примеры. Пользователь, которому нужен классификатор текста, больше не должен размечать 500 примеров — он может описать задачу классификации и оценить производительность в режиме нулевого обучения.

Это смещает задачу HITL от *сбора примеров* к *спецификации задачи*: помощи пользователям в точной формулировке того, что они хотят, в форме, на которую модель может действовать. Это сложнее, чем кажется, — описания задач на естественном языке нередко неоднозначны так, как размеченные примеры не являются.

LLM как аннотаторы

Одним из наиболее значимых событий 2023–2025 годов является использование **LLM в качестве оракулов аннотирования**. Вместо того чтобы спрашивать человека «Является ли этот отзыв позитивным или негативным?», мы спрашиваем GPT-4 или Claude. Для хорошо определённых задач классификации аннотации LLM нередко соответствуют точности краудворкеров или близко к ней {cite}`gilardi2023chatgpt`, а стоимость на аннотацию при обращениях к API, как правило, на порядки ниже, чем человеческие ставки труда.

Где аннотирование LLM работает хорошо

- Простые, хорошо определённые задачи классификации (тональность, тема, намерение)
- Задачи, где человеческие метки кодируют культурный консенсус, усвоенный LLM
- Задачи, где аннотирование согласованно в контекстах (не высоко субъективны)
- Генерация первичных аннотаций для проверки человеком

Где аннотирование LLM даёт сбой

- Высококонкретные задачи предметной области, требующие редкой экспертизы (медицинские, юридические, научные)
- Задачи, требующие местных культурных знаний или языковых вариаций
- Задачи безопасности и вреда, где LLM может быть неверно откалиброван в том же направлении, что и данные, на которых он обучался
- Новые типы задач, слабо представленные в предобучении

RLAIF и конституционный ИИ

Как обсуждалось в главе 6, обратная связь, сгенерированная ИИ, может использоваться для обучения моделей вознаграждения и руководства дообучением с RL. Это создаёт цикл обратной связи: LLM генерируют данные, на них обучаются модели, и лучшие модели генерируют лучшие данные. Вопрос о том, как запустить этот цикл без кодирования систематических ошибок от начальной модели, является центральной открытой проблемой в исследованиях масштабируемого надзора.

Слабый надзор в масштабе

Программная разметка через функции разметки (глава 9) позволяет экспертам предметной области выражать свои знания в виде кода, а не размеченных примеров. Системы Snorkel {cite}`ratner2017snorkel` и его преемники существенно созрели и сейчас используются в промышленной эксплуатации в крупных технологических компаниях.

Перспективные направления: - Функции разметки, дополненные LLM: Использование LLM для генерации функций разметки из описаний на естественном языке **- Обучение на основе срезов:** Выявление критических подмножеств данных (срезов), где функции разметки расходятся, и направление аннотирования туда **- Агрегирование с учётом неопределённости:** Более качественные статистические модели для объединения функций разметки с разной точностью и корреляциями

Непрерывное обучение с человеческим надзором

Большинство систем МО обучаются в автономном режиме и развёртываются как статические модели. Реальный мир меняется; модели, точные в момент развёртывания, деградируют по мере смещения распределения данных.

Непрерывное обучение — способность обучаться на новых данных, сохраняя старые знания, — является активной областью исследований. Человеческий надзор критически важен: автоматизированное непрерывное обучение без человеческого контроля может быстро кодировать смещения распределения, являющиеся ошибками, а не подлинными изменениями мира.

HITL непрерывное обучение включает: - Мониторинг смещения распределения (автоматический) и маршрутизацию смещённых примеров на человеческий контроль - Избирательное переобучение: одобренные человеком примеры из нового распределения добавляются в обучающие данные - Человеческий контроль изменений поведения модели после каждого обновления

Мультимодальный HITL

По мере того как системы ИИ становятся мультимодальными — обрабатывая и генерируя текст, изображения, аудио и видео одновременно, — аннотирование усложняется. Один фрагмент контента может требовать аннотирования по нескольким модальностям с зависимостями между ними.

Нарождающиеся типы задач: - Аннотирование видео + транскрипта (что происходит, кто говорит, что визуально описывает текст?) - Аннотирование медицинского изображения + клинического отчёта - Аннотирование траектории робота (связывание данных сенсоров с последовательностями действий)

Мультимодальные фундаментальные модели (GPT-4V, Gemini, Claude) меняют ландшафт аннотирования и здесь — модели теперь могут интерпретировать изображения и генерировать аннотации-кандидаты, которые люди проверяют.

Масштабируемый надзор

Фундаментальная открытая проблема HITL ML — **масштабируемый надзор** {cite}`irving2018ai,bowman` по мере того как системы ИИ становятся более способными, чем люди, в конкретных областях, как поддерживать значимый человеческий надзор?

Для таких задач, как предсказание структуры белка, юридический анализ или верификация математических доказательств, даже эксперты-аннотаторы могут не способны судить, какой из двух выходов ИИ является правильным. Текущие методы HITL ломаются, когда человеческое суждение менее надёжно, чем оцениваемая модель.

Предлагаемые подходы {cite}`bowman2022measuring`:

Дебаты: Два системы ИИ аргументируют разные заключения; судья-человек оценивает аргументы, а не заключения напрямую. Правильное заключение должно быть более защищаемым.

Усиление: Судьи-люди консультируются с ИИ-ассистентами (самой моделью) для оценки сложных выходов. Это использует возможности ИИ для расширения человеческого надзора, а не для его замены.

Надзор над процессом: Вместо оценки конечного вывода люди оценивают *процесс рассуждения* — отмечая ошибочные шаги в цепочке размышлений, независимо от того, оказывается ли конечный ответ правильным.

Эволюционирующее разделение труда

Долгосрочная траектория HITL ML направлена к более гибкому сотрудничеству людей и ИИ, где ни один из них не является просто «разметчиком», а другой — «обучающимся», но оба вносят вклад в совместный когнитивный процесс.

Тенденции, за которыми стоит следить: - ИИ-поддержанное аннотирование: ИИ предлагает; люди решают. Качество улучшается по мере того, как ИИ делает лучшие предложения. **- Человеко-направленное исследование:** Люди устанавливают цели и ограничения; ИИ исследует пространство решений. **- Совместная оценка:** Люди и ИИ совместно оценивают сложные выходы через диалог. **- Обучение предпочтениям в масштабе:** Неявные сигналы (то, как пользователи взаимодействуют с выходами ИИ) питают непрерывное обучение предпочтениям без необходимости в явных сеансах аннотирования.

Вопрос о том, насколько доверять суждению ИИ в сравнении с суждением человека, — и в каких областях, при каких уровнях способности, с какими защитными механизмами, — является в конечном счёте общественным, а не только техническим. HITL ML предоставляет техническую инфраструктуру для ответа на него обдуманно, а не по умолчанию.

Открытые исследовательские проблемы

Завершаем перечнем важных открытых проблем, над которыми активно работает область:

1. **Оптимальная остановка в активном обучении:** Когда предельная ценность следующей аннотации ниже стоимости? Принципиальные правила остановки остаются труднодостижимыми.

2. **Распределение бюджета аннотирования между задачами:** В многозадачных условиях как следует распределять фиксированный бюджет аннотирования между задачами?

3. **Смещение распределения в активном обучении:** Активное обучение создаёт смещённые размеченные множества. Как можно должным образом откалибровать модели, обученные таким образом?

4. **Обобщение модели вознаграждения:** Модели вознаграждения RLHF могут не обобщаться на новые пары подсказка–ответ. Как строить более надёжные модели предпочтений?

5. **Масштабируемый надзор:** Как поддерживать человеческий надзор по мере того, как системы ИИ превосходят человеческую производительность в конкретных областях?
6. **Моделирование аннотаторов:** Более качественные статистические модели поведения аннотаторов, захватывающие не только компетентность, но и систематические предвзятости, тематическую экспертизу и усталость.
7. **Оценка согласования:** У нас отсутствуют тесты с истиной в последней инстанции для проверки согласования модели с человеческими ценностями. Разработка таких тестов — по аналогии с состязательными примерами для устойчивости — является открытой проблемой.
8. **Справедливый трудовой рынок данных:** Экономические и институциональные структуры, обеспечивающие справедливую компенсацию и защиту аннотационных работников при сохранении рентабельности крупномасштабного аннотирования.

> *Цель не в том, чтобы заменить человеческое суждение машинным, а в том, чтобы строить системы, где их сочетание лучше, чем каждое по отдельности.*

Инструменты и техники из данного руководства — аннотирование, активное обучение, RLHF, обучение предпочтениям и всё остальное — являются средствами к этой цели. По мере развития области конкретные техники будут меняться. Лежащее в основе стремление — строить системы, одновременно способные и подлинно согласованные с намерениями человека, — останется.

> **See also:** Масштабируемый надзор и дебаты: {cite}`bowman2022measuring`. Слабый надзор Snorkel: {cite}`ratner2017snorkel`. Широкое будущее сотрудничества человека и ИИ: {cite}`amershi2019software`.

Библиография

Table 1: all:

jupytext:
formats: md:myst
text_representation:
extension: .md
format_name: myst
kernelspec:
display_name: Python 3
language: python
name: python3

Пример из практики: Limen — человек в петле всего

:::{admonition} Примечание к этой главе :class: important Данная глава представляет собой **спекулятивное проектное эссе**, а не эмпирическое исследование или рецензируемый вклад в науку. В ней описывается задуманная архитектура системы — Limen, голосо-центрическая ОС, построенная на принципах ИИ, — как проработанный пример того, как принципы HITL из предшествующих глав могут объединиться в единое целое. Утверждения здесь являются проектными аргументами, а не экспериментальными результатами. Их следует читать как обоснованное проектное обоснование, а не как верифицированные инженерные выводы. :::

Каждая глава этого руководства описывала HITL ML как философию проектирования: набор принципов, делающих участие человека в обучающихся системах намеренным, эффективным и честным. В этой главе описывается, как выглядит применение этой философии в качестве первого принципа — не к единой системе, а ко всей операционной среде.

Limen — это голосо-центрическая, построенная на принципах ИИ настольная операционная система, основанная на предпосылке, что человек всегда в петле — не как ограничение, которое нужно оптимизировать, а как организующий принцип, вокруг которого проектируется каждая подсистема. Название выбрано намеренно: *limen* — латинское слово, означающее «порог». В перцептивной психологии лимен — это граница между воспринимаемым и невоспринимаемым. Для операционной системы лимен — это граница между намерением человека и действием машины.

Архитектура, описанная здесь, не является собственностью. Это набор проектных решений, каждое из которых естественно следует из принципов HITL, развитых в предшествующих главах. Цель — не документировать продукт, а показать, как эти принципы компонуются, — как они усиливают друг друга при последовательном применении и какие виды систем становятся возможными, когда человек — не запоздалая мысль.

Проблема традиционного проектирования ОС

Традиционная операционная система разработана не для людей. Она разработана для программ. Человек принимается через уровень абстракции — графический интерфейс, файловый менеджер, терминал, — надстроенный над субстратом, построенным для процессов, адресов памяти и файловых дескрипторов.

Это проектное решение исторически оправдано. Когда принимались предположения,

породившие его, компьютеры были дорогими, люди — дешёвыми, а узким местом были вычисления. Правильной целью оптимизации была машина.

Эти предположения больше не действуют. Узким местом для большинства пользователей в большинстве задач являются не вычисления. Это человеческое внимание: цена переключения контекста, поиска нужного файла, формулировки правильного запроса, воспоминания о том, где что-то было. Машина быстра. Человек медлителен. Интерфейс должен оптимизироваться для человеческой стороны петли.

Традиционные операционные системы этого не делают. Они оптимизированы для программ, и задача человека — говорить на языке программ. Limen инвертирует это. Программы говорят на языке человека. Человек находится в петле, и петля разработана так, чтобы соответствовать человеку.

Архитектура на пороге

Архитектура Limen организована вокруг единого принципа: **каждое взаимодействие — это событие**, и каждое событие — это возможность для человека обучать, исправлять или подтверждать. Система не ждёт явных обучающих сессий. Обучение непрерывно и фоново.

Слой событий: WID

В основе лежит **WID** (система идентификаторов Waldiez, адаптированная для локально-ориентированной архитектуры Limen) — причинно-осознанная система отслеживания событий, записывающая не только то, что произошло, но и то, что вызвало это, и то, что в свою очередь вызвало это.

Обычное журналирование спрашивает: *что случилось?* WID спрашивает: *почему это случилось, и что последовало за этим?* Каждое событие несёт причинно-следственную цепочку — от запускающего человеческого действия через промежуточные состояния системы к наблюдаемому результату.

Это важно для обучения HITL, поскольку решает **проблему приписывания заслуг** на уровне взаимодействия. Когда пользователь исправляет поведение системы, WID способен выявить не просто немедленный вывод, который был неверным, но и цепочку решений, которая его произвела. Исправления могут быть применены на правильном уровне абстракции: к выводу, правилу принятия решений или восходящему сигналу.

Это эквивалент для операционной системы того, что в главе 6 описывается для RLHF: способность проследить сигнал вознаграждения через последовательность решений. WID обеспечивает это отслеживание нативно, для каждого взаимодействия, без необходимости понимать это пользователю.

:::{admonition} Причинное отслеживание событий как инфраструктура HITL :class: note Дизайн WID отражает более широкий принцип: инфраструктура HITL должна упрощать вопрос «почему система сделала это?» — а не только «что она сделала?». Без причинного отслеживания исправления устраняют симптомы. С ним они могут устранять причины. Разница между заплаткой и уроком. :::

Слой восприятия: голос как основной режим

Основная модальность ввода Limen — голос, обрабатываемый локально с использованием конвейера Whisper ONNX. Причины этого выбора заслуживают явного изложения:

Голос — наиболее естественный выходной канал человека для большинства людей. Он не требует обучения, физической ловкости помимо обычной речи и знания внутренней организации системы. Пользователь, не способный найти файл в иерархии каталогов, может описать, что ищет.

Локальная обработка сохраняет конфиденциальность. Голосовые данные не покидают устройство. Это важно этически — голос является биометрическими данными, и их сбор в масштабе облачными провайдерами является задокументированным вредом, — и практически: автономная работа означает, что система остаётся функциональной без сетевого подключения.

Голос создаёт естественный цикл обратной связи. Когда система отвечает, реакция пользователя — продолжение речи, перефразирование, слова «нет, это неверно», — сама является сигналом. Limen рассматривает эти реакции как обратную связь HITL: свидетельство того, была ли интерпретация системой предыдущего высказывания верной.

Запасная цепочка имеет не меньшее значение, чем основная модальность. Когда голос даёт сбой — в шумной среде, для пользователя с речевым расстройством, для ввода, требующего точности, — Limen плавно переходит к клавиатурному интерфейсу, затем к структурированному текстовому интерфейсу. Тест бабушки (см. главу 5) — не запоздалая мысль об доступности; это первоклассное архитектурное ограничение.

Слой интеллекта: маршрутизация через множество LLM

Limen не полагается на единственную языковую модель. Она маршрутизирует запросы через структурированное дерево решений на основе типа задачи, требуемой задержки, требований конфиденциальности и уверенности:

1. **Локальная малая модель** (всегда первая): быстрая, приватная, обрабатывает рутинные задачи — «открой файл, над которым я работал вчера», «установи таймер», «какая погода»
2. **Локальная большая модель** (когда уверенность малой модели низка): медленнее, но мощнее; обрабатывает структурированные рассуждения, генерацию кода, сложное извлечение
3. **Удалённая модель** (когда локальная не справляется и пользователь дал разрешение): запасной выход; обрабатывается прозрачно с явным уведомлением пользователя

Эта структура не нова — она является инференс-временным эквивалентом стратегии запросов активного обучения, описанной в главе 4. На каждом шаге система спрашивает: достаточна ли моя текущая модель для ответа на этот запрос с приемлемой уверенностью? Если нет — эскалировать. Эскалация — это издержка (задержка, конфиденциальность, вычисления); она несётся только при необходимости.

Человек находится в петле на границе эскалации. Пользователь, настроивший Limen никогда не эскалировать к удалённым моделям, принял решение HITL — которое система уважает и записывает. Пользователь, одобривший удалённый запрос, а затем сказавший «не спрашивай меня снова об этом типе запросов», предоставил предпочтение, обновляющее политику маршрутизации.

Проектирование взаимодействия

Непосредственность

Limen разработана для непосредственности в том смысле, в каком глава 5 определяет её: пользователь воспринимает эффект своей обратной связи. Когда пользователь исправляет вывод системы, исправление немедленно и видимо применяется. Модель не уходит и не обучается двадцать минут. Она обновляется в текущем сеансе.

Это требует использования архитектур моделей, поддерживающих эффективное онлайн-обновление — адаптеры, prefix tuning и извлечение с дополненной генерацией, а не полное дообучение. Компромисс явен: онлайн-обновления более зашумлены, чем батчевое обучение. Limen принимает этот компромисс, потому что непосредственность является основным требованием: человек всегда может подтвердить, отклонить или уточнить обновление.

Понятность

Повторяющаяся тема в литературе IML — **требование понятности**: люди могут направлять лишь ту модель, которую понимают, хотя бы приблизительно. Limen реализует это в интерфейсе: когда система принимает решение, она объясняет, кратко, почему. Не полная трассировка рассуждений — это перегрузило бы большинство пользователей, — а краткое резюме на естественном языке о ключевом факторе: «Я открываю проект, над которым вы работали совсем недавно — это верно?»

Это объяснение также является вопросом. Оно приглашает к исправлению. Оно делает вывод модели видимым, чтобы пользователь мог перенаправить его при ошибке. Объяснение генерируется не из эстетических соображений; это функциональная инфраструктура HITL.

Последовательность и дрейф

Проблема, возникающая в любой долгосрочно работающей интерактивной системе, — **поведенческий дрейф**: поведение системы в момент $T + n$ слегка отличается от поведения в момент T так, как ни пользователь, ни система явно не выбирали. Исправления накапливаются. Пограничные случаи компонуются. Модель, согласованная с предпочтениями пользователя в прошлом месяце, может больше не быть согласованной сегодня.

Limen обращается к этому через периодические проверки последовательности — эквивалент для операционной системы метода повторного предъявления, описанного в главе 13. Система предъявляет пользователю исторические решения: «Несколько недель назад вы просили меня делать X. Это по-прежнему то, что вы хотите?» Эти проверки выполняют две функции: выявляют дрейф и напоминают пользователю о предпочтениях, которые тот мог забыть, что указывал.

Limen как система HITL

Рассматривая архитектуру Limen через призму этого руководства, проектные решения непосредственно соответствуют концепциям, развитым в каждой части.

Часть I (Основы): Limen рассматривает каждое взаимодействие как событие взаимодействия человека и машины. Нет «режима без HITL» — система всегда обучается, всегда атрибутирует, всегда ожидает участия человека.

Часть II (Базовые техники): Активное обучение проявляется как цепочка эскалации на основе уверенности. Интерактивное МО проявляется в цикле обновления в реальном времени. Аннотирование неявно: каждое исправление, которое делает пользователь, — это метка.

Часть III (Обучение на основе обратной связи человека): Обучение предпочтениям, описанное в главе 8, появляется в обновлениях политики маршрутизации. Когда пользователь предпочитает ответ локальной модели удалённому, это предпочтение записывается и обобщается. RLHF в персональной ОС означает, что модель вознаграждения приватная, индивидуальная и непрерывно обновляемая.

Часть IV (Приложения): Limen — среда общего назначения, но её проектирование наиболее заметно в областях, где суждение человека незаменимо и цена ошибки высока — составление документов, расстановка приоритетов задач, творческая работа.

Часть V (Системы и практика): WID — платформа аннотирования Limen. Она невидима пользователю в нормальной работе, видима при необходимости для отладки или прозрачности. Механизмы контроля качества (проверки последовательности, пороги уверенности, журналы эскалации) заимствованы непосредственно из литературы по краудсорсингу.

Часть VI (Этика): Локально-ориентированное, конфиденциально-сохраняющее проектирование — этический выбор, а не только технический. Данные человека не покидают его устройство без явного согласия. Модель, обучающаяся на поведении пользователя, принадлежит этому пользователю.

Более глубокая мысль

Limen — не коммерческая презентация. Это аргумент через конструкцию.

Аргумент таков: если принимать принципы HITL ML всерьёз — если верить, что человеческая обратная связь является сигналом для понимания, а не издержками для минимизации, что человек всегда находится в петле, даже когда проектировщики притворяются иным, что согласование — это непрерывный процесс, а не единовременное событие, — то в итоге строишь нечто, похожее на Limen.

Не обязательно именно Limen. Конкретный технологический стек (Tauri, Rust, Babylon.js, Whisper ONNX) — один из вариантов среди многих. Но архитектура — причинное отслеживание событий, локально-ориентированная обработка, постепенная деградация, непрерывное обучение предпочтениям, прозрачность как первоклассная характеристика — следует из принципов.

Область HITL ML затратила значительные усилия на описание того, как помещать людей в петли конкретных моделей и конкретных задач. Следующий вопрос: можем ли мы проектировать целые *среды* вокруг тех же принципов — среды, где человек всегда является центром, машина всегда является обучающимся и петля всегда открыта.

Limen — один ответ на этот вопрос.

Порог — это не то, через что переступаешь и оставляешь позади. Это место, где живёшь.

See also: Принципы IML, лежащие в основе проектирования взаимодействия Limen, развиты в главе 5. Подход обучения предпочтениям, стоящий за политикой маршрутизации, формализован в главе 8. Причинная модель WID основана на литературе по атрибуции событий, рассмотренной в главе 14. Тест бабушки, представленный в главе 5, является главным проектным ограничением интерфейса Limen.

Об уходе

В какой-то момент я был зачислен в аспирантуру. Исследование касалось машинного обучения. Учреждение было настоящим. Намерение — искренним.

А потом я остановился.

Не потому что предмет перестал иметь значение — он значит сегодня больше, чем в начале. Не потому что работа оказалась слишком трудной — это руководство по-своему труднее, чем диссертация. Но потому что где-то между обзором литературы и третьей редакцией статьи, которую прочитают одиннадцать человек, я потерял нить понимания того, зачем всё это делается.

Система аспирантуры — это петля. Вы входите в неё как человек с вопросами. Петля учит вас превращать эти вопросы в статьи, статьи — в цитирования, а цитирования — в видимость прогресса. Ни в какой момент петля не спрашивает, полезен ли результат кому-либо за её пределами. Этот вопрос, в строгом техническом смысле слова, выходит за рамки.

Я не первый, кто это замечает. И не последний. Это не критика — это описание. Петля производит реальные вещи: строгие методы, накопленное знание, подготовленных исследователей. Я не жалею о времени, проведённом внутри неё. Но в какой-то момент я всё же заметил, что нахожусь в ней.

Уход не был драматическим решением. Это было медленное осознание того, что вещь, которую я на самом деле хотел создать, не требует разрешения от комитета. Что люди, которых я хотел достичь, — практики, студенты, инженеры, принимающие реальные решения о реальных системах, — не ждут формально заверенной диссертации. Что самое честное, что я могу сделать с тем, что знаю, — это записать это в форме, которую можно использовать.

Что я и сделал.

Это руководство — не диссертация. У него нет институциональной принадлежности, нет упоминания финансирования, нет одобрения этического комитета, нет экзамена. Оно не будет учтено ни в одной метрике, которую академия использует для измерения производства знаний.

Оно также — и я говорю это без ложной скромности, поскольку ложная скромность была бы своего рода нечестностью — более полное, более актуальное, более читабельное и более полезное, чем была бы диссертация.

Я — человек в петле. Я был в петле аспирантуры и вышел из неё. Сейчас я в петле написания о петлях. Вы — в петле чтения об этом.

Никто из нас не выбирал эти петли полностью свободно. Но иногда мы можем выбрать — быть честными в отношении того, в каких петлях мы находимся, и зачем, и производят

ли они что-то стоящее.

Эта книга — мой ответ на эти вопросы.

Это не заявление об уходе. Увольнение предполагает работу, а работа предполагает, что кто-то другой определял условия. Это нечто иное: запись того, что я выбрал сделать со временем, и тихий аргумент в пользу того, что выбор был правильным.

Вы можете судить об этом сами. В конце концов, вы — человек в петле.

— *Автор*